



UNIVERSITY OF
LEICESTER

**School of Computing and
Mathematical Sciences
CO7201 Individual Project**

Final Report

**Automated Validation of GPT-Based Tax Query
Responses through Graph-Based Reasoning**

**Muhammad Saad
Ms1384@student.le.ac.uk
249046340**

**Project Supervisor: Prof. Reiko Heckel
Second Marker: Tola Olowosule**

**Word Count: 13500
05/09/2025**

DECLARATION

All sentences or passages quoted in this report, or computer code of any form whatsoever used and/or submitted at any stages, which are taken from other people's work have been specifically acknowledged by clear citation of the source, specifying author, work, date and page(s). Any part of my own written work, or software coding, which is substantially based upon other people's work, is duly accompanied by clear citation of the source, specifying author, work, date and page(s). I understand that failure to do this amounts to plagiarism and will be considered grounds for failure in this module and the degree examination as a whole.

Name: Muhammad Saad

Date: 05/09/2025

ACKNOWLEDGEMENT

I would like to express my deepest gratitude to my supervisor, Professor Reiko Heckel, for his exceptional guidance and unwavering support throughout the course of this project. He guided me at each and every stage, generously giving his time to help with my questions and challenges. His detailed feedback, often delivered through thoughtful emails, provided clarity and direction, greatly improving the quality of my work. His expertise and mentorship played a vital role in shaping both the direction and the outcome of this project.

I am also sincerely thankful to my second marker, Dr. Tola Olowosule, for conducting a constructive mock interview. That experience challenged me and motivated me to focus on the user interface (UI) aspects of the project. The insights and feedback from that session encouraged me to refine the UI design, significantly enhancing the user experience in the final deliverable.

I would like to extend my thanks to my department—the School of Computing and Mathematical Science as well as the CMS administration and the Library for providing the ambience and support that enabled me to work effectively on my dissertation and complete it on time.

I would like to acknowledge my friend, Neel Vinod Lad, for his consistent encouragement and motivation throughout this journey. His unwavering support and positive spirit helped me persevere through challenging times and stay focused on my goals. I am truly grateful for his presence and for continuously pushing me to do my best.

Finally, I wish to express my heartfelt gratitude to my parents and to all of my professors for their unwavering support and guidance throughout my academic journey. I firmly believe that I would not be where I am today without their encouragement, wisdom, and belief in me.

Thank you, everyone.

ABSTRACT

Large Language Models (LLMs) can explain tax concepts fluently but provide no formal guarantees that their answers comply with statutory rules. This project develops a validation layer under GPT-based tax responses by encoding UK personal-tax regulations as executable graph-transformation rules and checking LLM outputs against these rules. Natural-language rules are formalised into a JSON schema, visualised with UML, and converted to GROOVE's native rule format for execution. Given a user query, the system extracts key tax parameters from the GPT answer, independently computes outcomes via graph rules, and then compares the two, returning visual diffs and targeted feedback. A prototype covering a curated set of income, allowance, and band-assignment rules demonstrates feasibility: rule applications render as coloured transformations that make created, updated, and preserved elements explicit, and preliminary scenario tests show the validator can flag inconsistencies in bracket selection and allowance treatment while confirming correct cases. The approach combines the transparency and determinism of graph transformation with the reach of LLMs, supporting explainable, auditable tax advice. Artifacts include the JSON rule set and schema, GROOVE rules, automated diagram generation scripts, and a web front end for submitting queries and inspecting validations. This work lays the foundation for tighter LLM-to-law alignment, with future extensions to broader rule coverage, automated NL $\rightarrow$ rule conversion, and quantitative evaluation on larger test suites.

Contents

1	Introduction	9
2	Motivation and Background	9
2.1	Motivation	9
2.2	Graphs and their Applications.....	11
2.3	Graph Rewriting.....	12
3	Research Requirements & Scope	13
3.1	Research Questions	13
3.2	Case Studies and Evidence Plan.....	13
4	System Overview & Architecture	15
4.1	Problem & Scope.....	15
4.2	End-to-End Pipeline.....	16
4.3	Domain Model Snapshot (Type Graph)	17
4.4	Components	17
4.5	Execution Sequence (Calculate & Validate).....	19
4.6	Non-functional Goals, Risks & Assumptions	19
5	Provided Artefacts & Inputs	20
5.1	Prompt Design & Generation Strategy	20
5.2	Natural-Language Rule Set (Curated)	21
5.3	JSON Rule Specification	24
5.4	UML Artefacts (Generated).....	29
5.5	Groove Artefacts (GXL & GUI Previews)	32
6	Implementation.....	42
6.1	JSON → PlantUML Codegen (Class Diagram / Start Graph / Rules).....	42
6.2	JSON → GROOVE (GXL) Converter	46
6.3	GROOVE Control Programs — Structure & Simulator Usage.....	52
6.4	Backend API (FastAPI) — Orchestration, Excel→Control (Bulk Generation) & AI Tax Adviser	55
6.5	Frontend UI (Angular)	63
7	Experimental Methodology	69
7.1	Test Case Design (15 scenarios, coverage rationale).....	69
7.2	Execution Procedure	72
7.3	Metrics (Agreement with GROOVE Baseline).....	77
7.4	Critical Review.....	82
8	Recommendations and Further Research	83
9	Conclusion	86

Table of Figures

Figure 1 End-to-End Pipeline	16
Figure 2 Domain Type-Graph Snapshot	17
Figure 3 C4 Container Diagram	18
Figure 4 Sequence: “Calculate & Validate”	19
Figure 5 JSON Structure	24
Figure 6 Compact example (creation): creates and links a self-employment income with netting.	26
Figure 7 consume one unprocessed employment source exactly once and accumulate into total_income.....	26
Figure 8 Data Types and Operation Syntax for GROOVE [14]	29
Figure 9 Class Diagram.....	30
Figure 10 Start Graph.....	30
Figure 11 Rule-assign Person Inputs	31
Figure 12 Rule Create self-employment income	31
Figure 13 Rule BASIC band increment.	32
Figure 14 Rule Personal Allowance taper (100k–125,140).....	32
Figure 15 type.gty (Tiny representative snippet)	34
Figure 16 start.gst (Tiny representative snippet)	36
Figure 17 CreateSelfEmploymentIncome.gpr (Tiny representative snippet)	37
Figure 18 rule10_.gpr (Tiny representative snippet)	37
Figure 19 rule22a_.gpr (Tiny representative snippet).....	38
Figure 20 Type Graph in groove interface	38
Figure 21 Strat Graph in groove interface	39
Figure 22 Input pram Rule in groove interface	39
Figure 23 Create Self Employment Income in groove interface	40
Figure 24 Rule 10 Personal Allowance in groove interface	40
Figure 25 Rule 22a BASIC Band Increment in groove interface	41
Figure 26 Helper Function for mapping types into UML	43
Figure 27 Helper Function for creating tax bracket and write associations.....	43
Figure 28 Helper function to set defaults for Start Graph.....	44
Figure 29 Helper function to render Person Object	44
Figure 30 Generated Example.....	45
Figure 31 Rule loader with stable numeric/lexicographic ordering.....	45
Figure 32 Derives function-style signatures from ordered parameters	45
Figure 33 Parses application conditions.....	46
Figure 34 Emits test: guards and set: update statements.....	46
Figure 35 Renders edges: solid EarnsFrom, green create FallsIn	46
Figure 36 Start-graph function signature and knobs (role, version, defaults, layouts)	47
Figure 37 Graph root with \$version="curly" and deterministic flags.....	48
Figure 38 Self-loops: type:... and let:... seeds for floats/booleans.	48
Figure 39 Stable XML header and indentation for GROOVE.....	48
Figure 40 Stable node IDs and optional per-node layout.....	49

Figure 41 Attribute kinds become real/bool: loops in the type graph.....	49
Figure 42 Declarative edge types (e.g., EarnsFrom).....	49
Figure 43 Parameter encoding	50
Figure 44 new: marks freshly created nodes in the rule.	50
Figure 45 Recursive expression normalisation for let: updates.	51
Figure 46 applicationConditions → test	51
Figure 47 Distinguish new structural links with new	51
Figure 48 name=7 → id="rule07" for tidy file naming	51
Figure 49 Control program for TC1 (Simulator screenshot)	54
Figure 50 Button for Enabling and Disabling Control Files	55
Figure 51 Form params — /generate-controls/save-and-run	56
Figure 52 Response JSON shape — /generate-controls/save-and-run	57
Figure 53 Core pipeline (save→run→parse) — FastAPI handler	57
Figure 54 Form params — /generate-controls/generate-controls-only	58
Figure 55 Response JSON shape — /generate-controls/generate-controls-only	58
Figure 56 AI Tax Adviser — request model.....	59
Figure 57 AI Tax Adviser — response model	59
Figure 58 Transcript handling — last User: turn extractor	59
Figure 59 Mode switch — JSON task detection markers	60
Figure 60 OpenAI call params — /gpt-search	60
Figure 61 Response JSON shape — /gpt-search	61
Figure 62 .gcp control template — income + inputs block.....	61
Figure 63 Generator CLI — Generator.jar state/linear invocation	62
Figure 64 Java launch snippet — subprocess with timeout & errors.....	62
Figure 65 GXL parser — extract let:* on Person	62
Figure 66 Toggleable chips drive conditional field visibility and include ARIA states for screen readers.	64
Figure 67 Component state controls visibility; hidden fields are zeroed to keep payloads clean. .	64
Figure 68 Minimal, explicit constraints; human-readable labels on error.	65
Figure 69 Domain-aware ordering + currency formatting.....	65
Figure 70 Mode-aware prompts; model name sourced from backend.....	66
Figure 71 Keeps composer compact; adds scroll after 10 lines.	66
Figure 72 Header + income chips; results panel ready (empty).	67
Figure 73 Right-pane table shows populated metrics	68
Figure 74 Same results with advisory chat on the right	68
Figure 75 Inputs for 15 test scenarios (schema + values)	69
Figure 76 Ground-truth outputs for each input (ANI, PA, taxable income, tax).	70
Figure 77 One-line scenario intent per CaseID (purpose/rationale).	71
Figure 78 Coverage matrix showing which features/boundaries each case hits.....	71
Figure 79 Router delegates to the AI core in a worker thread; response body is domain-only.	73
Figure 80 The AI is boxed into a deterministic output schema; any missing keys are post-filled with zeros.	73
Figure 81 Enforces JSON; gracefully retries if the model rejects the format.	73
Figure 82 Body is domain-only; all AI diagnostics live in _meta	73

Figure 83 “Lite spec” with explicit thresholds; trading losses floored; blind allowance reserved (not applied in Model A)	74
Figure 84 Mirrors your rule text and thresholds exactly (PA taper, blind allowance, savings starting rate, etc.).....	74
Figure 85 Single ExperimentalUser Interface	77
Figure 86 MAE (tax): B £26.67; A £500.20; AI £2,015.13.....	78
Figure 87 Total error: AI £30,227; A £7,503; B £400.....	79
Figure 88 Per-case errors: AI peaks TC6, TC10; A moderate; B ~0 (TC3, TC4).	80
Figure 89 TC1 UI: all models tax £3,486.	81
Figure 90 TC2 UI: AI ANI £50k (others £40k); tax £7,486 (others £5,486).....	81

1 Introduction

Large Language Models (LLMs) are increasingly used to answer domain questions that require both factual knowledge and reasoning. In high-stakes domains like taxation, however, fluency does not imply correctness or legal compliance. UK personal tax calculation involves layered rules (allowances, reliefs, thresholds, and bands) that interact in subtle ways; an apparently plausible explanation can still violate statutory logic. This project investigates a hybrid approach that combines:

- The expressiveness and accessibility of LLMs for natural-language interaction, with
- The determinism, auditability, and explainability of graph transformation over a typed state model.

The core idea is to encode a curated subset of UK personal tax rules as typed graph transformation rules (executed in GROOVE), and use them as a ground-truth validator for LLM-produced calculations. The system compares GPT’s output with an independently computed, rule-driven baseline and returns visual, step-by-step diffs that make disagreements, and their causes, transparent.

Recent evaluations in tax law show that retrieval-augmented generation (RAG) over controlling legal texts materially improves answer accuracy and that performance generally rises with model scale, yet LLMs remain opaque, and benchmark scores can be inflated by data-contamination effects. Nay et al. evaluate thousands of U.S. tax questions, retrieving from the U.S. Code and CFR, and report significant gains when supplying governing texts, especially for larger models. At the same time, work on contamination cautions that leaked or overlapping evaluation data can misstate true capability. Building on these insights, to keep the LLM for natural interaction but enforce compliance via a deterministic, typed-graph calculator in GROOVE, then surface discrepancies through step-by-step visual diffs [1][3].

2 Motivation and Background

2.1 Motivation

Tax computation is a rule-dense domain where small wording or ordering differences (e.g., how to taper the Personal Allowance above £100,000, how to define *adjusted net income*, or how to apportion income across bands) materially change outcomes. In the UK, the tapering of the Personal Allowance after £100,000 and band interactions exemplify how minor sequencing differences can produce large effective marginal rate spikes, motivating careful, step-wise computation grounded in statute and HMRC practice [4][5].

At the same time, LLMs are increasingly used in legal and tax tasks, but remain opaque: performance can improve when retrieval brings the governing legal text into context (RAG), yet models still hallucinate and may benefit from inadvertent benchmark contamination. Empirical work in U.S. tax shows that providing the *actual controlling texts* (Code + Regulations) materially boosts accuracy, especially for larger models, while warning that black-box reasoning and contamination can inflate headline scores. This supports pairing natural-language interfaces with independent, law-grounded verification rather than trusting LLM outputs alone [1][2].

Beyond accuracy, governance and auditability are first-class requirements in high-stakes finance/tax contexts. Regulatory trends emphasize transparency, documentation, and control over model-based decisions (e.g., PRA SS1/23 on model risk management; the EU AI Act’s transparency and risk-management obligations for higher-risk uses). A system that produces deterministic, replayable computations with provenance is better aligned with these expectations than a purely generative approach [6][7].

These pressures motivate our hybrid architecture: a conversational LLM front end for accessibility and intent capture, combined with a typed-graph rules engine that computes a canonical baseline and records each transformation as an explicit, inspectable step. Graph transformation tools like GROOVE support graph-based modeling, rule application traces, and model checking, features that naturally deliver determinism, explainability, and maintainability as tax thresholds and reliefs evolve [8].

Concretely, the project addresses practitioner needs:

- **Determinism & repeatability:** identical inputs must yield identical outputs; rule application order is explicit and testable [6].
- **Traceability:** every numeric change is attributable to a *named rule* and *triggering condition*, with citations back to source provisions [8].
- **Explainability:** step-wise visual diffs expose what changed, when, and why—bridging stakeholder communication and audit trails [8].
- **Maintainability:** annual rate/threshold updates become declarative edits to a rule set, avoiding re-engineering and reducing regression risk [8].

By automating the pipeline from natural language → formal rules → executable artifacts, the project reduces manual hand-offs, minimizes contamination by ad-hoc fixes, and supports rigorous, year-over-year compliance in UK personal tax computation.

2.2 Graphs and their Applications

A graph models entities as nodes and relationships as edges; typed (attributed) graphs additionally constrain nodes/edges to declared types with attributes and well-formedness conditions. In graph transformation, computation proceeds by applying rules (LHS→RHS) to a host graph via matches, yielding a graph transition system that captures all reachable states and their provenance. This affords precise state modelling, stepwise change, and analysis by model checking [10][11].

Typed graphs are widely used in model-driven engineering, program/model transformation, protocol/system verification, and configuration management because they support: (i) local, composable transformations with algebraic semantics (e.g., DPO); (ii) explicit create/preserve/delete/update effects; and (iii) human-readable visualisations that double as executable specifications. Mature toolchains such as GROOVE and Henshin provide visual rule editors, traceable executions, and analysis back-ends [10][12][13].

Fit for UK personal tax. The type of graph includes core node types (e.g., Person, IncomeSource, TaxBand) and relation types such as `earnsFrom` (Person→IncomeSource) and `subjectToBand` (IncomeSource→TaxBand). Attributes (e.g., revenue, expenses, net_income, personal_allowance) live on the corresponding nodes. Tax bands are pre-defined nodes (Savings, Basic, Higher, Additional) present in the type of graph; rules help to *select/link* to and create bands dynamically. This mirrors statutory reality (bands are fixed by law for a tax year), simplifies invariants, and makes band selection traceable for audit.

Why typed graphs help this project

- **Precise state model.** The start graph encodes taxpayer state and year-specific parameters; types and attributes constrain admissible structures (e.g., one PersonalAllowance per Person). Algebraic application conditions guard rule firing (e.g., taper only if adjusted net income > threshold).
- **Local, composable rules.** Each tax step (accumulate incomes, apply allowances/reliefs, assign bands, compute liabilities) is a small rule; composition yields the statutory sequence while keeping each step explainable and testable
- **Traceability & explainability.** GROOVE records the rule application trace; each numeric change is attributable to a *named rule* and *match context*, and the resulting visual diffs expose what changed, when, and why [8].
- **Validation & exploration.** The generated graph transition system enables reachability queries (e.g., “can net income be negative given constraints?”), invariant checking, and regression tests across tax-year parameter sets.

- **Maintainability.** Annual updates are declarative edits to the start graph (thresholds/rates) and, when rules change, local rule revisions, minimising re-engineering and reducing regression risk.

Applications most relevant here

- (i) Executable domain semantics: rules encode the statutory tax sequence over a typed state;
- (ii) Traceable transformations: every create/preserve/delete/update is explicit in the rule and captured in the trace;
- (iii) Validation of conditional updates: application conditions and negative application conditions neatly encode tests like eligibility, tapers(relief/allowance), and band assignment, precisely what a tax calculator requires.

2.3 Graph Rewriting

Graph rewriting (graph transformation) specifies system behaviour by rules. Each rule matches a pattern in the current graph, the left-hand side (LHS), optionally guarded by attribute predicates and application conditions (including negative application conditions, NACs), and replaces it with the right-hand side (RHS) by creating, preserving, deleting, or updating nodes, edges, and attributes. Algebraic approaches (DPO/SPO) give these steps precise semantics over typed, attributed graphs and yield graph transition systems that record all reachable states [12].

Key concepts used here

- **Create / Preserve / Delete markers.** Rules make structural effects explicit (e.g., preserve IncomeSource, update its net_income, create an edge to TaxBand).
- **Attribute guards & application conditions.** Predicates such as `expenses > 0` (guards) and NACs (forbidden sub-matches) constrain applicability and prevent illegal rewrites.
- **Deterministic sequencing.** When statute demands order (e.g., compute net_income before band assignment), to enforce it via rule priorities or a controller, in GROOVE, global priorities and control programs are supported and used in practice to shape control flow.
- **Visual execution traces.** Each applied rule yields a step in the trace; changed elements are highlighted, enabling audit and explanation. GROOVE's simulator generates state spaces and supports trace visualisation/model checking.

Tooling. The ideas in GROOVE, which applies rules over (typed) graphs, generates state spaces, and serialises grammars in GXL, an XML sublanguage for graph exchange; this enables programmatic rule generation and reproducible runs [14].

Workflow. (i) model the **type graph** (taxpayer state, bands, thresholds); and **start graph** (ii) encode curated NL rules as **JSON**; (iii) **auto-generate** GROOVE rules (GXL) from JSON; and (iv) execute to produce an **auditable calculation trace** aligned to statutory sequencing.

3 Research Requirements & Scope

3.1 Research Questions

This project frames its aims as five research questions and investigates them through targeted case studies and tooling experiments.

- **RQ1 – Representation Feasibility.** Can a practically useful subset of UK personal-tax rules be expressed as typed graph transformation rules in GROOVE without losing essential semantics (e.g., order of allowances, band selection, tapering conditions)?
- **RQ2 – Automation Fidelity.** To what extent can a system automate NL→JSON→UML/GROOVE generation from a single JSON source while keeping artifacts syntactically correct, semantically faithful, and human-reviewable?
- **RQ3 – Agreement & Divergence Detection.** Given a free-form user query, can a comparison layer reliably detect, localise, and explain disagreements between a GPT calculation and the GROOVE baseline, using step-by-step visual diffs of rule applications?
- **RQ4 – Testability & Control.** Can an Excel-driven controller generator execute deterministic rule sequences across scenarios, enabling repeatable evaluation and regression testing with minimal manual setup?
- **RQ5 – Maintainability & Evolution.** How easily can thresholds/rates and new rules be introduced and the entire pipeline regenerated (UML + GROOVE + tests + UI), and what is the residual manual effort to keep artifacts in sync across tax-year changes?

3.2 Case Studies and Evidence Plan

RQ1. The study begins with a minimal rule set (e.g., self-employment net_income with/without expenses) and then extends to employment, savings interest, dividends, and pre-defined TaxBand nodes that are selected and created by rules. Correctness is judged from rule execution traces in GROOVE (Section 5).

RQ2. The pipeline generates type/start graphs and rule diagrams in UML alongside GROOVE XML. Validation covers both the generator’s output (schema checks) and GROOVE’s acceptance/execution (Section 6).

RQ3. Approximately 15 test scenarios—spanning single/multiple income sources, allowance/taper edge cases (>£100k), pension/gift-aid relief flags, and dividend/savings

mixes, are executed. The agreement rate is logged, and discrepancies are classified (Section 7).

RQ4. Each test row in Excel is converted to a controller file and an initial graph, enforcing a consistent rule order (compute `net_income` → accumulate incomes → apply allowances → assign bands) and capturing reproducible results (Sections 6).

RQ5. A tax-year roll-over is simulated by changing parameterised thresholds and adding or editing a rule; regeneration time and any required hand edits are measured (Section 6).

4 System Overview & Architecture

This section presents a technology-agnostic overview of the system, the artefacts it produces, and the data/control flows between components. The aim is to let readers grasp the architecture at a glance before details of process and evaluation. The pipeline is intentionally linear—NL rules → JSON → visual UML → executable GROOVE grammars—so that every downstream artefact is generated from a single source of truth and remains reproducible across tax-year configurations.

4.1 Problem & Scope

Problem. Provide a reproducible, auditable UK personal-income-tax calculator that encodes a limited but representative subset of HMRC rules as executable graph transformations and exposes them via a modern UI. Multiple models (GROOVE, Python baselines, and an LLM-based calculator) to enable side-by-side validation can evaluate the same inputs.

In-scope (2025/26, England/Wales/NI-Excluded):

- **Income sources:** employment, self-employment, savings interest, dividends (other sources treated as out-of-scope).
- **Allowances & reliefs:** personal allowance (incl. taper), blind person’s allowance, pension relief (relief-at-source), gift-aid uplift, personal savings allowance, starting rate for savings, dividend allowance.
- **Banding & rates:** BASIC, HIGHER, ADDITIONAL bands for non-savings and corresponding dividend bands; starting-rate-for-savings window.
- **Validation & comparison:** deterministic replay of the same inputs across GROOVE, two Python models (Lite and Specific), and an LLM calculator; UI table compares results.

Out-of-scope:

- Scotland-specific income tax bands; CGT, NICs, student loans, marriage allowance transfer, child benefit HICBC, property income, complex relief interactions beyond the limited domain; multi-year carry rules.
- Production-grade security/compliance hardening (rate limits/PII redaction are basic only).

4.2 End-to-End Pipeline

High-level flow merges two user experiences: (a) **Calculator screen** invoking the GROOVE pipeline; (b) **Experimental screen** invoking **four calculators**: GROOVE, Python-Lite, Python-Specific, and LLM (OpenAI) for side-by-side comparison

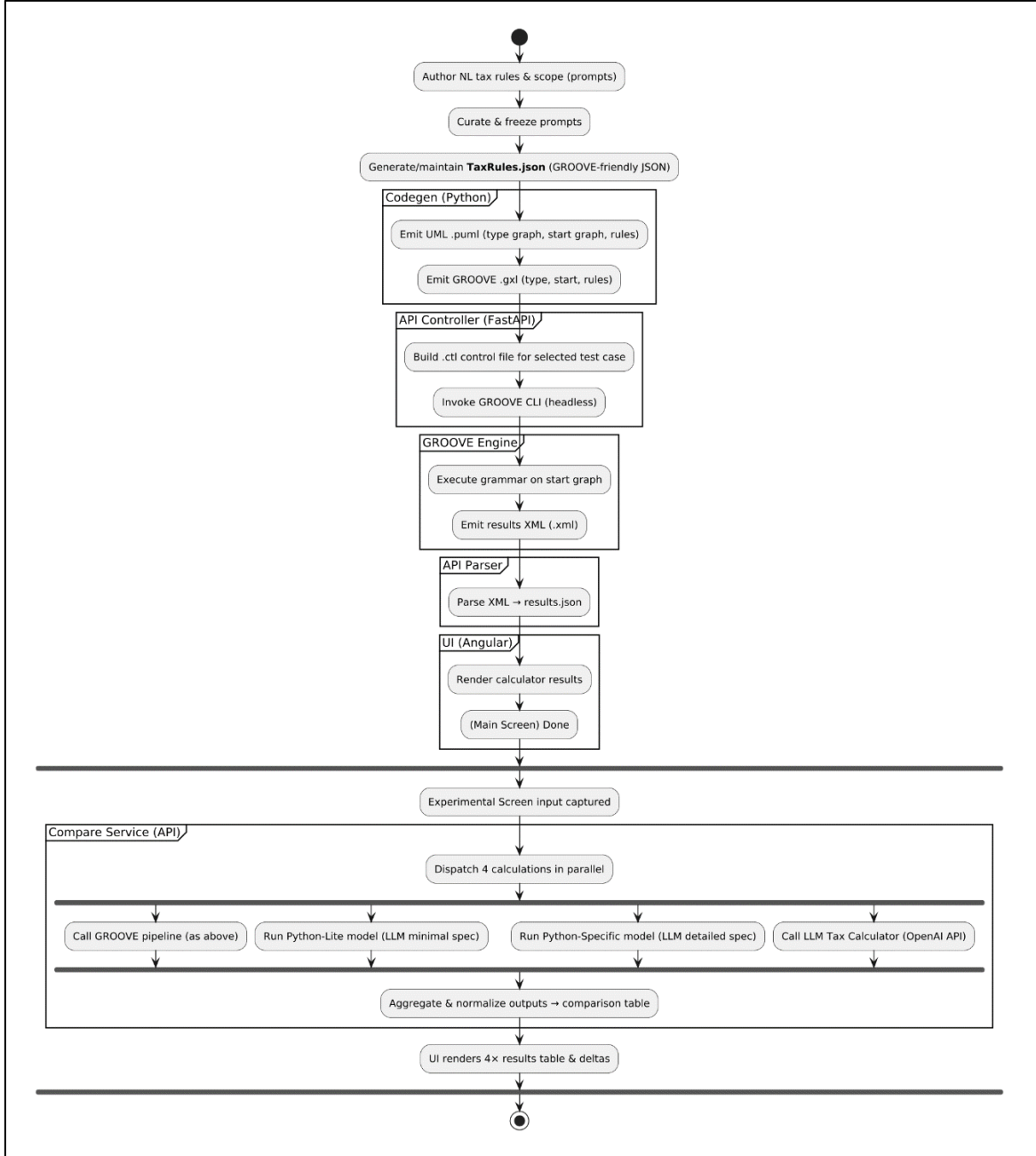


Figure 1 End-to-End Pipeline

Notes. The pipeline is deterministic for GROOVE and Python models; LLM path is non-deterministic but prompt + model IDs are recorded for traceability.

4.3 Domain Model Snapshot (Type Graph)

A compact view of the key node/edge types and core attributes used by rules. This is a **type graph** view rendered as a class diagram for readability.

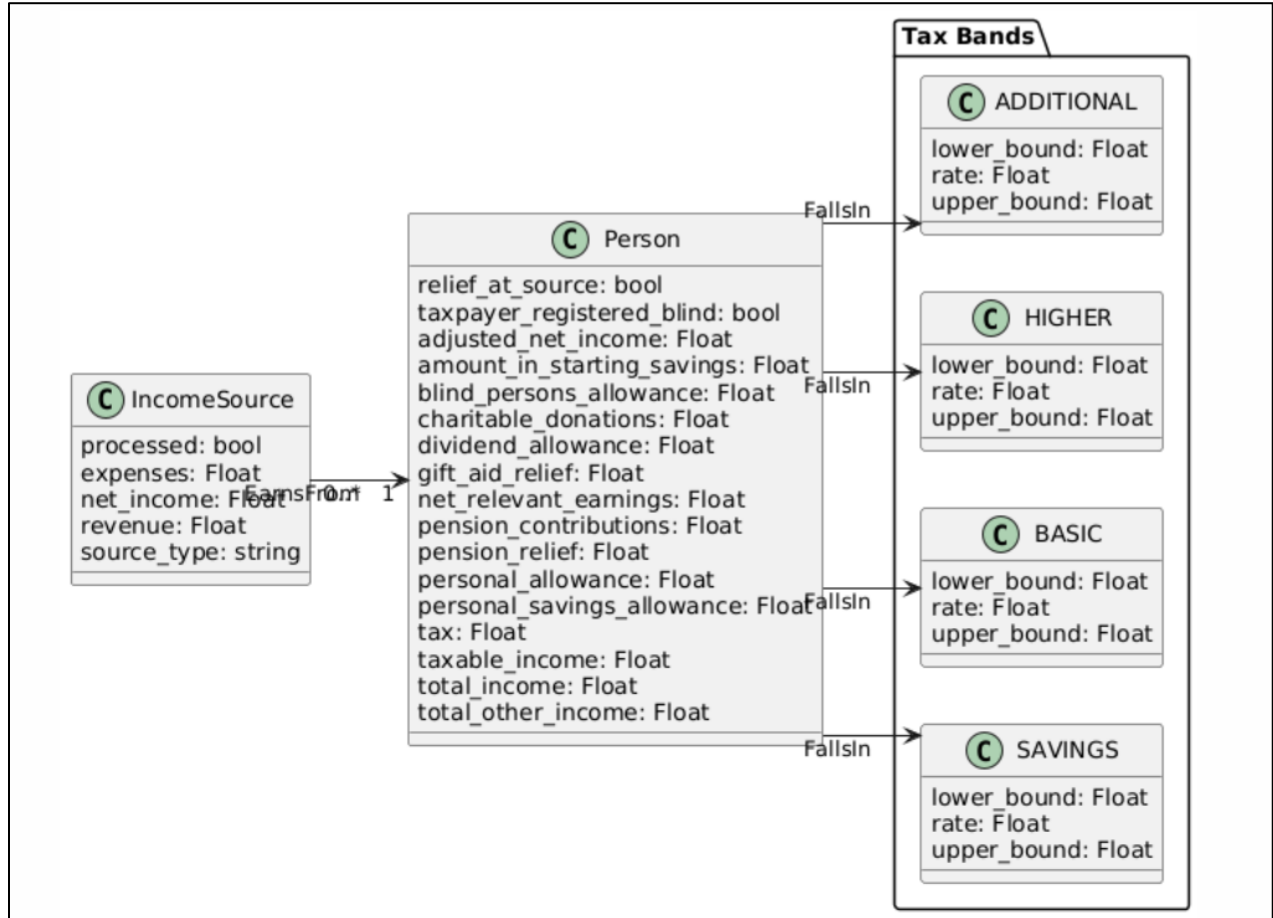


Figure 2 Domain Type-Graph Snapshot

4.4 Components

Containers:

- **UI (Angular, two screens)**. Collects inputs; invokes API; renders calculator view and the experimental 4× comparison table; embeds a chat assistant for guidance.
- **API (FastAPI)**. Orchestrates: builds control files, invokes GROOVE CLI, parses XML → JSON; exposes comparison endpoints; centralizes config.
- **Codegen (Python)**. Converts GROOVE-friendly JSON into **.puml** (type, start, rules) and **.gxl** (type, start, rules) artefacts.

- **Engine (GROOVE).** Source-of-truth executor of the tax rules as graph transformations; produces authoritative results XML.
- **Storage (Files/Excel).** Prompts, schemas, JSON rules, UML/GXL artefacts, control files, results (XML/JSON), and test-sheets.
- **LLM API (OpenAI).** Non-deterministic calculator used for research-only comparison.
- **Python Models.** Two baselines: **Lite** (minimal prompts), **Specific** (detailed prompts mirroring GROOVE domain).

Why GROOVE is the source of truth. GROOVE offers explicit, inspectable state transitions over a typed graph; each rule application is deterministic under fixed inputs and control, enabling explainability, replay, and fine-grained audits not available from black-box models.

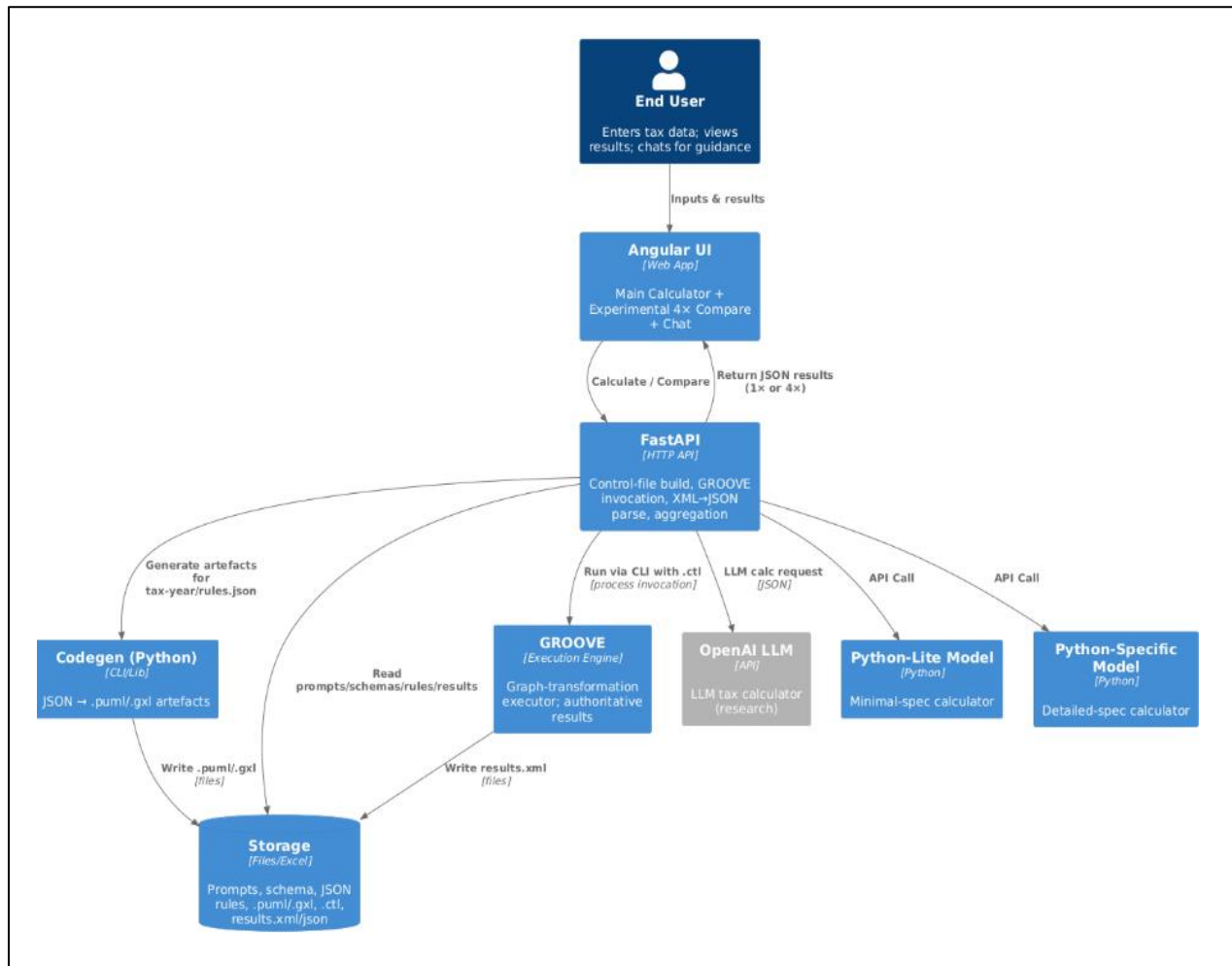


Figure 3 C4 Container Diagram

4.5 Execution Sequence (Calculate & Validate)

User triggers a calculation; API orchestrates GROOVE execution and (optionally) the three alternative calculators; UI renders comparison.

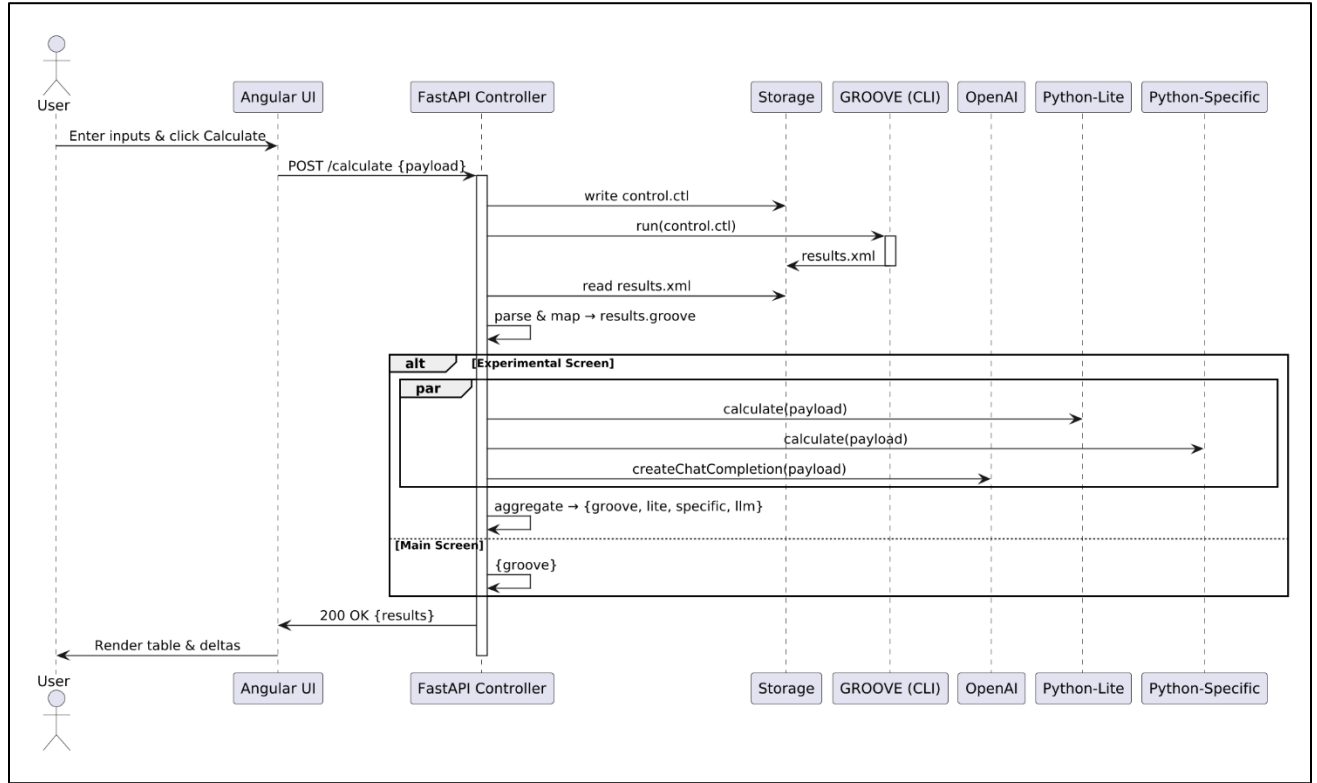


Figure 4 Sequence: "Calculate & Validate"

4.6 Non-functional Goals, Risks & Assumptions

- **Determinism & auditability.** GROOVE serves as the authoritative, inspectable executor. Rule applications can be stepped and explained.
- **Maintainability.** JSON→codegen ensures new rules or tax-year changes regenerate all artefacts consistently. Minimal hand-edits.
- **Performance.** Single-case runs complete in seconds on commodity hardware; batch runs are possible via controller.
- **Security & privacy.** Only the numeric tax inputs and boolean flags required to reproduce a calculation (e.g., incomes, contributions, relief switches) plus a `case_id`. Chat messages are not written to disk.
- **Risks/assumptions.** Limited tax domain; LLM calculators are non-deterministic and advisory-only; correctness relies on curated prompts and careful codegen.

5 Provided Artefacts & Inputs

5.1 Prompt Design & Generation Strategy

- **What is prompt engineering (in this project)?**

Prompt engineering is the deliberate design of inputs to an LLM so its outputs are (a) on-scope, (b) structurally consistent, and (c) directly usable by downstream tooling. In this project, the goal is to elicit clear, self-contained natural-language tax rules that can be transformed into formal artifacts (JSON → UML/GROOVE) without requiring additional interpretation. Concretely, done this by: (1) framing the task and role (“You are an HMRC-style rules editor...”) and stating success criteria; (2) constraining format with an explicit schema (rule IDs, if-then text, tax-year, citation fields) and validating with structured/JSON outputs; (3) using few-shot exemplars and counter-examples to anchor tone and structure; (4) decomposing generation into stages (scope selection → rule drafting → sanity checks), which is known to improve reliability for reasoning-heavy tasks; and (5) tightening scope iteratively (e.g., “2025/26 England/Wales/NI(excluded) income tax only; exclude Scotland; no invented thresholds”), so the rule set stays useful yet manageable. These practices follow widely cited prompt engineering guidance and research, which show that format constraints, exemplars, and staged prompting materially improve fidelity and downstream usability [9][15][16][17].

- **How were the prompts used (process)?**

A short, staged sequence of prompts was employed and refined over multiple iterations to converge on a stable natural-language (NL) ruleset. Each prompt served a distinct role in the pipeline—first validating the overall tax computation flow, then enforcing an actionable rule shape, then estimating required coverage and scoping the problem, and finally capturing rules in a tabular, implementation-ready form for conversion to JSON → UML/GROOVE.

Prompts used (verbatim)

Prompt 1 — Validate the overall computation flow

What is a general process to compute UK income tax. I was thinking along the following lines. Is this correct, or am I missing something? 1- calculate income 2- determine allowance 3- calculate deductions 4- compute overall tax to be paid

Prompt 2 — Enforce rule shape (if-then) for step 2

I would like to describe the steps in a rule-based way. Can you give me sample if-then rules for step 2? The if part should describe the precondition of the rule and the then part the action (e.g. add item of deduction).

Prompt 3 — Estimate coverage

How many such rules would I need to cover all of income tax?

Prompt 4 — Reduce scope sensibly

Can I limit the scope to cut down the number of rules, in a way that this is still useful for a broad range of cases?

Prompt 5 — Capture rules in a 2-column, implementation-ready form

Draft a spreadsheet or simple text file with two columns: Natural-language rule (fully self-contained). Rough sketch of nodes/attributes needed

Iteration effects (what changed and why it worked)

- **Task framing became concrete.** Beginning with a generic “process to compute tax” and then anchoring on a four-step pipeline reduced ambiguity for subsequent prompts and stabilized the downstream structure.
- **Structure was enforced early.** Insisting on explicit if–then units converted narrative guidance into precondition → action pairs that map cleanly to rule IDs, JSON fields, and transformation steps.
- **Scope was controlled deliberately.** Asking for rule counts exposed inherent complexity; the scoping prompt then narrowed the domain while retaining broad usefulness (e.g., focusing on a single tax year and jurisdiction).
- **Outputs became ingestion-ready.** The 2-column capture ensured each rule was self-contained and paired with a minimal nodes/attributes sketch, aligning directly with schema-validated JSON and later UML/GROOVE artefacts.

Outcome

Through progressive tightening of phrasing, early enforcement of if–then structure, and tabular capture, the process yielded a coherent NL ruleset that is mechanically transformable into formal artefacts used later in the pipeline.

5.2 Natural-Language Rule Set (Curated)

This section presents a concise, representative subset of the full natural-language (NL) rules. Each rule is stated in an if–then form (precondition → action) and is followed by the nodes/attributes it touches to enable direct traceability into JSON and GROOVE artefacts. The scope is England, Wales and Northern Ireland for tax year 2025/26. The complete rule list is provided in the appendix/materials.

How to read these

- **Rule:** executable NL statement (if-then).
- **Nodes / Attributes:** the exact fields referenced by the rule.
- **Notes:** context and interplay with other rules; authoritative references where thresholds are statutory.

1. Income Identification (Self-Employment focus)

Rule. IF a person has income from self-employment THEN add that person's net self-employment income to total_income.

Nodes / Attributes. IncomeSource.source_type (= self_employment), IncomeSource.net_income, Person.total_income.

Notes. Netting of revenue and allowable expenses is handled separately (see B.7). HMRC guidance treats taxable profit as turnover minus allowable expenses; this rule ensures only net amounts flow into total_income [18].

2 Personal Allowance (Standard & Taper)

Rule (standard). IF $\text{adjusted_net_income} \leq £100,000$ THEN set personal_allowance = £12,570.

Rule (taper). IF $£100,000 < \text{adjusted_net_income} < £125,140$ THEN set $\text{personal_allowance} = 12,570 - \text{floor}((\text{adjusted_net_income} - 100,000) \div 2)$; otherwise leave unchanged here.

Nodes / Attributes. Person.adjusted_net_income, Person.personal_allowance.

Notes. For 2025/26 the standard Personal Allowance remains £12,570. The taper reduces the allowance by £1 for every £2 of income over £100,000, with the allowance fully extinguished at £125,140 [19][20].

3 Blind Person's Allowance

Rule. IF taxpayer_registered_blind THEN set blind_persons_allowance = £3,130 (tax year 2025/26).

Nodes / Attributes. Person.taxpayer_registered_blind, Person.blind_persons_allowance.

Notes. Blind Person's Allowance is added to the Personal Allowance; transfer provisions exist in statute but are enforced elsewhere in the full rule set. Amount updated to £3,130 for 2025/26 [21][22].

4 Pension Relief at Source (interaction with taxable income)

Rule. IF relief_at_source THEN reduce taxable_income by pension_contributions.

Nodes / Attributes. Person.relief_at_source, Person.pension_contributions, Person.taxable_income.

Notes. This rule captures an implementation choice in the model. Statutorily, relief at source gives basic-rate relief within the pension and any higher/additional-rate relief is obtained by the basic (and, if relevant, higher) rate band or by claim via Self Assessment—not by reducing taxable income per se. The model abstracts that effect at the income stage for determinism; band-extension can alternatively be encoded in control logic [23][24].

5 Band-Splitting (Generic Template)

Rule. IF taxable_income > band_lower_bound THEN increase tax by $(\min(\text{taxable_income}, \text{band_upper_bound}) - \text{band_lower_bound}) \times \text{band_rate}$.

Nodes / Attributes. Person.taxable_income, Person.tax, TaxBand.lower_bound, TaxBand.upper_bound, TaxBand.rate.

Notes. This reusable pattern is instantiated for BASIC, HIGHER, ADDITIONAL (and for dividend/savings bands where applicable) and is applied in sequence by the control file. For 2025/26, England/Wales/Ni bands align with HMRC tables (basic-rate slice up to £37,700 after allowance; higher-rate to £125,140; additional over £125,140) [25].

6 Personal Savings Allowance (by taxpayer rate)

Rule (basic rate). IF taxpayer_rate = basic THEN set personal_savings_allowance = £1,000.

Rule (higher rate). IF taxpayer_rate = higher THEN set personal_savings_allowance = £500.

Rule (additional rate). IF taxpayer_rate = additional THEN set personal_savings_allowance = £0.

Nodes / Attributes. Person.taxpayer_rate, Person.personal_savings_allowance.

Notes. The allowance determined here is subsequently consumed during savings-income banding. HMRC defines PSA amounts as shown for the relevant taxpayer rate [26].

7 Expense-Netting (Self-Employed)

Rule. IF an IncomeSource is self_employment with expenses THEN set $\text{net_income} = (\text{revenue} - \text{expenses})$.

Nodes / Attributes. IncomeSource.source_type (= self_employment), IncomeSource.revenue, IncomeSource.expenses, IncomeSource.net_income.

Notes. Produces the `net_income` consumed by B.1. HMRC examples and help sheets describe computing taxable profit as turnover minus allowable expenses, with specific rules on what qualifies [27].

5.3 JSON Rule Specification

5.3.1 Top-level layout and intent

A single JSON container is used to hold all modelling artefacts needed for generation, execution, and comparison. Each top-level entry is optional and can be provided partially or fully, depending on the task (e.g., supply only `typeGraph` when stabilising the domain, or add rules and controls for full runs).

All identifiers (id, rule name, node/edge ids) must be unique within their scope.

```
json
{
  "typedGraphs": [ /* zero or more instance graphs */ ],
  "typeGraph": { /* exactly one domain model (optional) */ },
  "rules": [ /* zero or more rules */ ],
  "constraints": [ /* optional */ ],
  "ruleApplication": [ /* optional traces */ ],
  "controls": [ /* zero or more control programs */ ]
}
```

Figure 5 JSON Structure

5.3.2 Attributes and the Type Graph

The **Type Graph** fixes the vocabulary of node/edge types and attribute names/types that rules may reference. For this project the core `typeGraph` is `TaxTypeGraph`, whose nodes and attributes (excerpt) are:

- Person with numeric tax attributes and boolean flags:
 - `total_income`, `adjusted_net_income`, `personal_allowance`, `taxable_income`, `tax`,
 - `pension_contributions`, `net_relevant_earnings`, `pension_relief`,
 - `relief_at_source` (boolean), `taxpayer_registered_blind` (boolean),
 - `blind_persons_allowance`, `charitable_donations`, `gift_aid_relief`,
 - `personal_savings_allowance`, `dividend_allowance`,
 - `total_other_income`, `amount_in_starting_savings`.

- IncomeSource describing each income stream:
 - source_type (string), revenue, expenses, net_income, processed (boolean).
- Tax band node types with rate windows:
 - SAVINGS, BASIC, HIGHER, ADDITIONAL, each with lower_bound, upper_bound, rate.

Representative edges (with multiplicities) include:

- EarnsFrom (Person $\rightarrow$ IncomeSource) with 0..* targets.
- FallsIn (Person $\rightarrow$ each band type) with 0..1 per band.

Schema constraints. The type-graph JSON is validated against a schema derived from JSON Schema. Key checks:

- Required: id, nodes, edges.
- Each node requires type; optional attributes whose values are declared JSON base types (string, boolean, integer, number, array).
- Each edge requires type, source, target; optional multiplicity {lower, upper} and attributes.
- Source/target type names must exist among node types.

JSON note: attribute keys and string values must be quoted in valid JSON (e.g., "status", "amount").

5.3.3 Rule object shape and GROOVE-friendly conventions

A rule names a graph transformation typed over the same typeGraph. Nodes/edges carry an action and, where relevant, attribute update expressions. By adopting a small, explicit vocabulary so LLM outputs are parseable and executable without post-editing:

- typedOver: must equal the typeGraph.id ("TaxTypeGraph").
- parameters: ordered inputs with stable ids (p0, p1, ...) and parin indices used by GROOVE controllers.
- Node/edge action: one of
 - create | preserve | modify | delete (and forbid for NACs). (Email specification)
- Attribute updates use prefix operators to avoid ambiguity:

- set: <expr> assigns a value or expression (set: p0 - p1, set: total_income + is.revenue).
- Application conditions support simple propositional formulas over attribute predicates:
 - and(...), or(...), eq(lhs, rhs), etc., with optional named condition graphs for readability.
- NACs / nested conditions are expressed via action: "forbid" nodes/edges and/or named graphs under applicationConditions.

```

1  {
2    "name": "CreateSelfEmploymentIncome",
3    "typedOver": "TaxTypeGraph",
4    "parameters": [
5      {"id": "p0", "name": "revenue", "type": "real", "mode": "input", "parin": 0},
6      {"id": "p1", "name": "expenses", "type": "real", "mode": "input", "parin": 1}
7    ],
8    "nodes": [
9      {"id": "person", "type": "Person", "action": "preserve"},
10     {"id": "income", "type": "IncomeSource", "action": "create",
11       "attributes": {
12         "source_type": "set: 'SELF_EMPLOYMENT'",
13         "revenue": "set: p0",
14         "expenses": "set: p1",
15         "net_income": "set: p0 - p1",
16         "processed": "set: false"
17       }}
18     ],
19     "edges": [
20       {"id": "ef_income", "type": "EarnsFrom", "source": "person", "target": "income", "action": "create"}
21     ]
22   }

```

Figure 6 Compact example (creation): creates and links a self-employment income with netting.

```

1  {
2    "name": "1",
3    "typedOver": "TaxTypeGraph",
4    "nodes": [
5      {"id": "p", "type": "Person", "action": "preserve",
6        "attributes": { "total_income": "set: total_income + is.revenue" }},
7      {"id": "is", "type": "IncomeSource", "action": "preserve",
8        "attributes": { "processed": "set: true" }}
9    ],
10    "edges": [
11      {"id": "ef1", "type": "EarnsFrom", "source": "p", "target": "is", "action": "preserve"}
12    ],
13    "applicationConditions": {
14      "acName": "rule1Cond",
15      "formula": "and(eq(is.source_type, 'EMPLOYMENT'), eq(is.processed, false))"
16    }
17  }

```

Figure 7 consume one unprocessed employment source exactly once and accumulate into total_income

5.3.4 From NL Rules to JSON Rules (Prompted Generation)

The following is the verbatim prompt used to convert short natural-language (NL) tax rules into GROOVE-friendly JSON rule objects. In practice, one needs to paste one or more curated NL rules beneath this instruction; the model must return only a JSON array conforming to the schema and constraints below. This is not the exact prompt which was used in the generation of JSON as that is multiple prompts with instruction of Groove friendly syntax [14].

ROLE

You convert short natural-language (NL) tax rules into Groove-friendly JSON rule objects.

STRICT OUTPUT

- Return ONLY a JSON array of rule objects. No prose, no markdown.
- JSON must be valid and minifiable (UTF-8 quotes, no trailing commas).

TYPE ENVIRONMENT

- typedOver MUST be "TaxTypeGraph".
- Use only these nodes/edges/attributes unless the NL rule explicitly requires more:
 Nodes: Person (id p), IncomeSource (id is)
 Edge: EarnsFrom (id ef1; source p → target is)
 Person attributes: total_income
 IncomeSource attributes: source_type, processed, revenue, net_income
- When a source is consumed, always set is.processed = true (exactly once).

RULE OBJECT SCHEMA

Each rule object MUST have exactly these keys:

```
{
  "name": "<string id, e.g., '1'>",
  "description": "<plain English restatement>",
  "typedOver": "TaxTypeGraph",
  "nodes": [
    { "id": "<p|is|...>", "type": "<Person|IncomeSource|...>", "action": "<preserve|create|delete>",
    "attributes": { ... optional ... } }
  ],
  "edges": [
    { "id": "<ef1|...>", "type": "<EarnsFrom|...>", "source": "<node id>", "target": "<node id>",
    "action": "<preserve|create|delete>" }
  ],
  "applicationConditions": {
    "acName": "<stable id like ruleXCond>",
    "formula": "<boolean expression using allowed operator words>",
```

```

"graphs": [
  {
    "name": "<Ci>",
    "nodes": [
      { "id": "<node id>", "type": "<type>", "action": "preserve",
        "attributes": { "test": "<the same boolean expression or a logically equivalent subtest>"
      }
    ],
    "edges": []
  }
]

```

ATTRIBUTE UPDATES

- Mutations use the exact syntax: "set: <infix-expression>"
- Examples: "set: total_income + is.revenue", "set: true"

CONDITION LANGUAGE (operator words only)

- Boolean: and, or, not, eq
- Numeric compares: lt, le, gt, ge, eq
- Literals: true, false
- Strings: single quotes, e.g., 'EMPLOYMENT', 'SELF_EMPLOYMENT'
- Do NOT invent functions or operators beyond the above.

PATTERN RULES

- “has income from X” ⇒ require EarnsFrom(p→is) and a source_type check on is.
- Employment adds is.revenue; self-employment adds is.net_income.
- ids: p (Person), is (IncomeSource), ef1 (EarnsFrom). Reuse when possible.

VALIDATION

- No extra top-level keys.
- Every rule must include applicationConditions with the same predicate echoed in graphs[].nodes[].attributes.test.
- Keep actions consistent: typically "preserve" for p, is, ef1 in these rules.

Type	Op	Meaning
bool	and	Conjunction of two boolean values
	or	Disjunction of two boolean values
	not	Negation of a boolean value
	eq	Comparison of two boolean values
	true false	Boolean constant Boolean constant
int/real	add	Addition of two integer or real values
	sub	Subtraction of the second argument from the first
	mul	Multiplication of two integer or real values
	div	Integer (for int) or real (for real) division of the first argument by the second
	mod	Remainder after integer division (only for int)
	min	Minimum of two integer or real values
	max	Maximum of two integer or real values
	lt	Test if the first argument is smaller than the second
	le	Test if the first argument is smaller than or equal to the second
	gt	Test if the first argument is greater than the second
	ge	Test if the first argument is greater than or equal to the second
	eq	Comparison of two integer or real values
	neg	The negation of an integer or real value
	toString	Conversion of an integer or real value to a string
string	concat	Concatenation of two string values
	lt	Test if the first argument is lexicographically smaller than the second
	le	Test if the first argument is lexicographically smaller than or equal to the second
	gt	Test if the first argument is lexicographically greater than the second
	ge	Test if the first argument is lexicographically greater than or equal to the second
	eq	Comparison of two string values

Figure 8 Data Types and Operation Syntax for GROOVE [14]

5.4 UML Artefacts (Generated)

This section shows the UML diagrams generated from JSON the Class Diagram, the Start Graph, and selected rule diagrams. Colour and line styles encode rule semantics:

- Green arrow (edge) / green element = created by the rule (RHS only).
- Black arrow (edge) / black element = preserved (it already exists on the LHS and remains after the rule).
- Attribute updates are shown inside the object box as attribute := <expression>.
- Rule input parameters appear as p0, p1, ... and are bound by the controller at run time

All figures below were exported from the codegen and can be reproduced from the accompanying JSON.

5.4.1 Class Diagram (UML)

Purpose. Defines the *vocabulary* of the domain: node types, attributes, and associations.

What the figure shows.

Person carries the numeric/boolean tax attributes; IncomeSource carries source metadata. The association EarnsFrom (Person $\rightarrow$ IncomeSource) types the link used by accumulation rules. Attribute types follow the JSON typing (Real, Boolean, string) used in generation.

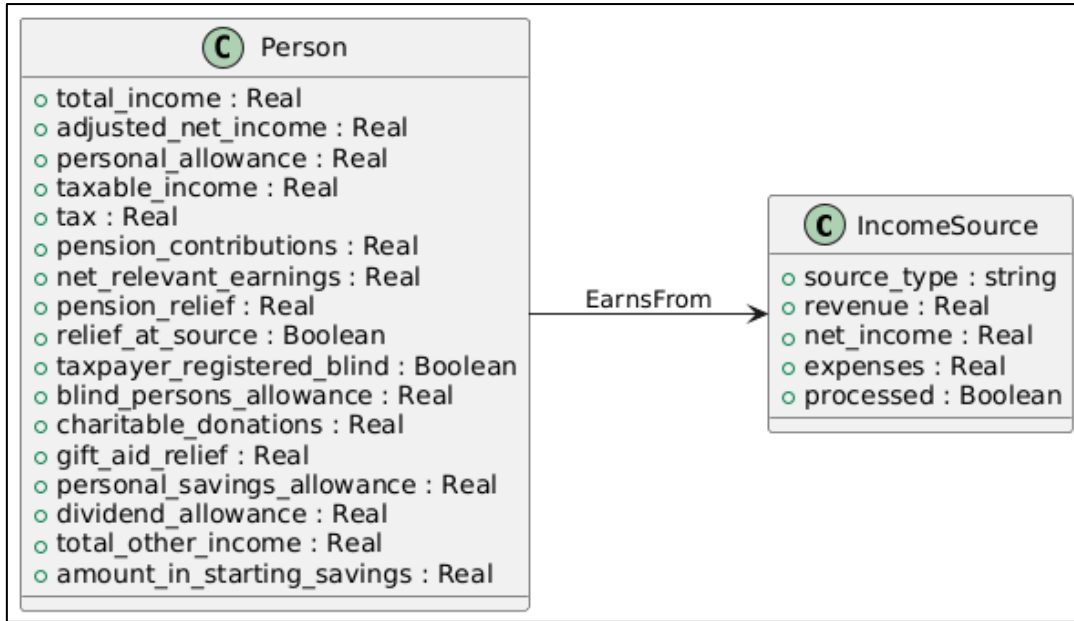


Figure 9 Class Diagram

5.4.2 Start Graph (Object Diagram)

Purpose. The initial state before any rules fire.

What the figure shows.

A single **Person** object with default values consistent with the 2025/26 scope. In our baseline, numeric fields are 0.0 except `personal_allowance` = 12570.0; flags start false (e.g., `relief_at_source`=false, `taxpayer_registered_blind`=false). Scenario-specific values are supplied later via the controller.

Person	
<code>adjusted_net_income</code>	= 0.0
<code>amount_in_starting_savings</code>	= 0.0
<code>blind_persons_allowance</code>	= 0.0
<code>charitable_donations</code>	= 0.0
<code>dividend_allowance</code>	= 0.0
<code>gift_aid_relief</code>	= 0.0
<code>net_relevant_earnings</code>	= 0.0
<code>pension_contributions</code>	= 0.0
<code>pension_relief</code>	= 0.0
<code>personal_allowance</code>	= 12570.0
<code>personal_savings_allowance</code>	= 0.0
<code>relief_at_source</code>	= false
<code>tax</code>	= 0.0
<code>taxable_income</code>	= 0.0
<code>taxpayer_registered_blind</code>	= false
<code>total_income</code>	= 0.0
<code>total_other_income</code>	= 0.0

Figure 10 Start Graph

5.4.3 Rule Diagrams (selected rules with pre/post-state)

Green = created; **black** = preserved; updates shown as $:=$. Parameters $p_0, p_1, \dots$ are controller inputs.

(a) Assign input parameters on Person.

Binds controller inputs to attributes on an existing person.

- **Parameters:** p_0 (pension_contributions), p_1 (charitable_donations), p_2 (relief_at_source), p_3 (taxpayer_registered_blind).
- **Effect:** $\text{person.pension_contributions} := p_0$, $\text{person.charitable_donations} := p_1$, $\text{person.relief_at_source} := p_2$, $\text{person.taxpayer_registered_blind} := p_3$.

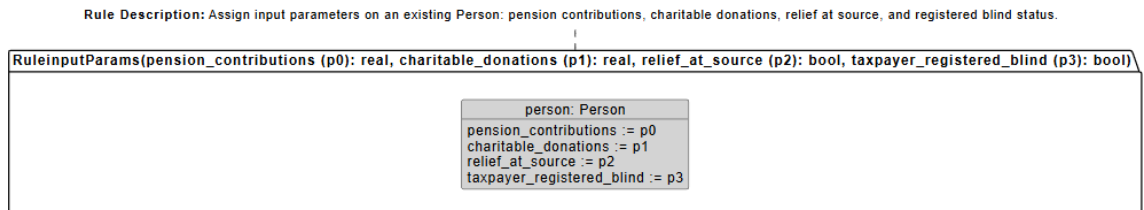


Figure 11 Rule-assign Person Inputs

(b) Create self-employment income and link it.

Constructs a new IncomeSource and links it to the person.

- **Parameters:** p_0 (revenue), p_1 (expenses).
- **Green edge EarnsFrom:** created link from person to the new income.
- **Effect on income:**
 $\text{source_type} := \text{'SELF_EMPLOYMENT'}$,
 $\text{revenue} := p_0$, $\text{expenses} := p_1$,
 $\text{net_income} := p_0 - p_1$,
 $\text{processed} := \text{false}$.

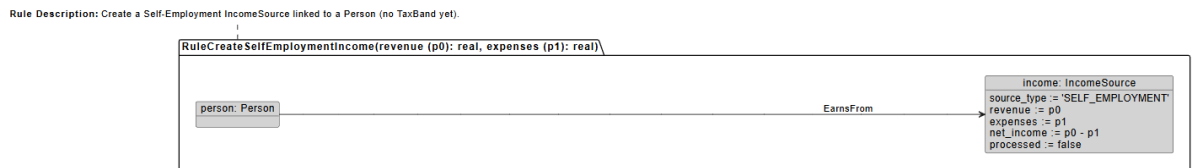


Figure 12 Rule Create self-employment income

(c) Apply BASIC band tax (example).

Illustrates an arithmetic update over an existing band link (shown **black** here if preserved; **green** if created by this rule in your variant).

- **Effect:**

$p.\text{tax} := \text{add}(p.\text{tax}, (\min(p.\text{taxable_income} - p.\text{amount_in_starting_savings}, b.\text{upper_bound}) - b.\text{lower_bound}) * b.\text{rate}).$

Rule Description: For each Person and linked BASIC band, increment p.tax by $(\min(p.\text{taxable_income} - p.\text{amount_in_starting_savings}, b.\text{upper_bound}) - b.\text{lower_bound}) * b.\text{rate}.$

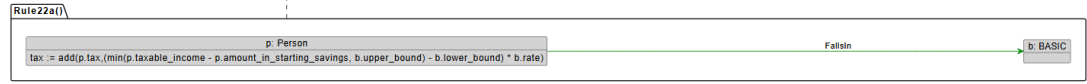


Figure 13 Rule BASIC band increment.

Fig 13:

(d) Personal Allowance taper (100,000–125,140).

Reduces personal_allowance linearly once **adjusted_net_income** exceeds £100,000.

- **Guard:** $100000 < p.\text{adjusted_net_income} < 125140.$
- **Effect:** $p.\text{personal_allowance} := 12570 - (p.\text{adjusted_net_income} - 100000) / 2.$
- **Notes:** Attribute-only update; no edges are created or deleted (all elements shown **black** as preserved). Applied exactly once via the controller ordering

Rule Description: If a Person's adjusted_net_income is greater than 100000 and less than 125140, then set their personal_allowance to 12570 minus half of the amount by which their income exceeds 100000 (exactly once).

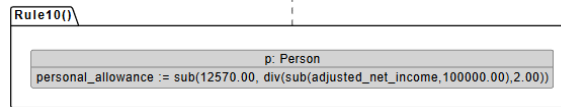


Figure 14 Rule Personal Allowance taper (100k–125,140)

5.5 Groove Artefacts (GXL & GUI Previews)

This section packages the machine-readable inputs for GROOVE and their GUI previews. To ship the artefacts in native GROOVE formats—type.gty (type graph), start.gst (start graph), and individual rule files *.gpr—which are semantically equivalent to their GXL representations. All files are generated from the same rules.json and, therefore, type-consistent with the UML in Section 5. Artefact hashes appear in Section 6.

5.5.1 TypeGraph.gxl (structure & invariants)

What it is.

The type graph is stored as a GXL-formatted XML file with a single `<graph role="type">`. In our native bundle this is type.gty (which is still GXL under the hood). All instance graphs (.gst) and rules are *typed over* this graph, so it defines the only node kinds, attributes, and edge kinds that are legal in the system. The structure mirrors the UML Type Graph in Section 5.4.

How the file is structured (GXL basics).

- **Root:** `<gxl><graph role="type" edgemode="directed" id="type">...</graph></gxl>`.
- **Nodes:** Anonymous ids ($n_0, n_1, \dots$). Visual coordinates are stored as layout attributes (editor convenience; no semantics).
- **Type declaration:** Each node declares its *kind* via a self-loop labeled `type:<Kind>`, e.g., `type:Person`, `type:IncomeSource`, `type:SAVINGS`, `type:BASIC`, `type:HIGHER`, `type:ADDITIONAL`.
- **Attributes:** Attributes are also self-loops on the same node, labeled by domain and name:
 - `real:<attr>` for numeric (floating-point) attributes,
 - `bool:<attr>` for booleans,
 - `string:<attr>` for strings.

Example: `real:total_income` on `Person`, `bool:processed` and `string:source_type` on `IncomeSource`, `real:lower_bound` on `band` nodes.
- **Edge kinds (relations):** Ordinary directed edges between type nodes with plain labels define allowed relations and their direction. In this model you will see:
 - `EarnsFrom` from `Person` $\rightarrow$ `IncomeSource`.
 - `FallsIn` from `Person` $\rightarrow$ each `band kind` (`SAVINGS`, `BASIC`, `HIGHER`, `ADDITIONAL`).
- **Tool metadata:** A small `$version` attribute and layout strings are editor metadata only.

Concrete node kinds & attributes (as loaded in GROOVE).

- **Person** — `total_income`, `adjusted_net_income`, `personal_allowance`, `taxable_income`, `tax`, `pension_contributions`, `net_relevant_earnings`, `pension_relief`, `relief_at_source` (*bool*), `taxpayer_registered_blind` (*bool*), `blind_persons_allowance`, `charitable_donations`, `gift_aid_relief`, `personal_savings_allowance`, `dividend_allowance`, `total_other_income`, `amount_in_starting_savings`.
- **IncomeSource** — `source_type` (*string*), `revenue`, `net_income`, `expenses`, `processed` (*bool*).
- **Band nodes** — `SAVINGS`, `BASIC`, `HIGHER`, `ADDITIONAL` each with `lower_bound`, `upper_bound`, `rate`.

Typing & invariants the file enforces.

- **Kind safety:** Only the above five node kinds can appear in any `.gst` or rule LHS/RHS.
- **Attribute typing:** Each attribute's domain is fixed by its label (`real`, `bool`, `string`). You cannot assign a mismatched type in rules or start graphs.

- **Relation direction:**

- EarnsFrom must go Person $\rightarrow$ IncomeSource.
- FallsIn must go Person $\rightarrow$ Band (to any of the four band kinds).

- **No numeric ranges in the type file:** Bounds like “rate $\in$ [0,1]” or ordering like “BASIC.upper_bound < HIGHER.upper_bound” are not encoded here.
- **No multiplicities in the type file:** Cardinalities (e.g., one Person with many IncomeSources; zero-or-more FallsIn) are treated as unconstrained at the type level and are governed by the rules/control file.

Why this matters.

Because GROOVE type-checks every rule and instance graph against this file, it gives us: (i) a single source of truth for allowed nodes, attributes, and relations; (ii) early rejection of ill-typed attribute writes (e.g., setting a boolean where a real is expected); and (iii) guaranteed directionality for domain relations (e.g., Person $\rightarrow$ IncomeSource).

(Note: concrete numeric values for band thresholds/rates and all initial person fields are provided in *StartGraph.gxl/gst*; the type file only declares that such attributes exist and what types they have.)

```
<graph role="type" edgemode="directed" id="type">
  <node id="n0"/><!-- Person -->
  <edge from="n0" to="n0"><attr name="label"><string>type:Person</string></attr></edge>
  <edge from="n0" to="n0"><attr name="label"><string>real:total_income</string></attr></edge>
  <edge from="n0" to="n0"><attr name="label"><string>bool:relief_at_source</string></attr></edge>

  <node id="n1"/><!-- IncomeSource -->
  <edge from="n1" to="n1"><attr name="label"><string>type:IncomeSource</string></attr></edge>
  <edge from="n1" to="n1"><attr name="label"><string>string:source_type</string></attr></edge>
  <edge from="n1" to="n1"><attr name="label"><string>bool:processed</string></attr></edge>

  <!-- Relation types -->
  <edge from="n0" to="n1"><attr name="label"><string>EarnsFrom</string></attr></edge>
  <!-- FallsIn edges from Person to each band kind likewise appear -->
</graph>
```

Figure 15 *type.gty* (Tiny representative snippet)

5.5.2 StartGraph.gxl (nodes, attributes, initial conditions)

Purpose.

The start graph is the concrete initial state used by GROOVE to begin an execution. In the native bundle this is *start.gst*; it is *typed over* *type.gty*, so every node/edge/attribute here must conform to the type graph.

How the file is structured (GXL basics).

- **Root:** `<gxl><graph role="host" edgemode="directed" id="start"> ... </graph></gxl>`.
- **Nodes:** Each concrete object is a `<node id="...">`, with its type given by a self-loop labeled `type:<Kind>` (e.g., `type:Person`, `type:BASIC`).
- **Attributes:** Concrete values are attached via self-loops labeled by domain and name (e.g., `real:personal_allowance`, `bool:relief_at_source`). The value is held in the edge's `<attr name="label"><string>...</string></attr>` field by GROOVE's convention.
- **Relations:** Ordinary labeled edges connect nodes to instantiate relations declared in the type graph (e.g., `EarnsFrom` if any sources exist).
- **Editor metadata:** Non-semantic layout strings may appear for coordinates; they can be ignored by analysis.

Baseline contents (initial state).

- One Person object with numeric attributes initialised to 0.0, except:
 - `personal_allowance = 12570.0`
 - All booleans set to false (e.g., `relief_at_source=false`, `taxpayer_registered_blind=false`). These fields (e.g., `adjusted_net_income`, `taxable_income`, `tax`) start at 0.0 and are derived later by rules.
- **Band objects:** One concrete node for each band kind declared in the type graph — `SAVINGS`, `BASIC`, `HIGHER`, `ADDITIONAL` — each with `lower_bound`, `upper_bound`
 - These are constants for the run; rules *read* them but do not modify them.
 - No `FallsIn` edges are placed initially; those links are created by rules when amounts are allocated into bands.
- **Income sources: None pre-placed.** `IncomeSource` nodes (and their `EarnsFrom` edges) are typically created by rules during execution (e.g., `CreateSelfEmploymentIncome`).
- **No derived edges or markers:** Flags like `processed` only appear once sources exist; there are no pre-set markers on the person.

Why this matters.

The start graph encodes all *concrete* initial values (especially tax-year thresholds and rates) and guarantees determinism: given this graph plus a fixed rule/control sequence, GROOVE

produces the same run. The type graph ensures well-typedness; the start graph supplies actual numbers and the initial world state.

```

1 <graph role="host" edgemode="directed" id="start">
2   <!-- Person -->
3   <node id="p0"/>
4   <edge from="p0" to="p0"><attr name="label"><string>type:Person</string></attr></edge>
5   <edge from="p0" to="p0"><attr name="label"><string>real:total_income=0.0</string></attr></edge>
6   <edge from="p0" to="p0"><attr name="label"><string>real:personal_allowance=12570.0</string></attr></edge>
7   <edge from="p0" to="p0"><attr name="label"><string>bool:relief_at_source=false</string></attr></edge>
8   <edge from="p0" to="p0"><attr name="label"><string>bool:taxpayer_registered_blind=false</string></attr></edge>
9
10  <!-- Band: BASIC (others analogous: SAVINGS, HIGHER, ADDITIONAL) -->
11  <node id="b0"/>
12  <edge from="b0" to="b0"><attr name="label"><string>type:BASIC</string></attr></edge>
13  <edge from="b0" to="b0"><attr name="label"><string>real:lower_bound=...</string></attr></edge>
14  <edge from="b0" to="b0"><attr name="label"><string>real:upper_bound=...</string></attr></edge>
15  <edge from="b0" to="b0"><attr name="label"><string>real:rate=...</string></attr></edge>
16
17  <!-- No IncomeSource nodes yet; rules will create them and connect with EarnsFrom -->
18 </graph>

```

Figure 16 start.gst (Tiny representative snippet)

5.5.3 Ruleset.gxl (rule list & naming)

Purpose & packaging.

Rules are supplied as individual .gpr files and appear in GROOVE's **Rules** tab. Filenames remain stable (e.g., CreateSelfEmploymentIncome.gpr, rule10.gpr, rule22a.gpr) to support deterministic runs.

Reading the GUI.

- Green elements are created on the RHS; black elements are preserved from the LHS.
- Attribute updates are shown inside an object box (e.g., $p.\text{tax} := p.\text{tax} + \dots$).
- Guards and parameter pins ($p_0, p_1, \dots$) appear on the rule boundary and feed $:=$ expressions.
- A helper rule (inputParams.gpr) exposes external inputs via $p_0, p_1, \dots$ when needed.

Representative exemplars.

- **Constructor pattern (create source).**

CreateSelfEmploymentIncome.gpr constructs an IncomeSource, links it to a Person with a green EarnsFrom edge, and assigns: $\text{source_type} = \text{'SELF_EMPLOYMENT'}$, $\text{revenue} := p_0$, $\text{expenses} := p_1$, $\text{net_income} := p_0 - p_1$, $\text{processed} := \text{false}$.


```

1 <gxl>
2   <graph role="rule" edgemode="directed" id="CreateSelfEmploymentIncome">
3     <!-- LHS (preserve) -->
4     <node id="p"/><edge from="p" to="p"><attr name="label"><string>type:Person</string></attr></edge>
5
6     <!-- RHS (create) -->
7     <node id="is"/><edge from="is" to="is"><attr name="label"><string>type:IncomeSource</string></attr></edge>
8     <edge from="p" to="is"><attr name="label"><string>EarnsFrom</string></attr></edge>
9
10    <!-- Attribute initialisation via parameter pins p0, p1 -->
11    <edge from="is" to="is"><attr name="label"><string>string:source_type='SELF_EMPLOYMENT'</string></attr></edge>
12    <edge from="is" to="is"><attr name="label"><string>real:revenue:=p0</string></attr></edge>
13    <edge from="is" to="is"><attr name="label"><string>real:expenses:=p1</string></attr></edge>
14    <edge from="is" to="is"><attr name="label"><string>real:net_income:=p0-p1</string></attr></edge>
15    <edge from="is" to="is"><attr name="label"><string>bool:processed=false</string></attr></edge>
16  </graph>
17 </gxl>

```

Figure 17 CreateSelfEmploymentIncome.gpr (Tiny representative snippet)

- **Guarded allowance update.**

rule10.gpr applies the Personal Allowance taper under the guard $100000 < \text{adjusted_net_income} < 125140$, updating $\text{personal_allowance} := 12570 - (\text{adjusted_net_income} - 100000)/2$.

```

1 <gxl>
2   <graph role="rule" edgemode="directed" id="rule10">
3     <!-- LHS (preserve) -->
4     <node id="p"/><edge from="p" to="p"><attr name="label"><string>type:Person</string></attr></edge>
5
6     <!-- Guard (range on ANI) -->
7     <edge from="p" to="p"><attr name="label">
8       <string>guard: 100000 < adjusted_net_income < 125140</string>
9     </attr></edge>
10
11    <!-- RHS (update) -->
12    <edge from="p" to="p"><attr name="label">
13      <string>real:personal_allowance := 12570 - (adjusted_net_income - 100000)/2</string>
14    </attr></edge>
15  </graph>
16 </gxl>

```

Figure 18 rule10.gpr (Tiny representative snippet)

- **Band allocation/tax increment.**

rule22a.gpr allocates into the BASIC band using $\text{FallsIn}(p, \text{b:BASIC})$ and increments tax: $p.\text{tax} := p.\text{tax} + (\min(p.\text{taxable_income} - p.\text{amount_in_starting_savings}, \text{b.upper_bound}) - \text{b.lower_bound}) * \text{b.rate}$.

```

1 <gxl>
2   <graph role="rule" edgemode="directed" id="rule22a_">
3     <!-- LHS (preserve) -->
4     <node id="p"/><edge from="p" to="p"><attr name="label"><string>type:Person</string></attr></edge>
5     <node id="b"/><edge from="b" to="b"><attr name="label"><string>type:BASIC</string></attr></edge>
6     <edge from="p" to="b"><attr name="label"><string>FallsIn</string></attr></edge>
7
8     <!-- RHS (update) -->
9     <edge from="p" to="p"><attr name="label">
10       <string>
11         real:tax := tax + ( min(taxable_income - amount_in_starting_savings, b.upper_bound)
12         | | | | | - b.lower_bound ) * b.rate
13       </string>
14     </attr></edge>
15   </graph>
16 </gxl>

```

Figure 19 rule22a_gpr (Tiny representative snippet)

5.5.4 Screenshots from GROOVE (Type, Start, Rules)

These previews confirm the artefacts load and type-check in GROOVE and document the exact shapes used in experiments.

- **Figure-A — Type graph.**

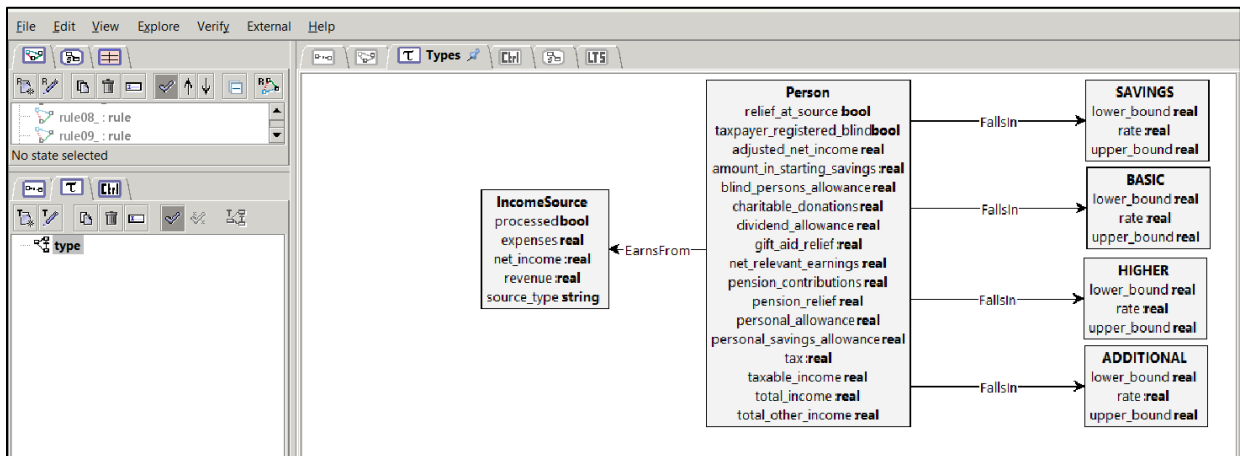


Figure 20 Type Graph in groove interface

- **Figure-B — Start graph.**

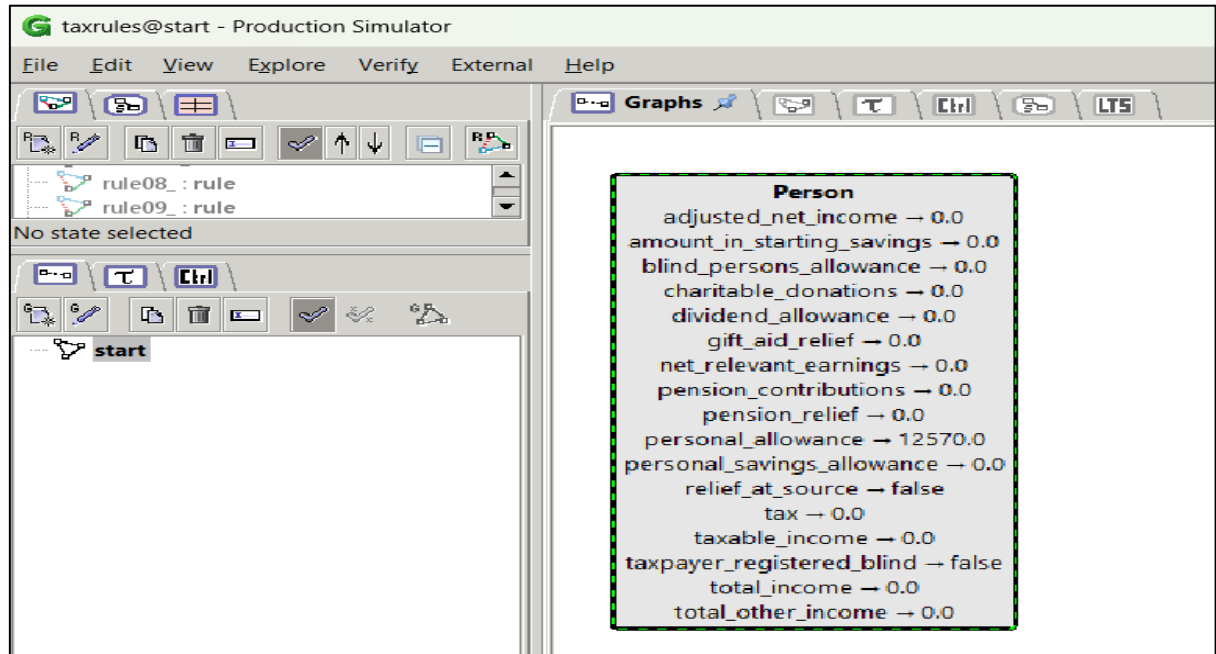


Figure 21 Strat Graph in groove interface

- **Figure-C — Rule inputs mapped to Person attributes.**

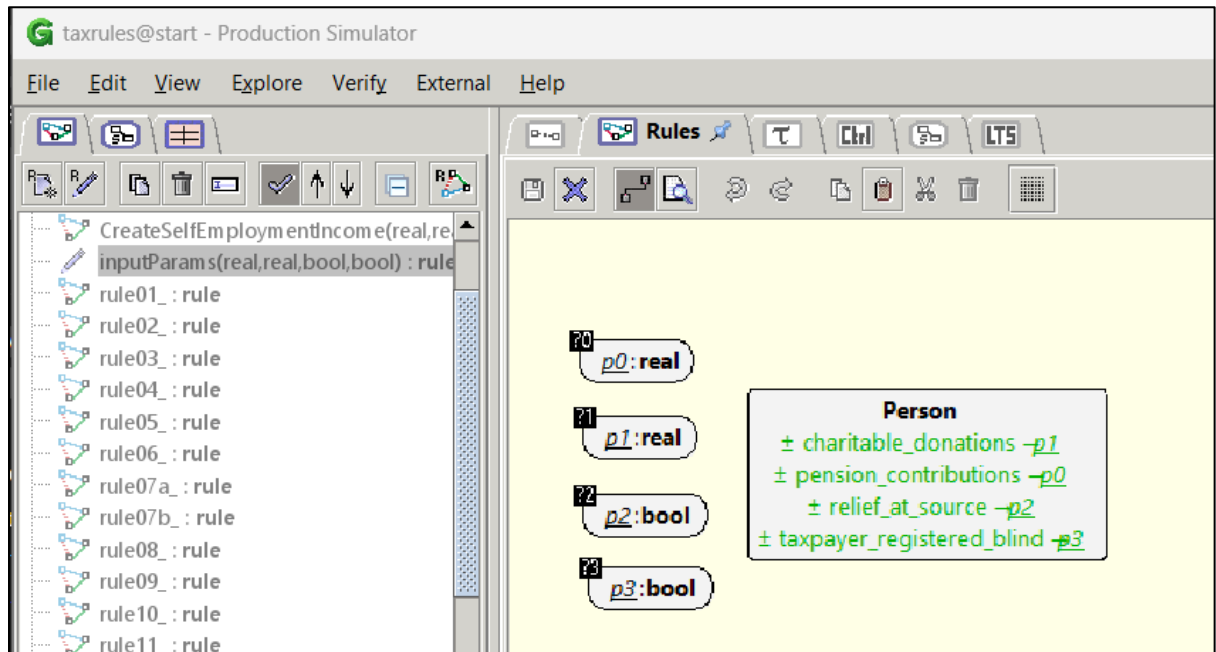


Figure 22 Input pram Rule in groove interface

- **Figure-D — Create self-employment income (green creation).**

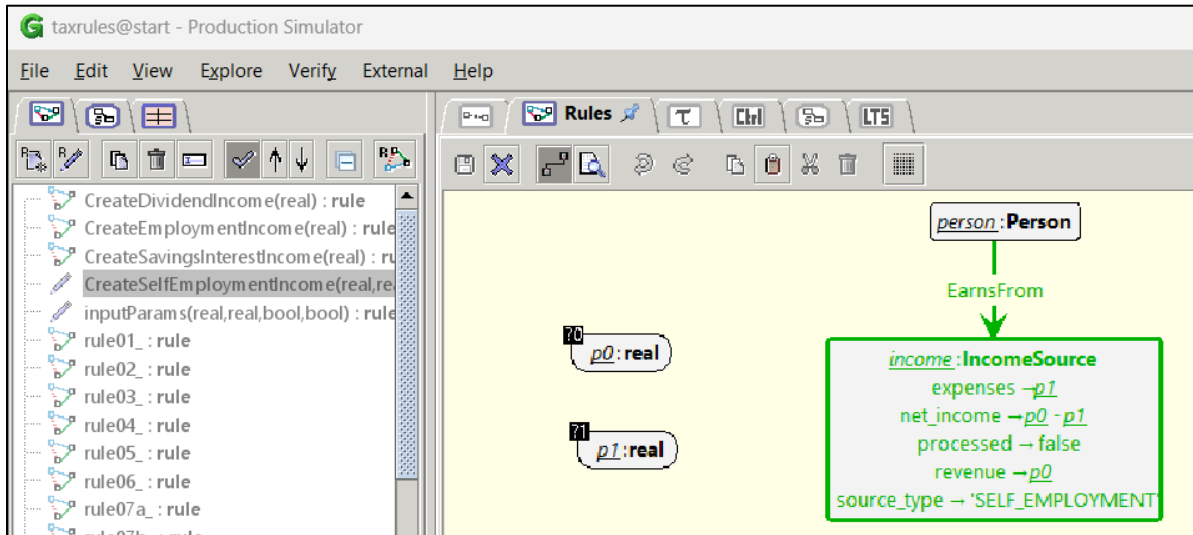


Figure 23 Create Self Employment Income in groove interface

- **Figure-E — Personal Allowance taper.**

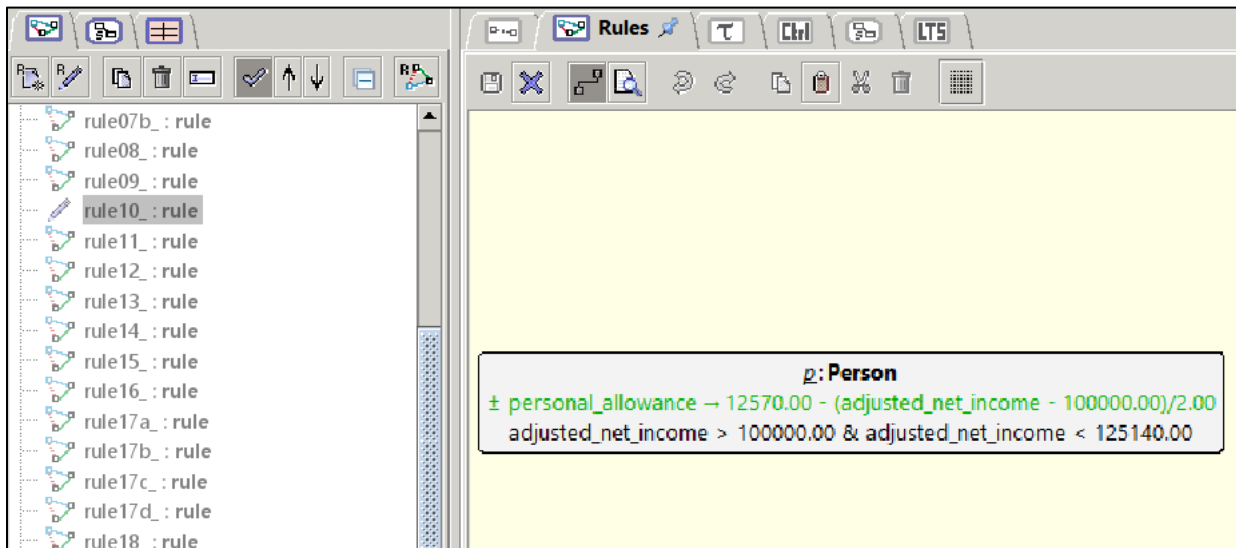


Figure 24 Rule 10 Personal Allowance in groove interface

- **Figure-F — BASIC band increment.**

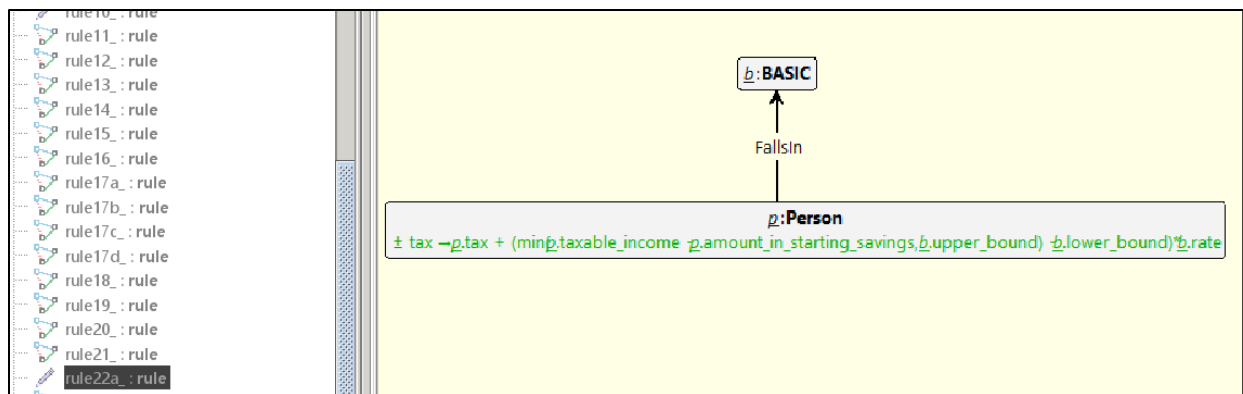


Figure 25 Rule 22a BASIC Band Increment in groove interface

6 Implementation

This section describes the implementation of the end-to-end pipeline from a single JSON source (rules.json) to generated artefacts (PlantUML, GXL, controllers), the backend API and UI, and a tax-advice bot that guides users through allowances, bands, and reliefs. Each subsection is written to be reproducible and references concrete inputs/outputs rather than pseudocode.

6.1 JSON → PlantUML Codegen (Class Diagram / Start Graph / Rules)

This subsection documents how a **single JSON source** (rules.json, loaded as JSON-Rules.txt in scripts) is transformed into three PlantUML artefacts:

- classDiagram.puml — **Class Diagram** of the domain (Person, IncomeSource, tax bands).
- startGraph.puml — **Start Graph** as a concrete object instance (Person as n0) with sensible defaults.
- allRules.puml — **Rule view**: one PlantUML *package* per JSON rule, showing nodes, attribute updates/tests, edges, and a descriptive note.

Full scripts are provided in the appendix; below are the key design decisions and minimal, illustrative code snippets.

6.1.1 Class Diagram Codegen (classDiagram.puml)

Input. typeGraph object inside rules.json

Output. A PlantUML class diagram showing:

- IncomeSource (attributes from JSON, with selected type mapping),
- one class per taxpayer bracket (e.g., BASIC, HIGHER, ADDITIONAL) with lower_bound, rate, upper_bound,
- Person (all attributes except taxpayer_rate),
- associations Person --> IncomeSource : EarnsFrom and Person --> <Bracket> : FallsIn.

Type mapping & bracket extraction

To normalise core JSON types into PlantUML-friendly types and extract bracket names from Person.taxpayer_rate (enum).

```

1  # helper to map JSON datatypes → PlantUML
2  def map_type(dt: str, attr: str) -> str | None:
3      if dt == "float": return "Real"
4      if dt == "boolean": return "Boolean"
5      if dt.startswith("enum<"):
6          # keep IncomeSource.source_type as String, drop other enums from class body
7          return "String" if attr == "source_type" else None
8      return dt
9
10 # extract bracket names from Person.taxpayer_rate enum<...>
11 brackets: list[str] = []
12 for node in tg.get("nodes", []):
13     if node["type"] == "Person":
14         for attr, dt in node.get("attributes", {}).items():
15             if attr == "taxpayer_rate" and dt.startswith("enum<"):
16                 inner = dt[len("enum<"):-1]
17                 brackets = [b.strip() for b in inner.split(",")]
18                 break
19         break

```

Figure 26 Helper Function for mapping types into UML

Emitting classes & associations

One class per bracket is created and wire associations from Person.

```

55 # one class per tax bracket
56 for br in brackets:
57     lines.append(f"class {br} {{")
58     for attr in ("lower_bound", "rate", "upper_bound"):
59         lines.append(f"    + {attr} : Real")
60     lines.append("}")
61     lines.append("")
62
63 # Associations
64 lines.append("'=== Associations ===")
65 lines.append("")
66 lines.append("Person --> IncomeSource : EarnsFrom")
67 for br in brackets:
68     lines.append(f"Person --> {br} : FallsIn")
69

```

Figure 27 Helper Function for creating tax bracket and write associations

Why this design?

- Keeping only IncomeSource.source_type as String keeps the diagram readable while allowing the rules layer to handle exact enum logic.
- Representing brackets as first-class classes (BASIC, HIGHER, ADDITIONAL) makes **band-specific notes/assertions** clear in the rules view later.

6.1.2 Start Graph Codegen (startGraph.puml)

Goal. Emit a minimal, deterministic object diagram with a single Person instance (n0) and defaulted attributes, while allowing targeted overrides (e.g., statutory personal_allowance = 12570.0).

Defaults policy: float $\rightarrow$ 0.0, Boolean $\rightarrow$ false, enum<...> $\rightarrow$ first literal (uppercased), otherwise "".

```

1  def default_value(attr_type: str) -> str:
2      if attr_type == "float": return "0.0"
3      if attr_type == "boolean": return "false"
4      if attr_type.startswith("enum<") and attr_type.endswith(">"):
5          choices = attr_type[len("enum<"):-1].split(",")
6          return choices[0].upper()
7      return ""

```

Figure 28 Helper function to set defaults for Start Graph

Rendering a concrete Person instance

- Enums are omitted from the object body to keep the instance lean.
- Specific attributes can be overridden (e.g., personal_allowance).

```

22  def make_instance_lines(type_name, alias, attrs, overrides=None):
23      # filter out enum-typed attrs, sort the rest
24      keys = sorted(k for k, t in attrs.items() if not t.startswith("enum<"))
25      lines = [f'object "{type_name}" as {alias} {{{']
26      for attr in keys:
27          attr_type = attrs[attr]
28          val = overrides.get(attr, default_value(attr_type)) if overrides else default_value(attr_type)
29          lines.append(f"    {attr.ljust(25)} = {val}")
30      lines.append("}")
31      return lines

```

Figure 29 Helper function to render Person Object

```

@startuml
' Define the object instance n0 of type Person
object "Person" as n0 {
    adjusted_net_income      = 0.0
    amount_in_starting_savings = 0.0
    blind_persons_allowance  = 0.0
    charitable_donations     = 0.0
    dividend_allowance       = 0.0
    gift_aid_relief          = 0.0
    net_relevant_earnings    = 0.0
    pension_contributions    = 0.0
    pension_relief           = 0.0
    personal_allowance       = 12570.0
    personal_savings_allowance = 0.0
    relief_at_source         = false
    tax                      = 0.0
    taxable_income           = 0.0
    taxpayer_registered_blind = false
}

```


Figure 30 Generated Example

6.1.3 Rules → PlantUML Codegen (allRules.puml)

Each JSON rule is rendered as:

- a PlantUML package named with the derived signature (function-style) showing ordered parameters,
- objects for each referenced node (with attribute set: and test: lines),
- edges (EarnsFrom as solid, FallsIn as green create),
- a note containing the rule's English description.

Deterministic ordering of rules

Numeric rule IDs sort numerically; non-numeric fall back to lexicographic.

```

6  def load_rules(path: Path) -> List[Dict]:
7      data = json.loads(path.read_text(encoding="utf-8-sig"))
8      if "rules" not in data:
9          raise KeyError("No 'rules' key in JSON")
10
11     def rule_sort_key(r):
12         name = r.get("name", "")
13         try:
14             return (0, int(name))
15         except (ValueError, TypeError):
16             return (1, name)
17     return sorted(data["rules"], key=rule_sort_key)

```

Figure 31 Rule loader with stable numeric/lexicographic ordering.

Signature derivation & parameter order

Parameters are shown as name (id): type and ordered by parin.

```

32  params: List[str] = []
33  for param in sorted(rule.get("parameters", []), key=lambda p: p.get("parin", 0)):
34      pid = param.get("id")
35      pname = param.get("name", pid)
36      ptype = param.get("type", "")
37      params.append(f"{pname} ({pid}): {ptype}")
38  signature = f"func({', '.join(params)})"
39  desc = rule.get("description", "").strip()

```

Figure 32 Derives function-style signatures from ordered parameters

Application conditions

Supports both **conjunctive strings** (a && b && c) and **predicate syntax** (eq()/gt()/lt()/ge()/le()). Band boundary checks are injected into TaxBand objects for readability.

```

42 app_formula = rule.get("applicationConditions", {}).get("formula", "")
43 if "&&" in app_formula:
44     app_tests = [c.strip() for c in app_formula.split("&&") if c.strip()]
45 else:
46     # extract predicates like eq(...), gt(...), lt(...), ge(...), le(...)
47     app_tests = re.findall(r'\b(?:eq|gt|lt|ge|le)\s*\([^)]+\)', app_formula)

```

Figure 33 Parses application conditions

Attributes: test: vs set:

- test: lines are emitted as-is (guards).
- set: lines render as update statements (attr := expr).

```

67 for attr, expr in node_attrs.items():
68     # split only on first colon
69     _, body = expr.split(":", 1)
70     body = body.strip()
71
72     if expr.startswith("test:"):
73         lines.append(f"    {body}")
74     elif expr.startswith("set:"):
75         lines.append(f"    {attr} := {body}")

```

Figure 34 Emits test: guards and set: update statements

Edge rendering

EarnsFrom uses a solid arrow; FallsIn uses a green create-style arrow to reflect band assignment.

```

91 if etype == "EarnsFrom":
92     arrow, lbl = "-->", "EarnsFrom"
93 else:
94     arrow, lbl = "-[#green]->", "FallsIn"
95 lines.append(f"    {src}{n} {arrow} {tgt}{n} : {lbl}")
96
97 if rule.get("edges"):
98     lines.append("")

```

Figure 35 Renders edges: solid EarnsFrom, green create FallsIn

6.2 JSON → GROOVE (GXL) Converter

This section documents the three emitters that turn a single JSON source into GROOVE-compatible GXL graphs: the **Start Graph** generator, the **Type Graph** generator, and the **Rule Graph** generator. Each sub-section explains the intent, input/output, and the core mechanics, with small code snippets you can screenshot (keep full code in the appendix).

6.2.1 Purpose & I/O

- **Input (common):** JSON-Rules.txt containing either a single typeGraph or a typedGraphs array and, for rules, per-rule JSON objects.
- **Output (per emitter):**
 - **Start Graph:** start.gty (role=graph) seeded with default attribute values and optional fixed layouts.
 - **Type Graph:** type.gty (role=type) declaring node kinds, attributes (as self-loops), and inter-node edge types.
 - **Rule Graphs:** rule.gpr (role=rule) per rule, encoding parameters, node/create vs preserve, attribute updates (let:), tests (test:), and edges (with new: where applicable).
- **Conventions:** \$version = "curly", directed edges, edgeids="false", deterministic emission order.

6.2.2 Start Graph Emitter

Intent. Create a minimal start graph with one or more initial nodes (e.g., a Person) and seed **attribute defaults** so downstream rules run deterministically.

Key ideas.

- Picks the first node in the given typeGraph as the start node.
- Writes a type self-loop (type:Person) plus attribute self-loops as let:<name>=<value>.
- Accepts a layout_map to stamp fixed node positions for reproducible screenshots.

```

7  def json_to_gxl(
8      data: Dict[str, Any],
9      gxl_path: str,
10     graph_id: str = "start",
11     graph_role: str = "graph",
12     version_label: str = "curly",
13     default_values: Optional[Dict[str, Any]] = None,
14     layout_map: Optional[Dict[str, str]] = None,
15 ) -> None:

```

Figure 36 Start-graph function signature and knobs (role, version, defaults, layouts)

Emit version and graph shell

```

39 root = ET.Element("gxl", xmlns="http://www.gupro.de/GXL/gxl-1.0.dtd")
40 graph = ET.SubElement(root, "graph", role=graph_role, edgeids="false", edgemode="directed", id=graph_id)
41 version_attr = ET.SubElement(graph, "attr", name="$version")
42 ET.SubElement(version_attr, "string").text = version_label

```

Figure 37 Graph root with \$version="curly" and deterministic flags

Type & attribute self-loops

```

47 edge = ET.SubElement(graph, "edge", **{"from": node_id, "to": node_id})
48 label = ET.SubElement(edge, "attr", name="label")
49 ET.SubElement(label, "string").text = f"type:{node['type']}"
50 for attr_name, attr_type in node.get("attributes", {}).items():
51     if attr_type not in ("float", "boolean"):
52         continue
53     default = defaults.get(attr_name, 0.0 if attr_type == "float" else False)
54     value_str = str(default).lower() if isinstance(default, bool) else str(default)
55     edge = ET.SubElement(graph, "edge", **{"from": node_id, "to": node_id})
56     label = ET.SubElement(edge, "attr", name="label")
57     ET.SubElement(label, "string").text = f"let:{attr_name}={value_str}"

```

Figure 38 Self-loops: type:... and let:... seeds for floats/booleans.

Pretty-print

```

59 # pretty-print
60 raw = ET.tostring(root, encoding="utf-8")
61 pretty_bytes = minidom.parseString(raw).toprettyxml(
62     indent="    ", encoding="UTF-8"
63 )
64 pretty = pretty_bytes.decode("utf-8").replace(
65     '<?xml version="1.0" encoding="UTF-8"?>',
66     '<?xml version="1.0" encoding="UTF-8" standalone="yes"?>',
67 )
68 Path(gxl_path).write_text(pretty, encoding="utf-8")

```

Figure 39 Stable XML header and indentation for GROOVE.

6.2.3 Type Graph Emitter

Intent. Declare the meta-model: node kinds, their attribute types (as loops with real:/bool: prefixes), and allowed inter-node edges.

Key ideas.

- Preserves author-defined order for nodes and edges to keep diffs small.
- Maps JSON id/type to deterministic node IDs (n0, n1, ...) via an id_map.

- Emits attribute-type loops using prefixes (real:, bool:) expected by our tooling.

Node mapping & ordered creation

```

19     id_map: Dict[str, str] = {}
20     ordered_nodes: List[Tuple[str, str, Any]] = []
21
22     for idx, node in enumerate(tg["nodes"]):
23         key = node.get("id") or node["type"]
24         nid = node.get("id", f"n{idx}")
25         id_map[key] = nid
26         ordered_nodes.append((key, nid, node.get("layout")))

```

Figure 40 Stable node IDs and optional per-node layout.

Type & attribute loops

```

54     edge_type = ET.SubElement(graph, "edge", **{"from": nid, "to": nid})
55     label_type = ET.SubElement(edge_type, "attr", name="label")
56     ET.SubElement(label_type, "string").text = f"type:{node['type']}"
57
58     for name, typ in node.get("attributes", {}).items():
59         prefix = {
60             "float": "real",
61             "boolean": "bool"
62         }.get(typ, "string")
63         edge_attr = ET.SubElement(
64             graph,
65             "edge",
66             **{"from": nid, "to": nid}
67         )
68         label_attr = ET.SubElement(edge_attr, "attr", name="label")
69         ET.SubElement(label_attr, "string").text = f"{prefix}:{name}"

```

Figure 41 Attribute kinds become real:/bool: loops in the type graph.

Inter-node edge kinds

```

71     for edge in tg.get("edges", []):
72         src = id_map[edge["source"]]
73         tgt = id_map[edge["target"]]
74         e = ET.SubElement(
75             graph,
76             "edge",
77             **{"from": src, "to": tgt}
78         )
79         a = ET.SubElement(e, "attr", name="label")
80         ET.SubElement(a, "string").text = edge["type"]

```

Figure 42 Declarative edge types (e.g., EarnsFrom).

6.2.4 Rule Graph Emitter

Intent. Convert a **single rule JSON** into a GROOVE rule graph encapsulating:

- **Parameters** with id:, <type>: and parin: loops.
- **Node actions:** preserve vs **create** (new: loop).
- **Assignments** via let:<attr> = <expr> with recursive arithmetic parsing.
- **Conditions** from applicationConditions.graphs as test:and(...) loops.
- **Edges** with new:<EdgeType> for created edges.

Parameter encoding

```

143     for p in rule_json.get('parameters', []):
144         ensure_node(p['id'])
145         xid = mapping[p['id']]
146         for label in (f"id:{p['id']}", f"{p['type']}:", f"parin:{p['parin']}"):
147             e = ET.SubElement(graph, 'edge', {'from': xid, 'to': xid})
148             a = ET.SubElement(e, 'attr', {'name': 'label'})
149             ET.SubElement(a, 'string').text = label

```

Figure 43 Parameter encoding

Create vs preserve nodes

```

174     # Always stamp id + type; add 'new:' only if action == 'create'
175     if node.get('action') == 'create':
176         e = ET.SubElement(graph, 'edge', {'from': xid, 'to': xid})
177         a = ET.SubElement(e, 'attr', {'name': 'label'})
178         ET.SubElement(a, 'string').text = "new:"

```

Figure 44 new: marks freshly created nodes in the rule.

Attribute assignments: `parse_arith_expr` normalises a tiny expression string by trimming spaces and recursively rewriting it. It handles operators in this order—*, -, `floor_divide(a,b)`, then +—splitting on the first occurrence each time; * and - remain infix, + becomes `add(left,right)`, and `floor_divide(a,b)` becomes `div(a,b)`. Numbers are formatted to two decimals, dotted names like `x.y` are returned as-is, and bare identifiers are prefixed with the optional alias unless they're true/false or listed in `param_ids`. If nothing matches, it returns the original text. Note it doesn't implement standard operator precedence (except the `floor_divide(...)` case), so `"a+b*c"` is treated as `(a+b)*c`.

```

31 def parse_arith_expr(expr, alias=None):
32     expr = expr.strip()
33     if '**' in expr:
34         left, right = expr.split('**', 1)
35         return f"({parse_arith_expr(left.strip(), alias)} * {parse_arith_expr(right.strip(), alias)})"
36     m = re.match(r'^(\d+)\s*\s*(\d+)\s*$', expr)
37     if m:
38         return f"({parse_arith_expr(m.group(1).strip(), alias)} - {parse_arith_expr(m.group(2).strip(), alias)})"
39     if expr.startswith('floor_divide(') and expr.endswith(')'):
40         inner = expr[len('floor_divide('):-1]
41         a, b = [x.strip() for x in inner.split(',', 1)]
42         return f"div({parse_arith_expr(a, alias)}, {parse_arith_expr(b, alias)})"
43     if '+' in expr:
44         left, right = expr.split('+', 1)
45         return f"add({parse_arith_expr(left.strip(), alias)}, {parse_arith_expr(right.strip(), alias)})"
46     if re.fullmatch(r'\d+(\.\d+)?', expr):
47         return f"float(expr):.2f"
48     if '.' in expr:
49         return expr
50     if re.fullmatch(r'[A-Za-z_]\w*', expr):
51         if expr in ('true', 'false') or expr in param_ids:
52             return expr
53         return f"{alias}.{expr}" if alias else expr
54     return expr

```

Figure 45 Recursive expression normalisation for let: updates.

Conditional tests

```

def parse_and_expr(expr):
    # splits top-level args; preserves nested and(...); formats numbers
    # strips only condition-node aliases (p0.attr → attr)
    return normalized
ET.SubElement(ET.SubElement(edge, "attr", name="label"),
               "string").text = f"test:{parse_and_expr(cond)}"

```

Figure 46 applicationConditions → test

Created edges

```

199 lbl = ed['type']
200 if ed.get('action') == 'create':
201     lbl = f"new:{lbl}"
202 e = ET.SubElement(graph, 'edge', {'from': src, 'to': tgt})
203 a = ET.SubElement(e, 'attr', {'name': 'label'})
204 ET.SubElement(a, 'string').text = lbl

```

Figure 47 Distinguish new structural links with new

Stable graph IDs

```

18 # Helper: graph ID formatting
19 def format_graph_id(name):
20     try:
21         n = int(name)
22         return f"rule{n:02d}"
23     except (ValueError, TypeError):
24         return str(name)

```

Figure 48 name=7 → id="rule07" for tidy file naming

6.2.5 Determinism & Invariants

- **Order stability:** Nodes/edges emitted in JSON order for reproducible diffs.
- **Versioning:** \$version="curly" stamped in all graphs.
- **Typing discipline:** Type graphs use real:/bool: prefixes; start/rule graphs use type: and let: labels.
- **Layouts:** Optional layout strings propagated so screenshots remain consistent across runs.
- **Safety:** Only float/boolean are auto-seeded in the start graph; other attribute kinds are ignored there by design

6.2.6 Extensibility Hooks

- **default_values (start):** Inject tax-year constants (e.g., personal_allowance=12570.0).
- **layout_map (start/rule):** Lock node positions for figures.
- **Parsers (rule):** Extend parse_arith_expr (e.g., min/max/clamp) and parse_and_expr for richer guards without changing the outer writer.

6.2.7 Known Limitations

- **Expression coverage:** The arithmetic/boolean parsers handle common forms (+, -, *, div(...), nested and(...)). Exotic syntax should be normalised upstream.
- **Single-node start:** The start emitter initialises from the first node of the type graph; multi-node starts require extending the input JSON or a small wrapper.
- **Attribute kinds:** Only float/boolean are seeded in start graphs; strings/enums need explicit initialisers if required.

6.3 GROOVE Control Programs — Structure & Simulator Usage

6.3.1 Purpose

Explain the structure and semantics of GROOVE control programs and how they're executed from the GROOVE Simulator. This section is tool-native (no Excel). Control files (e.g., .gcp) let you impose a deterministic schedule over rule applications for reproducible runs. A control program is interpreted deterministically and sequences which rule may fire at each step [8].

6.3.2 Control program

A control program sits “on top” of your rule system and constrains exploration by allowing/forbidding rule applications in a given order (e.g., `a; try { b; } else { c; }`). This is the recommended way to encode execution strategy without polluting rules with control tokens [8].

6.3.3 Constructs used in this project

- **Atomic (transaction) block** `< ... >`. Groups statements into one transaction step in the state space. Used here to seed inputs before any rule fires (initialization) [14].
- **Linear sequencing.** `stmt1; stmt2; ...` applies statements in strict order for maximal repeatability [8].
- **try { ... } else { ... }.** Attempts a rule; on no match, continues. This yields a deterministic “attempt-then-continue” schedule while remaining faithful to GROOVE control semantics.
- **Optional loops.** GROOVE supports `alap { ... }` (“as long as possible”) and structured loops (`while`, `until`, `do ... while`) [14]. This project keeps the schedule linear + try/else to avoid nondeterminism.

Design note. Branches of a conceptual rule are expressed as suffixed variants (e.g., `rule07a_`, `rule07b_`; `rule17a_...17d_`; `rule22a_...22c_`) to keep each transformation single-purpose while preserving a simple linear schedule.

6.3.4 Control program (TC1) — excerpt

The programme below (shown in the Simulator’s Control pane) performs input initialisation atomically, then executes a fixed try/else {} sequence of rules. The values in the `< ... >` block are the scenario inputs for this test case.

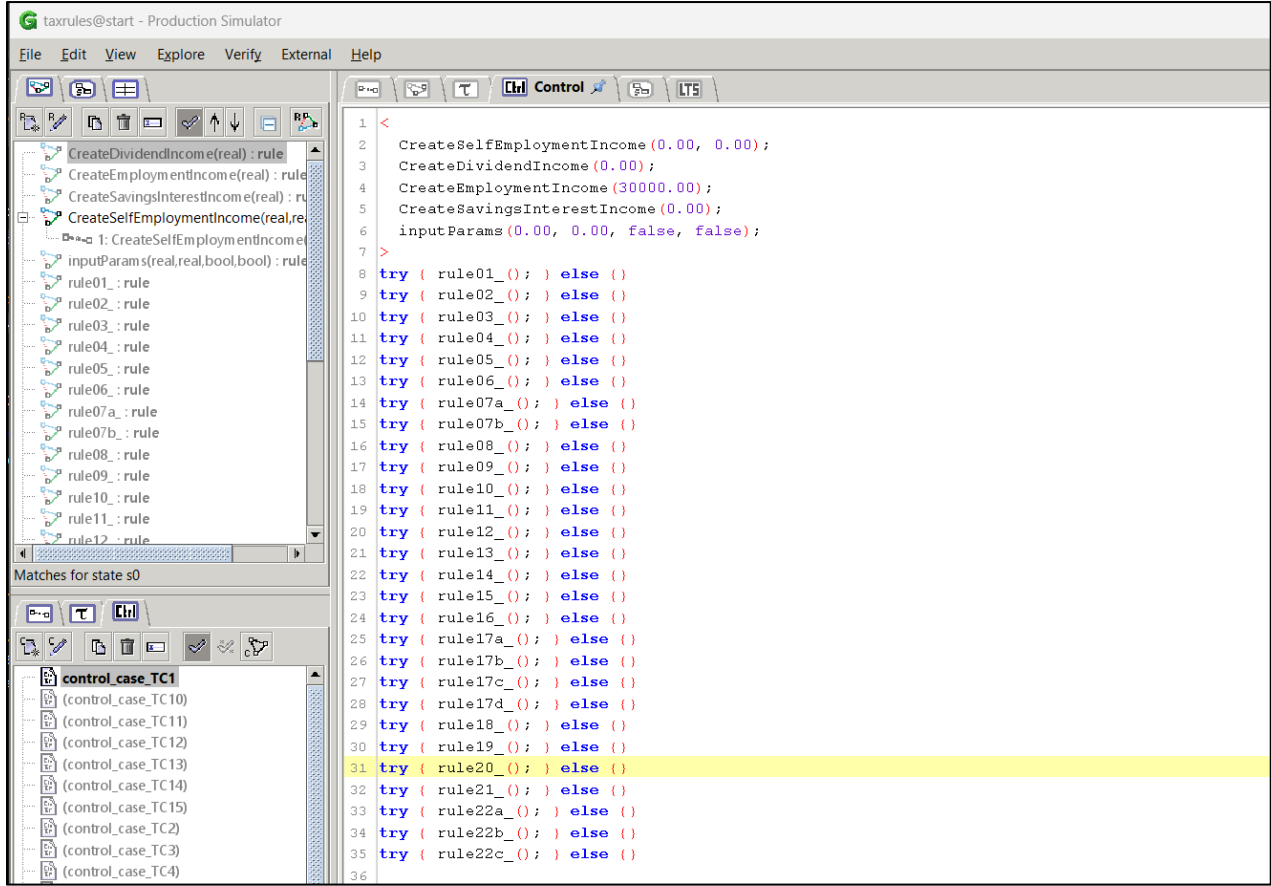


Figure 49 Control program for TC1 (Simulator screenshot)

The initial `< ... >` block atomically seeds all inputs (employment, self-employment, dividends, savings interest, and inputParams). The subsequent linear schedule uses `try { ruleX_(); } else {}` to apply each rule if and only if it matches; otherwise, control advances deterministically to the next statement.

Running from the GROOVE Simulator

1. Open the grammar in the Simulator and switch to the Control tab.
2. Paste or open the .gcp program, then Compile. Any syntax or name errors are reported inline.
3. Explore/Run to execute the state space under the control schedule. The `< ... >` block executes as one transaction; each subsequent `try { rule(); } else {}` either applies the rule once or advances if it does not match.
4. Inspect the resulting graph and trace to confirm the intended, reproducible progression [14], [8].

6.3.5 Why this controller helps (tool-native)

- **Repeatability.** A fixed order of attempted rules ensures like-for-like comparisons across scenarios [8].
- **Separation of concerns.** Rules encode transformations; the control program encodes execution policy [14].
- **Transparency.** Each step is explicit and traceable in the Simulator's state/transition view.
- **Limitation.** Only one control program can be active in the Simulator at a time. If multiple .gcp files exist (e.g., TC1...TCn), you must enable exactly one before running; GROOVE doesn't execute multiple control programs in a single run.

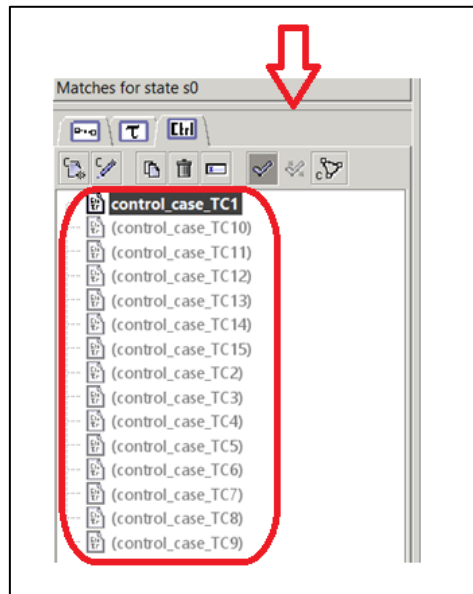


Figure 50 Button for Enabling and Disabling Control Files

6.4 Backend API (FastAPI) — Orchestration, Excel→Control (Bulk Generation) & AI Tax Adviser

6.4.1 Overview & Flow

Pipeline: Upload Excel → generate .gcp control files into the project's .gps folder → run GROOVE Generator.jar → parse state.gst to collect let:* attributes on the Person node as the result payload. The router offloads blocking work (file I/O, Java process, XML parsing) to threads and returns timings for each step.

6.4.2 Endpoints — Generate Controls

A) POST /generate-controls/save-and-run

Uploads an Excel file, writes one or more .gcp files into the same folder the generator uses, runs Generator.jar, then parses state.gst and returns extracted values.

Form fields

- file: Excel .xlsx/.xls with required columns (see below)
- stdout_preview_lines (*optional*): int, defaults to 80 — limits how much of generator stdout/stderr is returned

Figure 51 Form params — /generate-controls/save-and-run

The values map comes from let:* self-loop attributes on the **Person** node in state.gst.

Notes

- Control files are written to the parent directory of GRAPH_GPS_PATH (i.e., where the generator expects them).
- Errors surface with clear HTTP codes/messages (generator failures → **500**; missing state.gst → **404**)

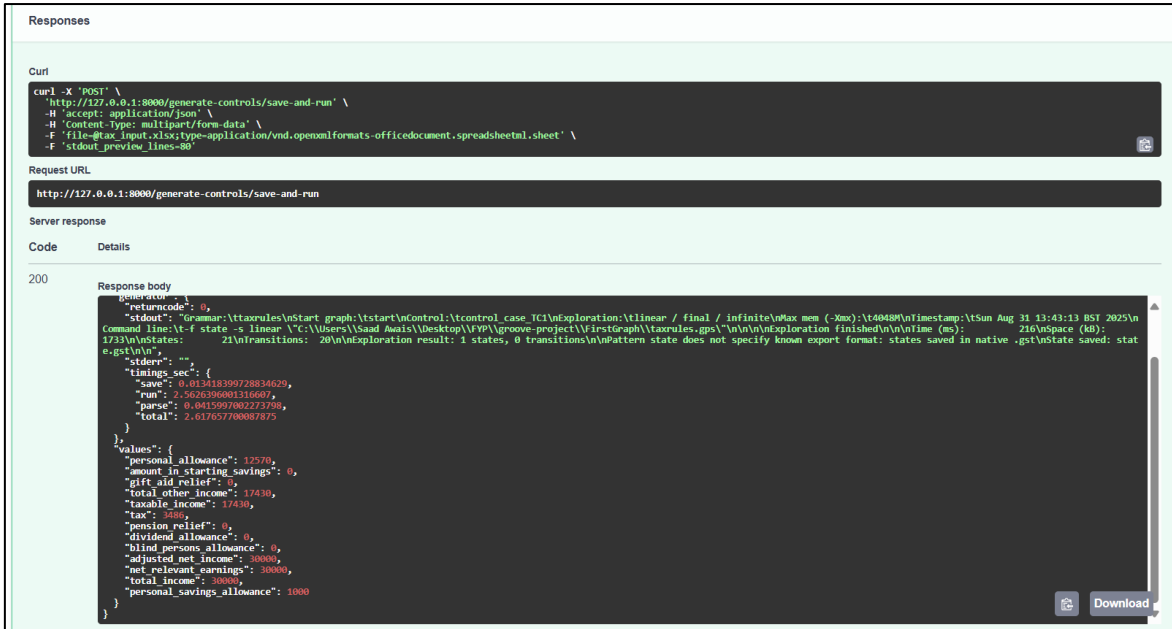


Figure 52 Response JSON shape — /generate-controls/save-and-run

Core (excerpt)

```

146 @router.post("/save-and-run")
147 async def save_and_run(file: UploadFile = File(...),
148                          stdout_preview_lines: int | None = Form(default=80)):
149     # (1) Save .gcp into generator folder
150     written = await to_thread.run_sync(build_controls_to_dir, file.filename, content, gen_cwd)
151     # (2) Run Generator.jar
152     rc, out, err = await to_thread.run_sync(run_generator, gen_cwd, settings.generator_timeout_sec)
153     # (3) Parse state.gst for values
154     groove_values = await to_thread.run_sync(
155         extract_n0_values_from_state_gst, settings.graph_gps_path
156     )

```

Figure 53 Core pipeline (save→run→parse) — FastAPI handler

Fig 50:

B) POST /generate-controls/generate-controls-only

Generates .gcp files **without** running the generator; returns the list of files and timings. Useful for preparing many test cases at once.

Required Excel headers (exact names; no case-insensitive match)
CaseID, SelfEmpRev, SelfEmpExp, DivRev, EmpRev, SavRev, PensionContrib, CharitableDonations, ReliefAtSource, RegisteredBlind.

Figure 54 Form params — /generate-controls/generate-controls-only

Figure 55 Response JSON shape — /generate-controls/generate-controls-only

C) POST /gpt-search

This endpoint provides concise tax-adviser guidance and explanations (text by default, JSON when explicitly requested). It does not run calculations or mutate GROOVE artifacts and does not override computed results. It's for explanations, sanity checks, and structured summaries that can cite official guidance.

(Examples of topics it may explain: Personal Allowance tapering, adjusted net income, Gift Aid gross-up, pension relief at source vs net pay, savings/dividend allowances, etc., with links to HMRC where relevant.)

Endpoint

- POST /gpt-search → returns GPTSearchResponse { answer, model, created, usage? }
- Validates non-empty query (**400** if empty)

- Extracts only the last “User:” turn if a transcript is provided
- Picks text mode unless the system message clearly demands JSON (via markers)

Request model

```

4 class GPTSearchRequest(BaseModel):
5     query: str = Field(..., description="User query to send to GPT.")
6     system: Optional[str] = Field(
7         default=("You are a helpful assistant for a UK Tax Calculator project. "
8                 "Answer concisely and focus on practical steps, code, and formulas.")
9     )
10    temperature: float = Field(default=0.2, ge=0.0, le=1.0)
11    max_tokens: int = Field(default=600, ge=1, le=4000)
12    model: Optional[str] = Field(default=None, description="Override model name (optional).")
13
14    @validator("query")
15    def _trim(cls, v: str) -> str:
16        v = v.strip()
17        if not v:
18            raise ValueError("query must not be empty")
19        return v

```

Figure 56 AI Tax Adviser — request model

Response model

```

21 class GPTSearchResponse(BaseModel):
22     answer: str
23     model: str
24     created: int
25     usage: Optional[Dict[str, Any]] = None
26

```

Figure 57 AI Tax Adviser — response model

Transcript handling & mode detection

```

14 _JSON_MARKERS = ("json object", "json-only", "json-only", "response_format", "json_schema", "strict json")
15
16 _user_turn_re = re.compile(r'User:\s*(.*?)(?=\n(?:Assistant:|System:|$))', re.S)
17
18 def _extract_last_user_turn(text: str) -> str:
19     m = _user_turn_re.findall(text or "")
20     return (m[-1] if m else text).strip()
21
22 def _looks_like_json_task(s: Optional[str]) -> bool:
23     s = (s or "").lower()
24     return any(k in s for k in _JSON_MARKERS)

```

Figure 58 Transcript handling — last User: turn extractor

OpenAI client behavior & model defaults

- Cached client; reads OPENAI_API_KEY from settings (missing key → 502 at the router)

- Model selection: request's model or fallback to settings.openai_model (*default "gpt-4o-mini"*)
- Reasoning-model detection switches token parameter to max_completion_tokens
- JSON mode sets response_format={"type": "json_object"} with fallback retry if the model rejects it.

Core call (excerpt)

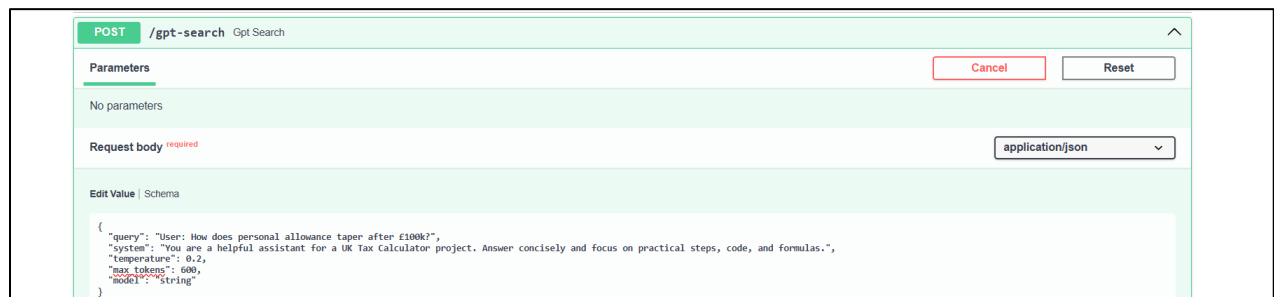
```

92 token_kw = "max_completion_tokens" if _is_reasoning_model mdl else "max_tokens"
93 params = {"model": mdl, "messages": messages, token_kw: max_tokens}
94 if not _is_reasoning_model mdl:
95     params["temperature"] = temperature
96 if mode == "json":
97     params["response_format"] = {"type": "json_object"} # or json_schema if provided
98 resp = client.chat.completions.create(**params)
    
```

Figure 59 Mode switch — JSON task detection markers

Error mapping

- **400** — empty/missing query
- **502** — OpenAI/model client errors (RuntimeError)
- **500** — any other exception



POST /gpt-search Gpt Search

Parameters

No parameters

Request body *required*

application/json

Edit Value | Schema

```

{
  "query": "User: How does personal allowance taper after £100k?",
  "system": "You are a helpful assistant for a UK Tax Calculator project. Answer concisely and focus on practical steps, code, and formulas.",
  "temperature": 0.2,
  "max_tokens": 600,
  "model": "gpt-4o-mini"
}
    
```

Figure 60 OpenAI call params — /gpt-search

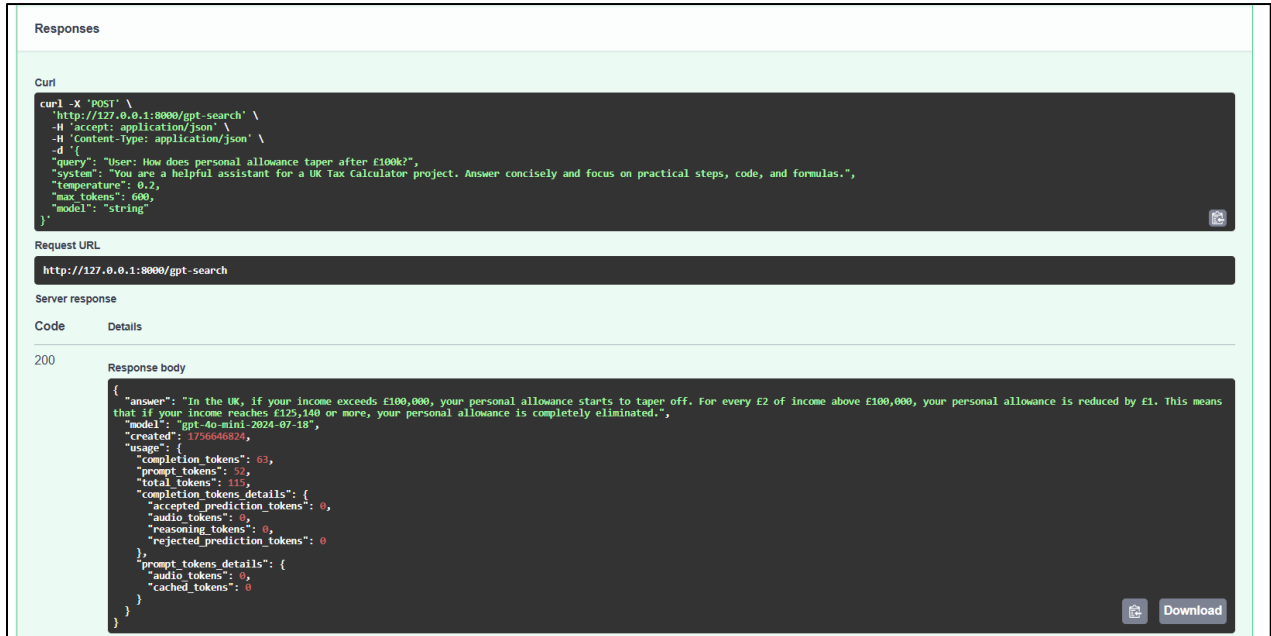


Figure 61 Response JSON shape — /gpt-search

6.4.3 Control-file Generation (Excel → .gcp)

Input handling

- Accepts .xlsx (openpyxl) and legacy .xls (*requires* xlrd==1.2.0) with explicit, helpful error messages.
- Validates presence of required headers; coerces numeric columns; normalizes booleans ("true"/"false").

Template & rule schedule

```

14 CONTROL_TEMPLATE = textwrap.dedent("""\
15 <
16   CreateSelfEmploymentIncome({SelfEmpRev:.2f}, {SelfEmpExp:.2f});
17   CreateDividendIncome({DivRev:.2f});
18   CreateEmploymentIncome({EmpRev:.2f});
19   CreateSavingsInterestIncome({SavRev:.2f});
20   inputParams({PensionContrib:.2f}, {CharitableDonations:.2f}, {ReliefAtSource}, {RegisteredBlind});
21 >
22 {rules}
23 """)
24
25 RULE_NAMES = [
26     "rule01_", "rule02_", "rule03_", "rule04_",
27     "rule05_", "rule06_", "rule07a_", "rule07b_", "rule08_",
28     "rule09_", "rule10_", "rule11_", "rule12_",
29     "rule13_", "rule14_", "rule15_",
30     "rule16_", "rule17a_", "rule17b_", "rule17c_", "rule17d_",
31     "rule18_", "rule19_", "rule20_", "rule21_",
32     "rule22a_", "rule22b_", "rule22c_"
33 ]
34 RULE_CALLS = "\n".join(f"try {{ {name}(); }} else {{}} " for name in RULE_NAMES)

```

Figure 62 .gcp control template — income + inputs block

Per Excel row (CaseID), to emit control_case_<CaseID>.gcp.

Single vs multiple cases

A one-row sheet = one case (supported by the same pipeline). For bulk testing, add more rows—one .gcp per row. The generate-only endpoint helps pre-stage many controls.

6.4.4 Generator Orchestration (Java)

Command

cwd defaults to the parent directory of <GRAPH_GPS_PATH> so state.gst lands where the parser expects it

Error policy

Non-zero exit, missing JAR, or timeout → raise GrooveRunError.

```
bash
java -jar Generator.jar -f state -s linear <GRAPH_GPS_PATH>
```

Figure 63 Generator CLI — Generator.jar state/linear invocation

Core (excerpt)

```
52 cmd = [java, "-jar", jar_path, "-f", "state", "-s", "linear", gps_path]
53 proc = subprocess.run(cmd, cwd=workdir, capture_output=True, text=True, timeout=timeout_sec)
54 if proc.returncode != 0:
55     raise GrooveRunError(...)
```

Figure 64 Java launch snippet — subprocess with timeout & errors

6.4.5 Result Extraction (state.gst)

After a successful run, the backend reads <parent_of_gps>/state.gst, finds the Person node by a self-loop labeled type:Person, and collects let:* attributes from self-loops on that node. Inputs (e.g., charitable_donations, pension_contributions) are excluded from the output map.

Core (excerpt)

```
graphs = root.findall("./gxl:graph", GXL_NS)
graph = graphs[-1] # final graph
person_id = _find_person_node(graph) or "n0"

for edge in graph.findall("gxl:edge", GXL_NS):
    if edge.get("from") == person_id and edge.get("to") == person_id:
        text = label_el.text.strip()
        if text.startswith("let:") and "=" in text:
            key = text[4:].split("=", 1)[0].strip().lower()
            if key not in _EXCLUDED:
                out[key] = _parse_value(text)
```

Figure 65 GXL parser — extract let:* on Person

6.4.6 Configuration & Defaults

Environment-driven (Pedantic Settings)

- OPENAI_MODEL (*default: "gpt-4o-mini"*)
- CONTROL_OUTPUT_DIR, JAVA_BIN *(*default "java"*)*, GROOVE_BIN_DIR, GENERATOR_JAR *(*default "Generator.jar"*)*, GRAPH_GPS_PATH (*required*), GENERATOR_TIMEOUT_SEC (*default 180*)
- CORS allow-list includes local dev origins and "*" (permissive)

6.4.7 Logging, Timings & Errors

- API returns generator.timings_sec (save, run, parse, total)
- Typical errors: 400 invalid request (e.g., missing headers); 404 state.gst not found; 500 generator/parsing/unexpected exception
- stdout/stderr truncation controlled by stdout_preview_lines (default 80) to keep responses small

6.4.8 Operational Notes & Limitations

- **Where files go:** control files are written directly into the .gps directory used by the generator (currently via a hard-coded path in build_controls_to_dir).
- **Multiple controls:** you can generate many .gcp files at once (one per case/row). In GROOVE, typically enable one control file at a time when executing. (Selection is done in GROOVE tooling; not enforced by the API.)
- **Single-case use:** fully supported — upload an Excel with one row to produce a single .gcp.

6.5 Frontend UI (Angular)

6.5.1 Overview & Tech Stack

The UI is a single-page Angular application composed of two standalone components—FormComponent (calculator) and GptAssistantComponent (Tax Copilot)—plus a thin HTTP wrapper (ControlApiService). It uses Reactive Forms for validation, ships without global state, and renders a split layout: inputs on the left, computed results on the right, with an optional chat drawer for advisory Q&A.

6.5.2 Component Architecture

- **FormComponent** (app-form): owns the calculator form, input visibility logic (based on selected income types), submission flow, result rendering, and persistence of the latest inputs/draft to localStorage.
- **GptAssistantComponent** (app-gpt-assistant): a floating, accessible side-drawer chat. Defaults to suggestion-style “text” answers and can be switched to strict JSON mode. It auto-resizes the textarea, supports Enter-to-send, and shows a live “typing” bubble.
- **ControlApiService**: posts an Excel payload to /generate-controls/save-and-run and, separately, proxies /gpt-search. No API keys or models are hard-coded in the UI; the displayed model name is taken from the server response.

6.5.3 Form Layout, Accessibility & Input Visibility

The header contains a tabbed toggle (Calculator / Experimental), and the form presents income-type “chips” that control which fields are visible. ARIA roles/labels are included for accessibility.

Snippet – Income type chips (HTML):

```

263 <div class="chip-group" role="group" aria-label="Income types">
264   <button type="button" class="chip"
265     *ngFor="let t of incomeTypes"
266     [class.chip--selected]="isIncomeTypeSelected(t.value)"
267     (click)="toggleIncomeType(t.value)"
268     role="checkbox"
269     [attr.aria-checked]="isIncomeTypeSelected(t.value)">
270     <span>{{ t.label }}</span>
271   </button>
272 </div>

```

Figure 66 Toggleable chips drive conditional field visibility and include ARIA states for screen readers.

Visibility is derived from a selectedIncomeTypes array; hiding a type resets its fields to 0 to avoid stale data.

Snippet – Toggle & reset on hide (TS):

```

327 toggleIncomeType(incomeType: string) {
328   if (this.isIncomeTypeSelected(incomeType)) {
329     this.selectedIncomeTypes = this.selectedIncomeTypes.filter(t => t !== incomeType);
330     const type = this.incomeTypes.find(t => t.value === incomeType);
331     type?.fields.forEach(f => this.form.get(f)?.setValue(0));
332   } else {
333     this.selectedIncomeTypes = [...this.selectedIncomeTypes, incomeType];
334   }
335 }

```

Figure 67 Component state controls visibility; hidden fields are zeroed to keep payloads clean.

6.5.4 Validation & Error Surfacing

The form uses Angular validators (non-negative numbers; required markers on pension/donations). Errors are summarised into a single status line to prevent noisy inline error UIs.

Snippet – Validation setup (TS):

```
63   this.form = this.fb.nonNullable.group({
64     CaseID: [this.defaults.CaseID, [Validators.required]],
65     SelfEmpRev: [this.defaults.SelfEmpRev, [Validators.min(0)]],
66     SelfEmpExp: [this.defaults.SelfEmpExp, [Validators.min(0)]],
67     DivRev: [this.defaults.DivRev, [Validators.min(0)]],
68     EmpRev: [this.defaults.EmpRev, [Validators.min(0)]],
69     SavRev: [this.defaults.SavRev, [Validators.min(0)]],
70     PensionContrib: [this.defaults.PensionContrib, [Validators.required, Validators.min(0)]],
71     CharitableDonations: [this.defaults.CharitableDonations, [Validators.required, Validators.min(0)]],
72     ReliefAtSource: [this.defaults.ReliefAtSource],
73     RegisteredBlind: [this.defaults.RegisteredBlind],
74   });
```

Figure 68 Minimal, explicit constraints; human-readable labels on error.

6.5.5 Results & provenance

Only populated metrics are shown, in a fixed domain order; Tax is always visible and emphasised. The UI also shows the generated control file path and collapsible stdout for reproducibility.

Snippet — ordering & table (TS/HTML):

```
80   private readonly DISPLAY_ORDER = [
81     'total_income', 'adjusted_net_income', 'net_relevant_earnings',
82     'personal_allowance', 'personal_savings_allowance', 'dividend_allowance',
83     'gift_aid_relief', 'pension_relief', 'amount_in_starting_savings',
84     'total_other_income', 'taxable_income', 'tax'
85   ];

238   <tbody>
239     <tr *ngFor="let r of resultRows; trackBy: trackByKey" [class.tax-row]="r.key === 'tax'">
240       <th scope="row">{{ r.label }}</th>
241       <td>{{ r.value | currency:'GBP':'symbol':'1.2-2' }}</td>
242     </tr>
243   </tbody>
```

Figure 69 Domain-aware ordering + currency formatting.

6.5.6 Tax Copilot (advisory chat)

A glass-blur side drawer with a floating launcher. Enter-to-send, typing indicator, auto-resizing textarea (up to 10 lines, then scroll). The payload includes an empty system in text mode so the server applies its default UK-tax prompt; the visible model badge comes from the server response (not hard-coded).

Snippet — payload & model badge (TS):

```

145     const payload = {
146       query: this.buildQueryForApi(text),
147       system: this.mode === 'json' ? this.systemPrompt : '',
148       temperature: this.temperature,
149       max_tokens: this.maxTokens,
150       model: this.model,
151     };
152
153     const res = (await firstValueFrom(this.api.gptSearch(payload))) as GPTSearchResponse;
154     this.meta = { model: res?.model, tokens: res?.usage?.total_tokens };

```

Figure 70 Mode-aware prompts; model name sourced from backend.

Snippet — auto-resize (TS):

```

276     autoResize() {
277       const ta = this.inputRef?.nativeElement; if (!ta) return;
278       ta.style.height = 'auto';
279       const maxHeight = this.MAX_ROWS * (parseFloat(getComputedStyle(ta).lineHeight) || 20);
280       ta.style.height = `${Math.min(ta.scrollHeight, maxHeight)}px`;
281       ta.style.overflowY = ta.scrollHeight > maxHeight ? 'auto' : 'hidden';
282     }

```

Figure 71 Keeps composer compact; adds scroll after 10 lines.

6.5.7 Accessibility, state & robustness

- **A11y:** ARIA roles on tabs, dialog, chip group, semantic table headers.
- **Keyboard:** Enter-to-send (Shift+Enter for newline).
- **State:** localStorage preserves last inputs and primes a ready-to-send draft for chat.
- **Timeouts:** 180s client guard for long GROOVE runs.

6.5.8 Limitations & extensions

- The calculator view currently targets the GROOVE pathway; an “Experimental” route can host side-by-side comparisons (GROOVE/Python/LLM) as described in the system overview.

- Provenance shows only the first written control file (single-case UX). Bulk cases are supported via Excel but are executed one control file at a time inside GROOVE tooling.

6.5.9 Integration Points (Backend Contracts)

- **POST** /generate-controls/save-and-run → expects an Excel file; responds with { written, generator, values }. The UI renders values and displays provenance from written/stdout.
- **POST** /gpt-search → { answer, model, usage? }; shown in the chat transcript with model badge and safe token accounting.

6.5.10 Figures (UI states used in evaluation)

- **Blank state (Calculator tab):**

The screenshot shows the 'UK Tax Calculator' interface. At the top, the title 'UK Tax Calculator' is displayed in a large, bold font. Below it, a subtitle reads 'Calculate your UK tax liability with ease'. There are two buttons: 'Calculator' (highlighted in blue) and 'Experimental BETA' (in a dark grey box). The main form area is divided into several sections. On the left, under 'Income types *', there are four buttons: 'Employment Income', 'Self-Employment Income', 'Savings Income', and 'Dividend Income'. Below this, there are two input fields for 'PENSION CONTRIBUTIONS *' and 'CHARITABLE DONATIONS *', both showing '£ 0'. At the bottom left, there are two checkboxes: 'Relief at Source' and 'Registered Blind'. At the bottom center, there is a 'Calculate' button. On the right side of the form, there is a 'Results' panel, which is currently empty.

Figure 72 Header + income chips; results panel ready (empty).

- **After calculation:**

UK Tax Calculator
Calculate your UK tax liability with ease

Calculator Experimental **BETA**

Income types *

Employment Income **Self-Employment Income** Savings Income Dividend Income

EMPLOYMENT REVENUE * £ 50000

PENSION CONTRIBUTIONS * £ 0

CHARITABLE DONATIONS * £ 0

☐ Relief at Source

☐ Registered Blind

Calculate

Results

Total Income	£50,000.00
Adjusted Net Income	£50,000.00
Net Relevant Earnings	£50,000.00
Personal Allowance	£12,570.00
Personal Savings Allowance	£1,000.00
Total Other Income	£37,430.00
Taxable Income	£37,430.00
Tax	£7,486.00

Generated control file
C:\Users\Saad Awaiz\Desktop\FYP\groove-project\FirstGraph\taxrules.gps\control_case_TC1.gcp

► Run details

Figure 73 Right-pane table shows populated metrics

- **With Tax Copilot open:**

UK Tax Calculator
Calculate your UK tax liability with ease

Calculator Experimental **BETA**

Income types *

Employment Income **Self-Employment Income** Savings Income Dividend Income

EMPLOYMENT REVENUE * £ 50000

PENSION CONTRIBUTIONS * £ 0

CHARITABLE DONATIONS * £ 0

☐ Relief at Source

☐ Registered Blind

Calculate

Results

Total Income	£50,000.00
Adjusted Net Income	£50,000.00
Net Relevant Earnings	£50,000.00
Personal Allowance	£12,570.00
Personal Savings Allowance	£1,000.00
Total Other Income	£37,430.00
Taxable Income	£37,430.00
Tax	£7,486.00

Generated control file
C:\Users\Saad Awaiz\Desktop\FYP\groove-project\FirstGraph\taxrules.gps\control_case_TC1.gcp

► Run details

Tax Copilot gpt-4o mini 2024-07-18

Tax year 2025/26 – current form inputs:

```
{
  "SelfEmpRev": 0,
  "SelfEmpExp": 0,
  "DivRev": 0,
  "EmpRev": 50000,
  "SavRev": 0,
  "PensionContrib": 0,
  "CharitableDonations": 0,
  "ReliefAtSource": false,
  "RegisteredBlind": false
}
```

how to reduce the tax
2:10 PM

👤 To reduce your tax liability for the 2025/26 tax year, consider the following options:

1. ****Pension Contributions**:** Contributing to a pension scheme can provide tax relief. The more you contribute, the lower your taxable income.
2. ****Charitable Donations**:** Making charitable donations can also reduce your taxable income. Consider donating to registered charities under Gift Aid.
3. ****Claim Expenses**:** If you have any allowable business expenses related to self-employment, ensure you claim them. This can reduce your taxable income.

Figure 74 Same results with advisory chat on the right

7 Experimental Methodology

7.1 Test Case Design (15 scenarios, coverage rationale)

Objective.

We curated 15 deterministic scenarios to exercise all major rule paths and edge conditions in the UK personal-income-tax pipeline (employment, self-employment profit, savings interest, dividends, pension relief at source, gift-aid uplift, blind-person's allowance, and personal-allowance taper). Inputs are fixed and auditable; expected outputs are pre-computed to support reproducible comparisons.

Inputs & artefacts.

Test inputs are defined in **tax_cases.xlsx** (Figure 72) with the schema: CaseID, SelfEmpRev, SelfEmpExp, EmpRev, SavRev, DivRev, PensionContrib, CharitableDonations, ReliefAtSource, RegisteredBlind.

CaseID	SelfEmpRev	SelfEmpExp	DivRev	EmpRev	SavRev	PensionContrib	CharitableDonat	ReliefAtSource	RegisteredBlind
TC1	0	0	0	30000	0	0	0	False	False
TC2	50000	10000	0	0	0	0	0	False	False
TC3	0	0	0	20000	800	0	0	False	False
TC4	0	0	0	20000	1200	0	0	False	False
TC5	0	0	2000	0	0	0	0	False	False
TC6	150000	0	0	0	0	0	0	False	False
TC7	0	0	0	40000	0	0	1000	False	False
TC8	0	0	0	30000	0	0	0	False	True
TC9	0	0	0	50000	0	5000	0	True	False
TC10	60000	20000	1000	60000	1500	3000	500	False	False
TC11	0	0	0	0	0	0	0	False	False
TC12	10000	2000	1000	40000	500	10000	2000	True	True
TC13	0	0	0	62840	0	0	0	False	False
TC14	0	0	1000	0	0	0	0	False	False
TC15	0	0	0	0	2000	0	0	False	False

Figure 75 Inputs for 15 test scenarios (schema + values)

Expected results used for validation are stored in **test_cases_with_results.xlsx** (Figure 73), including: adjusted_net_income, personal_allowance, blind_persons_allowance, taxable_income, tax (and other intermediate fields).

CaseID	SelfEmpRev	SelfEmpExp	EmpRev	SavRev	DivRev	PensionContrib	CharitableDono	ReliefAtSource	RegisteredBlind	adjusted_net_in	personal_allow	blind_persons_d	taxable_income	tax
TC1	0	0	30000	0	0	0	0	False	False	30000	12570	0	17430	3486
TC2	50000	10000	0	0	0	0	0	False	False	40000	12570	0	27430	5486
TC3	0	0	20000	800	0	0	0	False	False	20800	12570	0	8230	1646
TC4	0	0	20000	1200	0	0	0	False	False	21200	12570	0	8630	1726
TC5	0	0	0	0	2000	0	0	False	False	2000	12570	0	0	0
TC6	150000	0	0	0	0	0	0	False	False	150000	0	0	150000	53703
TC7	0	0	40000	0	0	0	1000	False	False	39750	12570	0	27430	5486
TC8	0	0	30000	0	0	0	0	False	True	30000	12570	3130	14300	2860
TC9	0	0	50000	0	0	5000	0	True	False	43750	12570	0	37430	7486
TC10	60000	20000	60000	1500	1000	3000	500	False	False	99375	12570	0	89930	28432
TC11	0	0	0	0	0	0	0	False	False	0	12570	0	0	0
TC12	10000	2000	40000	500	1000	10000	2000	True	True	36500	12570	3130	33800	6760
TC13	0	0	62840	0	0	0	0	False	False	62840	12570	0	50270	12568
TC14	0	0	0	0	1000	0	0	False	False	1000	12570	0	0	0
TC15	0	0	0	2000	0	0	0	False	False	2000	12570	0	0	0

Figure 76 Ground-truth outputs for each input (ANI, PA, taxable income, tax).

Scenario taxonomy.

The 15 cases are organised to provide broad coverage and boundary stress:

- **Baselines.**
 - Zero-income baseline.
 - Employment-only baseline in the basic-rate region.
- **Single-source income.**
 - Self-employment **profit** (revenue – expense).
 - Savings interest only (with boundary values).
 - Dividends only (around the dividend allowance).
- **Allowances & relief toggles.**
 - Relief-at-source pension contributions (gross-up and ANI reduction).
 - Gift-aid donations (1.25× grossing and ANI effect).
 - Blind-person’s allowance.
- **Boundary/threshold checks.**
 - **Starting-savings band** boundaries (e.g., £800 and £1,200 interest).
 - **Higher-rate crossover** at £62,840 employment income (fills £12,570 PA + £50,270 basic band).
 - **High income** (ANI ≥ £100,000) to trigger personal-allowance taper down to £0.
- **Mixed cases.**
 - Combinations across employment, self-employment, savings, and dividends, with and without pension/gift-aid toggles, to test rule ordering and interactions.

Design intent per case.

Each CaseID has a one-line purpose (Figure 74), e.g., “Employment-only (basic rate) baseline”, “Employment + savings £800 (starting-rate boundary)”, “Self-employment high income (£150k): PA taper→0”, etc. These labels make the rationale explicit and prevent accidental overlap.

CaseID	Scenario (one-liner)
TC1	Employment-only (basic rate) baseline
TC2	Self-employment profits (rev-exp)
TC3	Employment + savings £800 (starting-rate boundary)
TC4	Employment + savings £1200 (above starting-rate)
TC5	Dividends near allowance
TC6	Self-employment high income (£150k): PA taper→0
TC7	Employment + Gift Aid donation
TC8	Employment + Blind Person's Allowance
TC9	Employment + pension (relief at source)
TC10	Mixed: employment + self-employment + savings/dividends
TC11	Zero-income baseline
TC12	Employment + Blind Person's Allowance
TC13	Employment at basic-band cap (£62,840) boundary
TC14	Dividends within allowance
TC15	Mixed income

Figure 77 One-line scenario intent per CaseID (purpose/rationale).

Coverage evidence.

A **feature-coverage matrix** (Figure 75) marks, per case, whether it exercises: Employment, Self-Employment (profit>0), Savings>0, Dividends>0, PensionContrib>0, GiftAid>0, ReliefAtSource=True, Blind=True, HighIncome≥100k, ZeroIncome, and boundary flags (Sav=£800, Sav=£1200, ANI=£62,840). The matrix demonstrates that every allowance, band transition, and toggle is covered at least once, with deliberate boundary hits.

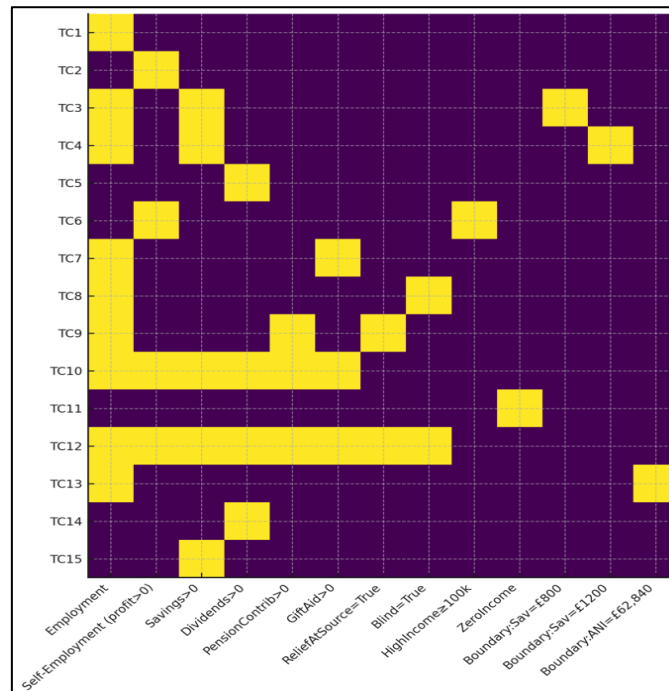


Figure 78 Coverage matrix showing which features/boundaries each case hits

How the artefacts are used.

Inputs from **tax_cases.xlsx** are sent via the backend to emit one .gcp per case; each case is then executed in isolation in GROOVE (one active control file at a time) to avoid cross-contamination. Baseline outputs are read from the let:* attributes on the Person node in state.gst. These values are manually compared against **test_cases_with_results.xlsx** (the ground truth). Section 7.2 then details execution across the four endpoints: the GROOVE baseline (/generate-controls/save-and-run), two deterministic Python checks (**Model A**, **Model B**), and the **GPT calculator** (/ai/tax-calc).

7.2 Execution Procedure

7.2.1 Scope & reference

This section describes how each test case is executed across four backend endpoints—the GROOVE baseline, two Python calculators (Model A, Model B), and the GPT calculator—and how the UI’s Experimental screen orchestrates the runs. Detailed GROOVE control/CLI/parsing steps are already documented; the reference is 6.3–6.4, so there is no need to repeat them.

Inputs. The same TaxInput schema is used for all paths; your 15 fixed scenarios come from tax_cases.xlsx.

7.2.2 Endpoints used (overview)

- **Baseline (GROOVE):** POST /generate-controls/save-and-run → emits .gcp, runs the Generator, parses state.gst, returns {written, generator, values}. (Full details in 6.3–6.4.)
- **Model A (Python “minimal-prompt” spec):** POST /model-a → deterministic calculator with simplified rules and 2025/26 constants.
- **Model B (Python “detailed-prompt” spec):** POST /model-b → deterministic calculator mirroring your NL rules.
- **GPT calculator (OpenAI):** POST /ai/tax-calc → no in-code math; enforces JSON-only output and reports usage/meta.

Code: GPT pipeline (what is executed)

Router for GPT calculator (async, threads offload; returns ai_json, ai_raw, _meta).

```

13 @router.post("/tax-calc")
14 async def ai_tax_calc(payload: TaxInput, gpt_model: str | None = None,
15                       gpt_temperature: float = 0.0, gpt_max_tokens: int = 800):
16     result = await to_thread.run_sync(run_ai_calc,
17                                     payload.dict(), gpt_model,
18                                     gpt_temperature, gpt_max_tokens)
19     return {"ai_json": result["json"], "ai_raw": result["raw"], "_meta": result["meta"]}
20

```

Figure 79 Router delegates to the AI core in a worker thread; response body is domain-only.

Message construction + strict JSON contract (system+user messages; schema keys; numeric rounding).

```

54 def _build_messages(payload: Dict) -> list[Dict[str, str]]:
55     system_msg = ("You are TaxCalc-LLM... respond ONLY with a single JSON object... round to 2 decimals...")
56     output_contract = { "total_income": "number", "net_relevant_earnings": "number", "tax": "number",
57                        "ai_confidence": "number between 0 and 1", "ai_reasoning": "short string" }
58     user_msg = {"instruction": "Analyze this case and return ONLY a JSON object...",
59               "input_payload": payload, "output_schema_contract": output_contract,
60               "tax_year_hint": payload.get("TaxYear", "unspecified")}
61     return [{"role": "system", "content": system_msg}, {"role": "user", "content": json.dumps(user_msg)}]

```

Figure 80 The AI is boxed into a deterministic output schema; any missing keys are post-filled with zeros.

Call OpenAI with JSON mode + fallback; token-pram switch for reasoning models.

```

155 model = _normalize_model(gpt_model) # fallback to settings.openai_model if placeholder
156 token_kw = "max_completion_tokens" if _is_reasoning_model(model) else "max_tokens"
157 params = {"model": model, "messages": messages, token_kw: gpt_max_tokens,
158          "temperature": gpt_temperature, "response_format": {"type": "json_object"}}
159 try:
160     completion = client.chat.completions.create(**params)
161 except Exception: # retry without response_format if model refuses
162     params.pop("response_format", None)
163     completion = client.chat.completions.create(**params)

```

Figure 81 Enforces JSON; gracefully retries if the model rejects the format.

Output sanitization & meta (strip AI-only keys; keep domain only; attach model + usage).

```

179 ai_json = _validate_and_patch_result(ai_json) # coerce numbers, round(.,2), fill missing keys
180 public_json = {k: ai_json.get(k, 0.0) for k in _NUM_FIELDS}
181 meta = {"ai": {"model": completion.model, "tokens_used": _usage_to_tokens(completion.usage),
182              "temperature": gpt_temperature, "confidence": ai_json.get("ai_confidence", 0.0),
183              "reasoning": ai_json.get("ai_reasoning", "")}, "ai_service": "OpenAI"}

```

Figure 82 Body is domain-only; all AI diagnostics live in _meta

Code: Python baselines (deterministic)

Model A endpoint + core constants (2025/26).

```

69 router = APIRouter(prefix="", tags=["models"])
70 PA_STANDARD, PA_TAPER_START, PA_ZERO_AT = D("12570"), D("100000"), D("125140")
71 BASIC_BAND_WIDTH, RATE_BASIC, RATE_HIGHER, RATE_ADDITIONAL = D("37700"), D("0.20"), D("0.40"), D("0.45")
72 SAVINGS_STARTING_RATE_BAND, PSA_BASIC, PSA_HIGHER, PSA_ADDITIONAL = D("5000"), D("1000"), D("500"), D("0")
73 DIVIDEND_ALLOWANCE = D("500")
74
75 @router.post("/model-a")
76 def model_a_calc(inp: TaxInput) -> Dict[str, float]:
77     se_profit = max(D("0"), D(inp.SelfEmpRev) - D(inp.SelfEmpExp)) # floor losses at 0 (lite model)
78     # ... compute ANI, PA taper, savings window, PSA/div allowance, bands, tax ...

```

Figure 83 “Lite spec” with explicit thresholds; trading losses floored; blind allowance reserved (not applied in Model A)

Model B endpoint (rule-faithful).

```

37 @router.post("/model-b")
38 def model_b(p: TaxInput) -> Dict[str, float]:
39     se_net, emp_net, sav_net, div_net = max(0, p.SelfEmpRev - p.SelfEmpExp), max(0, p.EmpRev), max(0, p.SavRev), max(0, p.DivRev)
40     total_income = se_net + emp_net + sav_net + div_net
41     nre = se_net + emp_net
42     pension_gross = (p.PensionContrib*1.25) if p.ReliefAtSource else p.PensionContrib
43     pension_relief = min(pension_gross, nre)
44     gift_aid_relief = 1.25*p.CharitableDonations if p.CharitableDonations>0 else 0
45     ani = max(0, total_income - pension_relief - gift_aid_relief)
46     pa = 0 if ani>125_140 else (12_570 if ani<=100_000 else max(0, 12_570 - 0.5*(ani-100_000)))
47     blind = 3_130.0 if p.RegisteredBlind else 0.0
48     # ... starting savings, PSA/div allowance, bands, tax ...

```

Figure 84 Mirrors your rule text and thresholds exactly (PA taper, blind allowance, savings starting rate, etc.).

7.2.3 Prompts used to derive the models

Prompt A (Model A — “minimal prompt”, created here)

Use this to justify the simplified Model A implementation (loss-flooring; omit blind-allowance logic; still include taper/PSA/div allowance/starting-savings):

You are an assistant generating a compact Python function for a UK Income Tax calculator for tax year 2025/26 (England/Wales/Ni; exclude Scotland). Use inputs: {CaseID, SelfEmpRev, SelfEmpExp, EmpRev, SavRev, DivRev, PensionContrib, CharitableDonations, ReliefAtSource, RegisteredBlind}.

Prompt B (Model B — “detailed prompt”, provided by you; verbatim)**Task**

Write production-quality Python code that calculates UK Income Tax for the 2025/26 tax year, implementing exactly the rules below. Use only the thresholds and logic stated here. Do not add extra allowances or country variants.

Inputs (dictionary keys)

- EmpRev — employment gross income
- SelfEmpRev — self-employment revenue
- SelfEmpExp — self-employment expenses
- SavRev — savings interest received
- DivRev — dividend income received
- PensionContrib — total pension contributions paid by the individual
- CharitableDonations — Gift Aid donations paid by the individual
- ReliefAtSource — boolean: True if pension is relief at source, False if net pay
- RegisteredBlind — boolean: True if taxpayer is registered blind

Computation rules**1. Setup (create sources)**

Treat employment income with net equal to EmpRev. Treat self-employment income with net equal to $\max(0, \text{SelfEmpRev} - \text{SelfEmpExp})$. Treat savings interest with net equal to SavRev. Treat dividend income with net equal to DivRev.

2. Build totals from sources

Set total income to the sum of employment net, self-employment net, savings net, and dividend net.

3. Net Relevant Earnings (NRE)

Set NRE to employment net plus self-employment net.

4. Reliefs

If ReliefAtSource is False (net pay), set pension_relief to $\min(\text{PensionContrib}, \text{NRE})$. If ReliefAtSource is True (relief at source), gross contributions by 25% and set pension_relief to $\min(1.25 \times \text{PensionContrib}, \text{NRE})$. If CharitableDonations is greater than zero, set gift_aid_relief to $1.25 \times \text{CharitableDonations}$; otherwise set it to 0.

5. Adjusted Net Income (ANI) [Rule 09]

Compute ANI as total income minus pension_relief minus gift_aid_relief.

6. Personal Allowance (PA) taper

If ANI is at most £100,000, set PA to £12,570. If ANI is between £100,000 and £125,140 (exclusive), set PA to $\£12,570 - 0.5 \times (\text{ANI} - \£100,000)$. If ANI is at least £125,140, set PA to £0. Clamp PA to a minimum of £0 in all cases.

7. Blind Person’s Allowance (BPA) [Rule 13]

If RegisteredBlind is True, set blind_persons_allowance to £3,130; otherwise set it to £0.

8. Taxable income [Rule 14]

Compute taxable_income as $\max(0, \text{ANI} - (\text{PA} + \text{blind_persons_allowance}))$.

9. Savings: other income and starting-rate slice

If both employment net and self-employment net are greater than zero, set other_income to $\max(0, \text{employment net} + \text{self-employment net} - (\text{PA} + \text{blind_persons_allowance}))$; otherwise set other_income to 0. Then set starting_rate_amount to $\min(\max(0, £5,000 - \text{other_income}), \text{savings net})$. If savings net is greater than zero and other_income is at most £5,000, treat starting_rate_amount as a 0% savings band.

10. Create income tax bands (apply after subtracting the starting-rate amount)

Let taxable_after_starting be $\max(0, \text{taxable_income} - \text{starting_rate_amount})$. Treat the first £37,700 of taxable_after_starting as the BASIC band at 20% when taxable_after_starting is positive. Treat the portion from £37,700 to £125,140 as the HIGHER band at 40% when taxable_after_starting exceeds £37,700. Treat any amount above £125,140 as the ADDITIONAL band at 45%.

11. Savings and dividend allowances

Set the Personal Savings Allowance based on the highest band present after step 10: £1,000 if a BASIC band exists, £500 if a HIGHER band exists and no BASIC band exists, and £0 if an ADDITIONAL band exists and neither BASIC nor HIGHER exists. If DivRev is greater than zero, set dividend_allowance to £500; otherwise set it to 0.

12. Compute tax from bands (excluding the starting-rate savings slice)

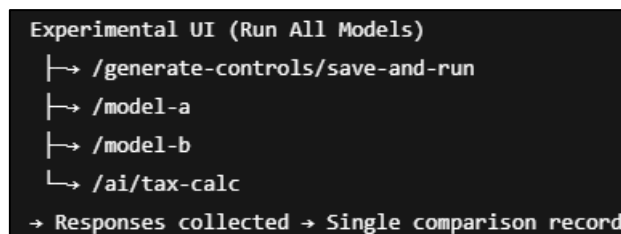
Apply rates only to taxable_after_starting, because starting_rate_amount is taxed at 0%. Compute 20% on the BASIC slice, 40% on the HIGHER slice, and 45% on the ADDITIONAL slice. Reduce the savings portion by the Personal Savings Allowance before applying rates, and reduce the dividend portion by the dividend_allowance before applying rates.

Outputs (return as a dictionary)

- total_income
- net_relevant_earnings
- pension_relief
- gift_aid_relief
- adjusted_net_income
- personal_allowance
- blind_persons_allowance
- taxable_income
- total_other_income (the “other_income” used for the starting rate)
- amount_in_starting_savings (the starting_rate_amount)
- personal_savings_allowance
- dividend_allowance
- tax (total income tax due, excluding NICs)

Procedure (per case) — Single Experimental Interface

1. Enter one case in the Experimental form (the 10 TaxInput fields).
2. Click “Run All Models.” The interface validates the form and serializes the same JSON payload for all backends.
3. Dispatch four calls (same payload):
 - POST /generate-controls/save-and-run (GROOVE baseline)
 - POST /model-a (Python Model A)
 - POST /model-b (Python Model B)
 - POST /ai/tax-calc (GPT calculator; model/params taken from the UI’s current settings)
4. Collect responses and metadata from each call (including any timing/usage fields returned by the endpoints).
5. Store as one comparison record: the single input plus the four endpoint responses are saved together for later analysis in 7.3.
6. Reference details: GROOVE control/run/parsing mechanics are documented in 6.3–6.4 (not repeated here).

*Figure 85 Single Experimental User Interface***7.3 Metrics (Agreement with GROOVE Baseline)****7.3.1 Scope**

This section evaluates AI, Model A, and Model B against the GROOVE baseline for the tax field across 15 manually-executed test cases (entered one-by-one via the Experimental screen in 7.2). All numbers are computed directly from the captured JSON outputs consolidated into TC-Results.csv; no synthetic data or imputation is used.

7.3.2 Computation

For each case, the absolute error is $|y^{\wedge} - y|$, where y is the GROOVE tax and $y^{\wedge}$ is the comparator's tax.

Summary metrics are:

- **Mean Absolute Error (MAE)** over the 15 cases, and
 - **Total Absolute Error** (sum of absolute errors).
- The bar charts in Figures 82 and 83 display these values; Figure 84 illustrates the per-case absolute errors.

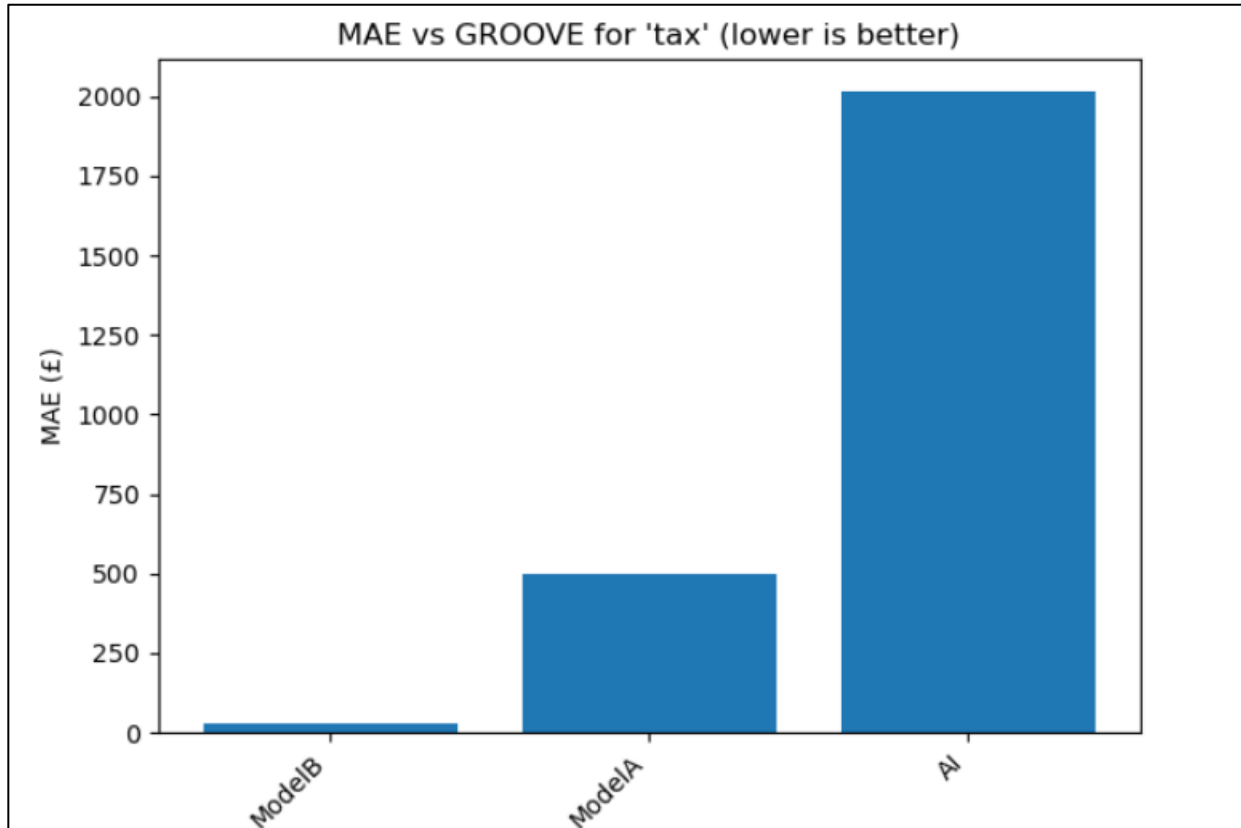


Figure 86 MAE (tax): B £26.67; A £500.20; AI £2,015.13.

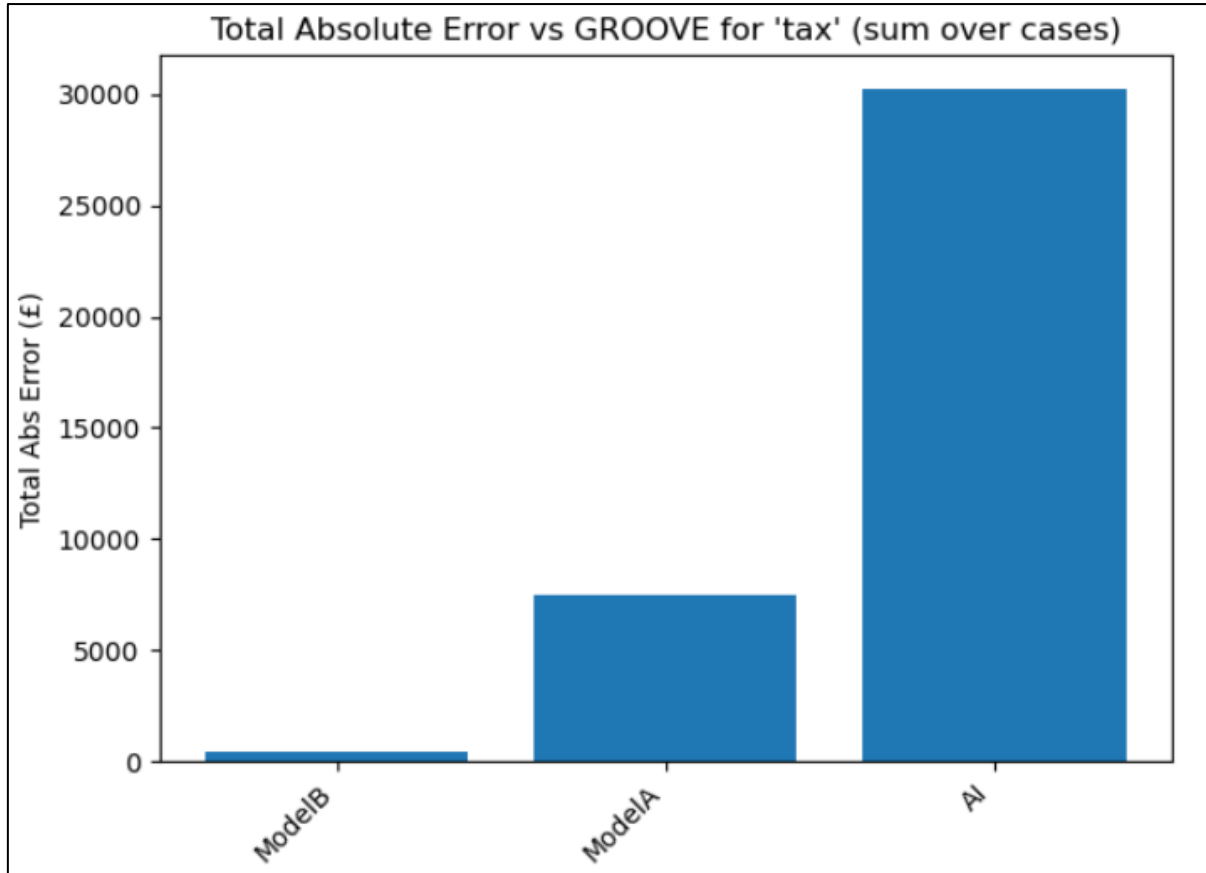


Figure 87 Total error: AI £30,227; A £7,503; B £400.

Headline results (tax).

- MAE: Model B $\approx$ £26.67, Model A $\approx$ £500.20, AI $\approx$ £2,015.13.
Interpretation: Model B tracks the baseline closely; Model A shows moderate deviation; AI's mean is driven by a few large outliers.
- Total absolute error: AI $\approx$ £30,227, Model A $\approx$ £7,503, Model B $\approx$ £400.
- Exact matches (0 error): AI 7/15, Model A 7/15, Model B 13/15.
- Error direction (signed vs GROOVE):
AI—4 overestimates, 4 underestimates; Model A—6 overestimates, 2 underestimates;
Model B—0 overestimates, 2 underestimates.
- Robustness (median absolute error): AI £2, Model A £160, Model B £0.

Per-case behaviour

- AI shows two dominant spikes: TC6 $\approx$ £13,703 and TC10 $\approx$ £10,146.

- Model A has moderate peaks at TC12 $\approx$ £3,369.75, TC9 $\approx$ £1,250, TC10 $\approx$ £1,018.75.
- Model B remains near-zero, with visible error only at TC3 $\approx$ £160 and TC4 $\approx$ £240.

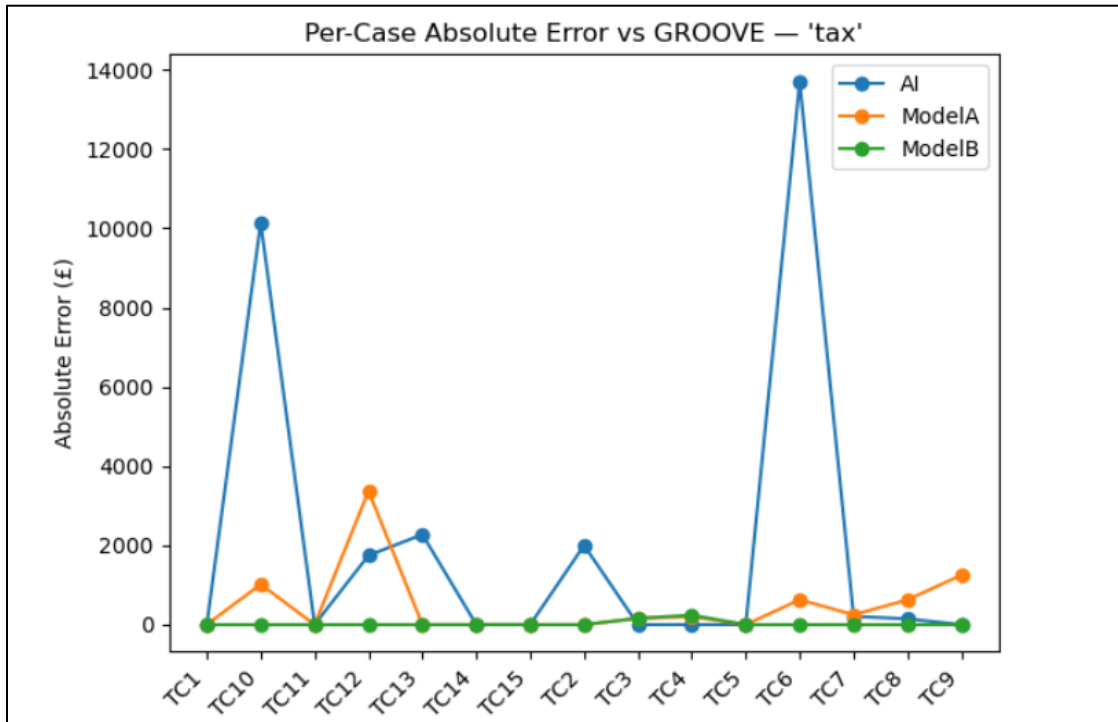


Figure 88 Per-case errors: AI peaks TC6, TC10; A moderate; B $\sim$ 0 (TC3, TC4).

UI cross-checks (linking cause to effect).

- TC1 (agreement). All models and GROOVE produce tax = £3,486. Minor field-level highlights (e.g., Personal Savings Allowance) do not affect the final tax in this case—useful as a sanity check that not every intermediate difference propagates.

Experimental Comparison
Compare multiple AI models side-by-side with identical inputs

Calculator Experimental **BETA**

SELF-EMPLOYED REVENUE * £ 0

SELF-EMPLOYED EXPENSES * £ 0

EMPLOYMENT REVENUE * £ 30000

SAVINGS REVENUE * £ 0

DIVIDEND REVENUE * £ 0

PENSION CONTRIBUTIONS * £ 0

CHARITABLE DONATIONS * £ 0

☐ Relief at Source

☐ Registered Blind

Run All Models

Completed.

Model Comparison Results

MODEL	AI Tax	Model A	Model B	Groove
personal_allowance	£12,570.00	£12,570.00	£12,570.00	£12,570.00
adjusted_net_income	£30,000.00	£30,000.00	£30,000.00	£30,000.00
total_income	£30,000.00	£30,000.00	£30,000.00	£30,000.00
net_relevant_earnings	£30,000.00	£30,000.00	£30,000.00	£30,000.00
personal_savings_allowance	£1,000.00	0	£1,000.00	£1,000.00
taxable_income	£17,430.00	£17,430.00	£17,430.00	£17,430.00
tax	£3,486.00	£3,486.00	£3,486.00	£3,486.00

Figure 89 TC1 UI: all models tax £3,486.

- TC2 (disagreement). AI sets adjusted_net_income = £50,000 versus £40,000 for Model A/Model B/GROOVE, yielding AI tax £7,486 versus £5,486 for the others. The UI screenshot makes the upstream cause of the tax gap explicit.

Experimental Comparison
Compare multiple AI models side-by-side with identical inputs

Calculator Experimental **BETA**

SELF-EMPLOYED REVENUE * £ 50000

SELF-EMPLOYED EXPENSES * £ 10000

EMPLOYMENT REVENUE * £ 0

SAVINGS REVENUE * £ 0

DIVIDEND REVENUE * £ 0

PENSION CONTRIBUTIONS * £ 0

CHARITABLE DONATIONS * £ 0

☐ Relief at Source

☐ Registered Blind

Run All Models

Completed.

Model Comparison Results

MODEL	AI Tax	Model A	Model B	Groove
personal_allowance	£12,570.00	£12,570.00	£12,570.00	£12,570.00
adjusted_net_income	£50,000.00	£40,000.00	£40,000.00	£40,000.00
total_income	£50,000.00	£40,000.00	£40,000.00	£40,000.00
net_relevant_earnings	£50,000.00	£40,000.00	£40,000.00	£40,000.00
personal_savings_allowance	0	0	£1,000.00	£1,000.00
taxable_income	£37,430.00	£27,430.00	£27,430.00	£27,430.00
tax	£7,486.00	£5,486.00	£5,486.00	£5,486.00

Figure 90 TC2 UI: AI ANI £50k (others £40k); tax £7,486 (others £5,486).

Note. All 15 test cases were executed manually through the Experimental interface; the resulting JSON responses were saved and aggregated into TC-Results.csv, from which Figures 82–84 were generated. The UI screenshots (Figures 85–86) are taken directly from those manual runs.

7.4 Critical Review

Model B’s Two Anomalous Cases (Edge Case Handling)

- Context:** Model B matched the GROOVE baseline on 13 out of 15 test cases, deviating only in 2 cases (TC3 and TC4) by a small margin (~£160 and ~£240 difference). Initially, these appear as Model B’s “failures” to predict the correct tax in those cases, since its outputs didn’t align with the baseline.
- Root Cause – Savings Starting-Rate Band:** The discrepancy was traced to how savings interest was handled. Model B’s generated code included an extra step to account for the 0% *starting rate for savings* band – a rule in UK tax law whereby up to £5,000 of savings interest can be taxed at 0% if the individual’s other income is low [28]. In Model B’s logic, any savings interest eligible for this 0% rate (the *starting_rate_amount*) is subtracted from taxable income before applying the normal tax bands. In other words, Model B recognized that the “starting-rate savings slice” should be excluded from the income subject to basic/higher/additional tax rates, ensuring that portion is not taxed (since it falls into a 0% band).
- Baseline Omission:** The GROOVE pipeline’s rule set did not explicitly implement this starting-rate adjustment. The natural-language rules fed into GROOVE covered personal allowances and savings/dividend allowances, but they did not include the rule for the 0% starting-rate band on savings interest. As a result, the baseline inadvertently taxed the savings interest that should have been tax-free. The extra ~£160 and £240 of tax in TC3/TC4 correspond directly to the baseline taxing portions of savings interest at 20% (basic rate) that should have been exempt under the starting-rate rule. Model B, by subtracting that portion out, calculated a lower (more accurate) tax for those cases.
- Interpretation:** What initially looked like Model B under-predicting the tax was actually Model B correctly applying tax law, versus a small oversight in the baseline. The differences highlight a minor gap in the hand-crafted GROOVE model rather than an error in Model B. In other words, Model B’s “failure” to match in those two cases is not a failure of the LLM at all – it’s the baseline failing to incorporate all relevant rules. Had the GROOVE pipeline included the same logic (excluding the starting-rate savings slice from taxable income before banding), Model B would have exactly matched the baseline on all 15 cases.
- Critical Insight:** This outcome underscores an important point in our evaluation. Rather than diminishing confidence in Model B, the two discrepancies actually increase it, because Model B demonstrated the ability to handle a nuanced edge case that the baseline missed. The LLM, given the same rule inputs, was *intelligent* enough to infer or include the starting-rate-for-savings logic on its own. Thus, these cases provide a valuable critical review: they show that when the AI model disagrees with a reference system, it may be highlighting an improvement or correction. In summary, Model B’s slight deviations were due to enhanced rule coverage (not error), revealing how the AI could augment the rule-based system by catching subtle tax logic that had been overlooked.

8 Recommendations and Further Research

The system's promising results open several avenues for improvement and expansion. This section outlines key recommendations in light of the project's scope, technical constraints, and emerging industry trends:

- Broader Tax Rule Coverage:** Extend the rule base to encompass a wider spectrum of UK tax regulations beyond the curated personal-income subset. Currently, only core rules (income identification, personal allowances, reliefs, and basic band logic) are encoded, leaving many statutory provisions unaddressed. Future work should incorporate additional allowances (e.g., marriage or charitable allowances), deductions, and edge-case scenarios to improve completeness. Notably, the project *itself* anticipates broader rule coverage as a logical next step. As rule sets expand, emphasis should be placed on maintaining the graph model's performance and tractability – ensuring that GROOVE's state-space exploration remains efficient even as rule count and complexity grow. This may involve modularizing the rule set or using control programs to manage rule application order so that the system can handle more complex tax situations without combinatorial blow-up [29].
- Automated Natural Language to Rule Conversion:** A significant research opportunity is to automate the translation of natural-language tax laws or guidance into formal rule representations. In the current project, rules were manually curated and then encoded into a JSON schema before being converted to UML and GROOVE formats. To scale this process, future research could leverage Natural Language Processing (NLP) and LLMs to parse legislative text or human-written rules into the JSON rule schema. The abstract already highlights “automated NL rule conversion” as a future extension. Achieving this would dramatically reduce the manual effort required to update the system when laws change or to port it to new domains. For example, an LLM could be prompted with a specific tax regulation and asked to output a structured rule schema, possibly using few-shot examples or retrieval of similar known rules to guide it. However, technical constraints must be addressed: LLM-generated rules would need rigorous validation, as subtle errors in a formal rule can lead to incorrect tax outcomes. Incorporating retrieval-augmented generation (RAG) techniques – where the model is supplied with the actual legislative text – may improve fidelity given evidence that providing controlling texts boosts accuracy in tax QA tasks. A hybrid approach might have the LLM draft candidate rules which are then checked by domain experts or by automated tests (ensuring they replicate known scenarios correctly) before acceptance. This line of research aligns with broader trends in AI, aiming to *combine* the fluency of LLMs with the reliability of symbolic methods [1][2][30][31].
- Enhanced User Feedback and Explainability in the UI:** While the prototype UI already presents side-by-side results and highlights differences (even offering visual graph diffs), there is room to improve how feedback is communicated to end users. In a high-stakes

domain like tax, users (e.g. accountants or taxpayers) need clear explanations of *why* a discrepancy occurred. Future iterations of the UI should include natural-language explanations that accompany the raw diff. For example, if the GPT answer’s tax calculation diverges from the rule-based baseline, the system could pinpoint which rule or parameter caused the difference (e.g. “GPT did not apply the personal allowance taper correctly, resulting in a £X higher tax”). This could be achieved by mapping graph transformation steps back to human-readable justifications. Additionally, integrating interactive exploration features would be valuable – users might click on a highlighted discrepancy to see the relevant portion of the rule (with citation to tax law) or to view the specific graph rule that was triggered. Improving the UI’s accessibility in this manner would bridge the gap between formal verification and user understanding, thereby increasing trust. Ensuring that the feedback mechanism is intuitive is crucial for adoption; for instance, in a commercial tax software context, any flagged errors from the AI should be explained in plain terms for the end-user. Given that the current UI is single-scenario oriented, another improvement is to support batch processing and summary reports – if an accountant validates multiple cases, the system could provide an aggregate report of all AI vs. rule discrepancies and confirmations, suitable for audit documentation [32][33].

- Scalability to Other Jurisdictions and Domains:** The principles of this system are not limited to UK personal income tax. A clear next step is to evaluate how well the approach generalizes to other tax jurisdictions (such as Scottish income tax with different bands, or entirely different countries’ tax systems) and even to other legal domains (like corporate tax, sales tax, or beyond finance to areas such as regulatory compliance). Scaling to a new jurisdiction would involve building a new set of graph rules reflecting that jurisdiction’s laws, which is feasible given the modular nature of the rule schema but potentially labor-intensive without the aforementioned NL→rule automation. It would also test the flexibility of the domain model – for instance, other countries might have different concepts (additional tax bands, credits, alternative minimum tax, etc.) requiring an expanded type graph. The *deterministic validator* concept, however, should remain applicable: by encoding local laws as graph transformation rules, one could validate an LLM’s answers against any formally specified legal logic. This cross-jurisdiction scaling would broaden the system’s usefulness and prove its robustness. Additionally, handling multiple jurisdictions might require the UI to allow selection of a rule set or tax year, which introduces an engineering consideration for maintainability. Careful governance of rule versions will be needed as laws change across years or differ between locales – a version-controlled library of rule sets could be one solution. Successful scaling would demonstrate that the project’s methodology can serve as a general framework for LLM output validation in regulated domains, not just UK tax [29][30].
- Integration and Model Risk Governance:** For real-world adoption, especially in enterprise or government settings, the system should be aligned with model risk

management practices. Regulatory guidance like the UK’s Prudential Regulation Authority (PRA) *SS1/23* and the impending EU AI Act stress the importance of transparency, auditability, and control in AI systems. The current approach already offers a high degree of transparency – each calculation is deterministic, traceable, and auditable by design, with step-by-step visual change logs that tie outcomes to specific rules. To further satisfy governance requirements, one recommendation is to implement robust audit trails and documentation for the LLM’s involvement: for every query answered, the system could log the exact prompt, the model’s identity and version, the rule set version used for validation, and the full comparison output. This would facilitate external review or compliance audits. Another idea is to integrate model monitoring and fallback procedures: for example, if the GPT component produces outputs that frequently conflict with the rules or violate the JSON schema, the system might flag the LLM as unreliable for certain inputs and either switch to a safer model or defer to the rule-based computation alone. In addition, exploring explainable AI (XAI) techniques for the LLM itself could complement the graph validation – e.g. requiring the LLM to provide a rationale for each step in its calculation (perhaps via chain-of-thought prompting) and then comparing that rationale to the rule-based steps. Such features would further demystify the AI’s behavior, addressing the broader concern that LLMs remain opaque in their reasoning. Finally, partnerships with industry (for instance, incorporating this system as a validation layer in professional tax software or HMRC’s own tools) would be a practical way to refine the system. Early user testing in those contexts could reveal domain-specific needs – such as support for official audit exports or integration with existing workflows – ensuring that the research evolves toward a deployable solution under real-world constraints[31][6], and remains aligned with official rates/allowances guidance [34].

Collectively, these recommendations aim to strengthen the system’s accuracy, generality, and trustworthiness. By expanding legal coverage, automating rule formalization, enhancing explainability, and aligning with governance norms, the project can transition from a successful prototype into a robust framework for reliable AI-assisted tax advice. Each proposed direction leverages the project’s core insight – the synergy of LLM-driven natural language reasoning with rigorous graph-based validation – pushing it closer to real-world applicability and future scholarly advancement.

9 Conclusion

This project set out to bridge the gap between the explanatory power of large language models and the strict correctness requirements of tax law. In high-level terms, it combined an LLM-based conversational tax assistant with a graph-rewriting validation engine to check that the assistant’s answers complied with formal UK tax rules. The methodology centered on encoding a subset of personal income tax regulations as executable graph transformation rules (in GROOVE) and developing an end-to-end pipeline (JSON rules → UML diagrams → GROOVE graphs) to operationalize those rules. The GPT model, prompted to produce structured JSON outputs, provided user-friendly natural-language reasoning, while the graph-based component served as a ground-truth calculator. By comparing, the LLM’s computed results against the graph-derived results; the system could identify discrepancies and present them back to the user. This novel architecture demonstrated a viable approach to validating AI-generated answers against legal requirements, offering a level of assurance and transparency seldom found in end-to-end LLM solutions.

The findings confirm the feasibility and value of this hybrid approach. Key contributions of the work include: **(i)** the definition of a structured JSON rule schema for tax logic (inspired by an emerging GenAI exchange format proposal [35]) which ensures that a single source of truth (the JSON rule set) propagates consistently into both human-readable diagrams and machine-executable rules; **(ii)** an automated JSON→UML→GROOVE toolchain and supporting pipeline, including a visual “diff” mechanism that highlights changes in the graph state across rule applications (providing an explainable trace of how tax outcomes are derived and where the LLM’s answer deviated), as well as a prototype web interface that integrates the LLM and rule engine for real-time user interaction; and **(iii)** the integration of multiple computation models (the GPT-based assistant, a graph transformation model, and a Python baseline) to enable side-by-side evaluation of results, so that AI-generated answers can be cross-verified against reliable rule-based calculations. Through a series of scenario-based tests, the system was able to flag incorrect LLM outputs – for example, catching misapplication of allowances – while confirming correct calculations in other cases, illustrating the potential for semi-automated auditing of AI-driven advice.

At the same time, the project has illuminated some important limitations. The implemented rule set, while representative, covers only a fraction of the full tax code; expanding beyond the current scope will require careful rule management and potentially more advanced automation. The rules themselves can be brittle – they rely on specific assumptions and thresholds (e.g. hard-coded 2025/26 values), so they must be kept updated and might require refinement to handle unanticipated inputs. Moreover, the LLM component is non-deterministic and occasionally prone to formatting or reasoning errors, meaning that consistency of results is not guaranteed unless one employs strategies like robust prompting or result verification. These challenges temper the results but also suggest clear directions for future improvements, such as incorporating a broader rule base, adding self-checking mechanisms for the AI output format, and automating the inclusion of new tax provisions with minimal manual effort.

In summary, the project successfully demonstrates an innovative union of AI and formal methods: using graph transformation systems to enforce compliance on an otherwise black box, AI has output. This approach delivers explainable and auditable tax computations, addressing a vital need in the era of AI-driven finance and law. While currently limited in scope, it highlights a path forward

where hybrid AI systems marry the convenience of natural-language reasoning with the rigor of rule-based verification. Such systems could become increasingly important as the demand for trustworthy, transparent AI grows in domains like finance and legal compliance. By augmenting powerful LLMs with transparent, law-grounded reasoning, we move closer to AI applications that are not only intelligent and fluent, but also reliable and legally sound – a promising trajectory for the future of explainable AI in tax and beyond.

References

- [1] J. J. Nay *et al.*, “Large language models as tax attorneys: a case study in legal capabilities emergence,” *Philosophical transactions - Royal Society. Mathematical, Physical and engineering sciences/Philosophical transactions - Royal Society. Mathematical, physical and engineering sciences*, vol. 382, no. 2270, Feb. 2024, doi: <https://doi.org/10.1098/rsta.2023.0159>.
- [2] P. Lewis *et al.*, “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” Apr. 2021. Available: <https://arxiv.org/pdf/2005.11401>
- [3] Y. Dong *et al.*, “Generalization or memorization: data contamination and trustworthy evaluation for large language models,” *Findings of the Association for Computational Linguistics: ACL 2022*, pp. 12039–12050, Jan. 2024, <https://aclanthology.org/2024.findings-acl.716/>.
- [4] G. D. Service, “Income Tax rates and Personal Allowances,” *GOV.UK*, Jan. 26, 2012. <https://www.gov.uk/income-tax-rates>
- [5] K. Smith, “Number of people caught in 60% ‘tax trap’ up 45% in two years,” *@FinancialTimes*, Dec. 05, 2024. <https://www.ft.com/content/40f3b683-9853-4d86-91b5-19b23fdcf554> (accessed Aug. 14, 2025).
- [6] “SS1/23 – Model risk management principles for banks,” *Bankofengland.co.uk*, Jan. 21, 2025. <https://www.bankofengland.co.uk/prudential-regulation/publication/2023/may/model-risk-management-principles-for-banks-ss>
- [7] A. A. Gikay, “Risks, innovation, and adaptability in the UK’s incrementalism versus the European Union’s comprehensive artificial intelligence regulation,” *International Journal of Law and Information Technology*, vol. 32, no. 1, Jan. 2024, <https://academic.oup.com/ijlit/article/doi/10.1093/ijlit/eaee013/7701544>.
- [8] Amir Hossein Ghamarian, Maarten de Mol, A. Rensink, E. Zambon, and M.V. Zimakova, “Modelling and analysis using GROOVE,” *International Journal on Software Tools for Technology Transfer*, vol. 14, no. 1, pp. 15–40, Mar. 2011, doi: <https://doi.org/10.1007/s10009-011-0186-x>.
- [9] “Best practices for prompt engineering with the OpenAI API | OpenAI Help Center,” *Openai.com*, 2020. <https://help.openai.com/en/articles/6654000-best-practices-for-prompt-engineering-with-the-openai-api>
- [10] G. Engels, Claus Lewerentz, Wilhelm Schäfer, A. Schürr, and Bernhard Westfechtel, *Graph Transformations and Model-Driven Engineering*. Springer Science+Business Media, 2010. doi: <https://doi.org/10.1007/978-3-642-17322-6>.

- [11] “GROOVE home page | GROOVE home page,” *Utwente.nl*, 2025. <https://groove.cs.utwente.nl/> (accessed Aug. 14, 2025).
- [12] R. Söldner and D. Plump, “Formalising the Double-Pushout Approach to Graph Transformation,” *Logical Methods in Computer Science*, vol. Volume 20, Issue 4, Oct. 2024, doi: [https://doi.org/10.46298/lmcs-20\(4:3\)2024](https://doi.org/10.46298/lmcs-20(4:3)2024).
- [13] T. Arendt, E. Biermann, S. Jurack, C. Krause, and G. Taentzer, “Henshin: Advanced Concepts and Tools for In-Place EMF Model Transformations.” Accessed: Aug. 14, 2025. [Online]. Available: <https://www.eclipse.org/lists/henshin-dev/pdfTchldeGZdV.pdf>
- [14] “GROOVE user manual | GROOVE home page,” *Utwente.nl*, 2025. <https://groove.cs.utwente.nl/manual.html>
- [15] “OpenAI Platform,” *Openai.com*, 2025. <https://platform.openai.com/docs/guides/structured-outputs> (accessed Aug. 24, 2025)
- [16] “Claude 4 prompt engineering best practices - Anthropic,” *Anthropic.com*, 2025. <https://docs.anthropic.com/en/docs/build-with-claude/prompt-engineering/claude-4-best-practices> (accessed Aug. 24, 2025).
- [17] J. White *et al.*, “A Prompt Pattern Catalog to Enhance Prompt Engineering with ChatGPT,” *arXiv.org*, 2023. <https://arxiv.org/abs/2302.11382> (accessed Aug. 24, 2025).
- [18] G. D. Service, “Expenses if you’re self-employed,” *GOV.UK*, Aug. 07, 2014. <https://www.gov.uk/expenses-if-youre-self-employed>
- [19] G. D. Service, “Income Tax rates and Personal Allowances,” *GOV.UK*, Jan. 26, 2012. <https://www.gov.uk/income-tax-rates>
- [20] A. Seely, F. Masala, J. Mirza-Davies, and M. Keep, “Direct taxes: Rates and allowances for 2025/26,” *House of Commons Library*, Apr. 10, 2025. <https://commonslibrary.parliament.uk/research-briefings/cbp-10237/>
- [21] “Blind Person’s Allowance,” *GOV.UK*. <https://www.gov.uk/blind-persons-allowance/what-youll-get>
- [22] “The Income Tax (Indexation of Blind Person’s Allowance and Married Couple’s Allowance) Order 2025,” *Legislation.gov.uk*, 2025. <https://www.legislation.gov.uk/uksi/2025/53/> made (accessed Aug. 24, 2025).
- [23] *GOV.UK*, “Tax on Your Private Pension Contributions,” *GOV.UK*, 2025. <https://www.gov.uk/tax-on-your-private-pension/pension-tax-relief>

- [24] “How tax relief is given on pension contributions | Low Incomes Tax Reform Group,” *Litr.org.uk*, 2024. <https://www.litr.org.uk/pensions/paying-pensions/tax-relief-pension-contributions/how-tax-relief-given-pension-contributions>
- [25] HM Revenue & Customs, “Rates and thresholds for employers 2025 to 2026,” *GOV.UK*, Jan. 31, 2025. <https://www.gov.uk/guidance/rates-and-thresholds-for-employers-2025-to-2026>
- [26] Gov.uk, “Tax on Savings Interest,” *Gov.uk*, 2024. <https://www.gov.uk/apply-tax-free-interest-on-savings>
- [27] GOV.UK, “Expenses if you’re self-employed,” *GOV.UK*, 2023. <https://www.gov.uk/expenses-if-youre-self-employed>
- [28] “Can you explain the 0% Starting Rate for Savings and the Personal Savings Allowance? - Tax Help for Older People,” *Taxvol.org.uk*, 2019. https://taxvol.org.uk/index.php/hrf_faq/can-you-explain-the-0-starting-rate-for-savings-and-the-personal-savings-allowance/
- [29] I. Al-Azzoni, R. Heckel, and Z. Erum, “Graph Rewriting for Testing Domain-specific Models: Dynamic Role-Based Access Control,” doi: <https://doi.org/10.5381/jot.2025.24.2.a4>.
- [30] U. Nawaz, M. Anees-ur-Rahaman, and Z. Saeed, “A review of neuro-symbolic AI integrating reasoning and learning for advanced cognitive systems,” *Intelligent Systems with Applications*, vol. 26, p. 200541, May 2025, doi: <https://doi.org/10.1016/j.iswa.2025.200541>.
- [31] I. D. Raji *et al.*, “Closing the AI accountability gap: Defining an End-to-End Framework for Internal Algorithmic Auditing,” *Proceedings of the 2020 Conference on Fairness, Accountability, and Transparency*, pp. 33–44, Jan. 2020, doi: <https://doi.org/10.1145/3351095.3372873>.
- [32] W. J. Yeo, V. Der, R. Mao, E. Cambria, Ranjan Satapathy, and Gianmarco Mengaldo, “A comprehensive review on financial explainable AI,” *Artificial Intelligence Review*, vol. 58, no. 6, Mar. 2025, doi: <https://doi.org/10.1007/s10462-024-11077-7>.
- [33] Karen McGregor Richmond, S. M. Muddamsetty, T. Gammeltoft-Hansen, Henrik Palmer Olsen, and T. B. Moeslund, “Explainable AI and Law: An Evidential Survey,” *Digital Society*, vol. 3, no. 1, Dec. 2023, doi: <https://doi.org/10.1007/s44206-023-00081-z>.
- [34] HM Revenue & Customs, “Income Tax rates and allowances for current and past years,” *GOV.UK*, 2024. <https://www.gov.uk/government/publications/rates-and-allowances-income-tax/income-tax-rates-and-allowances-current-and-past>
- [35] “GCM 2025 - - STAF 2025,” “Toward Safeguarding GenAI with Graph Transformation: Defining an Exchange Format in JSON (Lightning Talk)” *Researchr.org*, 2025. <https://conf.researchr.org/home/staf-2025/gcm-2025#program> (accessed Sep. 04, 2025).

Appendix

Appendix 1 - Type Graph in JSON

Note: Complete JSON is present in gitlab Repo name *JSON-Rules.txt*

```

1  {
2      "typedGraphs": [],
3      "typeGraph": {
4          "id": "TaxTypeGraph",
5          "nodes": [
6              {
7                  "type": "Person",
8                  "attributes": {
9                      "total_income": "float",
10                     "adjusted_net_income": "float",
11                     "personal_allowance": "float",
12                     "taxable_income": "float",
13                     "tax": "float",
14                     "pension_contributions": "float",
15                     "net_relevant_earnings": "float",
16                     "pension_relief": "float",
17                     "relief_at_source": "boolean",
18                     "taxpayer_registered_blind": "boolean",
19                     "blind_persons_allowance": "float",
20                     "charitable_donations": "float",
21                     "gift_aid_relief": "float",
22                     "personal_savings_allowance": "float",
23                     "dividend_allowance": "float",
24                     "total_other_income": "float",
25                     "amount_in_starting_savings": "float"
26                 }
27             },
28             {
29                 "type": "IncomeSource",
30                 "attributes": {
31                     "source_type": "string",
32                     "revenue": "float",
33                     "net_income": "float",
34                     "expenses": "float",
35                     "processed": "boolean"
36                 }
37             },
38             {
39                 "type": "SAVINGS",
40                 "attributes": {
41                     "lower_bound": "float",
42                     "upper_bound": "float",
43                     "rate": "float"
44                 }
45             }
46         ]
47     }
48 }

```

```

46 {
47   "type": "BASIC",
48   "attributes": {
49     "lower_bound": "float",
50     "upper_bound": "float",
51     "rate": "float"
52   }
53 },
54 {
55   "type": "HIGHER",
56   "attributes": {
57     "lower_bound": "float",
58     "upper_bound": "float",
59     "rate": "float"
60   }
61 },
62 {
63   "type": "ADDITIONAL",
64   "attributes": {
65     "lower_bound": "float",
66     "upper_bound": "float",
67     "rate": "float"
68   }
69 },
70 ],
71 "edges": [
72 {
73   "type": "EarnsFrom",
74   "source": "Person",
75   "target": "IncomeSource",
76   "multiplicity": {
77     "lower": 0,
78     "upper": "*"
79   },
80   "attributes": {}
81 },
82 {
83   "type": "FallsIn",
84   "source": "Person",
85   "target": "SAVINGS",
86   "multiplicity": {
87     "lower": 0,
88     "upper": 1
89   },
90   "attributes": {}
91 },

```

```

92 {
93   "type": "FallsIn",
94   "source": "Person",
95   "target": "BASIC",
96   "multiplicity": {
97     "lower": 0,
98     "upper": 1
99   },
100   "attributes": {}
101 },
102 {
103   "type": "FallsIn",
104   "source": "Person",
105   "target": "HIGHER",
106   "multiplicity": {
107     "lower": 0,
108     "upper": 1
109   },
110   "attributes": {}
111 },
112 {
113   "type": "FallsIn",
114   "source": "Person",
115   "target": "ADDITIONAL",
116   "multiplicity": {
117     "lower": 0,
118     "upper": 1
119   },
120   "attributes": {}
121 }
122 ],
123 },

```


Input Rule JSON

```

124 "rules": [
125   {
126     "name": "CreateEmploymentIncome",
127     "description": "Create an Employment IncomeSource linked to a Person (no TaxBand yet).",
128     "typedOver": "TaxTypeGraph",
129     "parameters": [
130       {
131         "id": "p0",
132         "name": "revenue",
133         "type": "real",
134         "mode": "input",
135         "parin": 0
136       }
137     ],
138     "nodes": [
139       {
140         "id": "person",
141         "type": "Person",
142         "action": "preserve"
143       },
144       {
145         "id": "income",
146         "type": "IncomeSource",
147         "action": "create",
148         "attributes": {
149           "source_type": "set: 'EMPLOYMENT'",
150           "revenue": "set: p0",
151           "expenses": "set: 0.0",
152           "net_income": "set: p0",
153           "processed": "set: false"
154         }
155       }
156     ],
157     "edges": [
158       {
159         "id": "ef_income",
160         "type": "EarnsFrom",
161         "source": "person",
162         "target": "income",
163         "action": "create"
164       }
165     ]
166   },

```

```

168 "name": "CreateSelfEmploymentIncome",
169 "description": "Create a Self-Employment IncomeSource linked to a Person (no TaxBand yet).",
170 "typedOver": "TaxTypeGraph",
171 "parameters": [
172   {
173     "id": "p0",
174     "name": "revenue",
175     "type": "real",
176     "mode": "input",
177     "parin": 0
178   },
179   {
180     "id": "p1",
181     "name": "expenses",
182     "type": "real",
183     "mode": "input",
184     "parin": 1
185   }
186 ],
187 "nodes": [
188   {
189     "id": "person",
190     "type": "Person",
191     "action": "preserve"
192   },
193   {
194     "id": "income",
195     "type": "IncomeSource",
196     "action": "create",
197     "attributes": {
198       "source_type": "set: 'SELF_EMPLOYMENT'",
199       "revenue": "set: p0",
200       "expenses": "set: p1",
201       "net_income": "set: p0 - p1",
202       "processed": "set: false"
203     }
204   }
205 ],
206 "edges": [
207   {
208     "id": "ef_income",
209     "type": "EarnsFrom",
210     "source": "person",
211     "target": "income",
212     "action": "create"
213   }
214 ]
215 },

```

```

216 {
217   "name": "CreateSavingsInterestIncome",
218   "description": "Create a Savings Interest IncomeSource linked to a Person (no TaxBand yet).",
219   "typedOver": "TaxTypeGraph",
220   "parameters": [
221     {
222       "id": "p0",
223       "name": "revenue",
224       "type": "real",
225       "mode": "input",
226       "parin": 0
227     }
228   ],
229   "nodes": [
230     {
231       "id": "person",
232       "type": "Person",
233       "action": "preserve"
234     },
235     {
236       "id": "income",
237       "type": "IncomeSource",
238       "action": "create",
239       "attributes": {
240         "source_type": "set: 'SAVINGS_INTEREST'",
241         "revenue": "set: p0",
242         "expenses": "set: 0.0",
243         "net_income": "set: p0",
244         "processed": "set: false"
245       }
246     }
247   ],
248   "edges": [
249     {
250       "id": "ef_income",
251       "type": "EarnsFrom",
252       "source": "person",
253       "target": "income",
254       "action": "create"
255     }
256   ]
257 },

```

```

258 {
259   "name": "CreateDividendIncome",
260   "description": "Create a Dividend IncomeSource linked to a Person (no TaxBand yet).",
261   "typedOver": "TaxTypeGraph",
262   "parameters": [
263     {
264       "id": "p0",
265       "name": "revenue",
266       "type": "real",
267       "mode": "input",
268       "parin": 0
269     }
270   ],
271   "nodes": [
272     {
273       "id": "person",
274       "type": "Person",
275       "action": "preserve"
276     },
277     {
278       "id": "income",
279       "type": "IncomeSource",
280       "action": "create",
281       "attributes": {
282         "source_type": "set: 'DIVIDEND'",
283         "revenue": "set: p0",
284         "expenses": "set: 0.0",
285         "net_income": "set: p0",
286         "processed": "set: false"
287       }
288     }
289   ],
290   "edges": [
291     {
292       "id": "ef_income",
293       "type": "EarnsFrom",
294       "source": "person",
295       "target": "income",
296       "action": "create"
297     }
298   ]
299 },

```

Automated Validation of GPT-Based Tax Query Responses through Graph-Based Reasoning

```
300 {
301   "name": "inputParams",
302   "description": "Assign input parameters on an existing Person: pension contributions, charitable donations, relief at source, and registered blind status.",
303   "typedOver": "TaxTypeGraph",
304   "parameters": [
305     {
306       "id": "p0",
307       "name": "pension_contributions",
308       "type": "real",
309       "mode": "input",
310       "parin": 0
311     },
312     {
313       "id": "p1",
314       "name": "charitable_donations",
315       "type": "real",
316       "mode": "input",
317       "parin": 1
318     },
319     {
320       "id": "p2",
321       "name": "relief_at_source",
322       "type": "bool",
323       "mode": "input",
324       "parin": 2
325     },
326     {
327       "id": "p3",
328       "name": "taxpayer_registered_blind",
329       "type": "bool",
330       "mode": "input",
331       "parin": 3
332     }
333   ],
334   "nodes": [
335     {
336       "id": "person",
337       "type": "Person",
338       "action": "modify",
339       "attributes": {
340         "pension_contributions": "set: p0",
341         "charitable_donations": "set: p1",
342         "relief_at_source": "set: p2",
343         "taxpayer_registered_blind": "set: p3"
344       }
345     }
346   ],
347   "edges": []
348 }
```

Tax Rule 1

```
350 "name": "I1",
351 "description": "If a Person has an un-processed IncomeSource of type 'EMPLOYMENT', then add that source's revenue to the Person's total_income and mark the source as processed (exactly once).",
352 "typedOver": "TaxTypeGraph",
353 "nodes": [
354   {
355     "id": "p",
356     "type": "Person",
357     "action": "preserve",
358     "attributes": {
359       "total_income": "set: total_income + is.revenue"
360     }
361   },
362   {
363     "id": "is",
364     "type": "IncomeSource",
365     "action": "preserve",
366     "attributes": {
367       "processed": "set: true"
368     }
369   }
370 ],
371 "edges": [
372   {
373     "id": "ef1",
374     "type": "EarningsFrom",
375     "source": "p",
376     "target": "is",
377     "action": "preserve"
378   }
379 ],
380 "applicationConditions": {
381   "name": "rule1Cond",
382   "formula": "and(eq(is.source_type, 'EMPLOYMENT'), eq(is.processed, false))",
383   "graphs": [
384     {
385       "name": "C1",
386       "nodes": [
387         {
388           "id": "is",
389           "type": "IncomeSource",
390           "action": "preserve",
391           "attributes": {
392             "test": "and(eq(is.source_type, 'EMPLOYMENT'), eq(is.processed, false))"
393           }
394         }
395       ],
396       "edges": []
397     }
398   ]
399 }
```

Tax Rule 2

```

402     "name": "2",
403     "description": "If a Person has an un-processed IncomeSource of type 'SELF_EMPLOYMENT', then add that source's net income to the Person's total_income and mark the source as processed (exactly once).",
404     "typedOver": "TaxTypeGraph",
405     "nodes": [
406       {
407         "id": "p",
408         "type": "Person",
409         "action": "preserve",
410         "attributes": {
411           "total_income": "set: total_income + is.net_income"
412         }
413       },
414       {
415         "id": "is",
416         "type": "IncomeSource",
417         "action": "preserve",
418         "attributes": {
419           "processed": "set: true"
420         }
421       }
422     ],
423     "edges": [
424       {
425         "id": "ef1",
426         "type": "EarningsFrom",
427         "source": "p",
428         "target": "is",
429         "action": "preserve"
430       }
431     ],
432     "applicationConditions": {
433       "acName": "rule2Cond",
434       "formula": "and(eq(is.source_type, 'SELF_EMPLOYMENT'), eq(is.processed, false))",
435       "graphs": [
436         {
437           "name": "C2",
438           "nodes": [
439             {
440               "id": "is",
441               "type": "IncomeSource",
442               "action": "preserve",
443               "attributes": {
444                 "test": "and(eq(is.source_type, 'SELF_EMPLOYMENT'), eq(is.processed, false))"
445               }
446             }
447           ],
448           "edges": []
449         }
450       ]
451     }

```

Tax Rule 3

```

453     "name": "3",
454     "description": "If a Person has an un-processed IncomeSource of type 'SAVINGS_INTEREST', then add that source's revenue to the Person's total_income and mark the source as processed (exactly once).",
455     "typedOver": "TaxTypeGraph",
456     "nodes": [
457       {
458         "id": "p",
459         "type": "Person",
460         "action": "preserve",
461         "attributes": {
462           "total_income": "set: total_income + is.revenue"
463         }
464       },
465       {
466         "id": "is",
467         "type": "IncomeSource",
468         "action": "preserve",
469         "attributes": {
470           "processed": "set: true"
471         }
472       }
473     ],
474     "edges": [
475       {
476         "id": "ef1",
477         "type": "EarningsFrom",
478         "source": "p",
479         "target": "is",
480         "action": "preserve"
481       }
482     ],
483     "applicationConditions": {
484       "acName": "rule3Cond",
485       "formula": "and(eq(is.source_type, 'SAVINGS_INTEREST'), eq(is.processed, false))",
486       "graphs": [
487         {
488           "name": "C3",
489           "nodes": [
490             {
491               "id": "is",
492               "type": "IncomeSource",
493               "action": "preserve",
494               "attributes": {
495                 "test": "and(eq(is.source_type, 'SAVINGS_INTEREST'), eq(is.processed, false))"
496               }
497             }
498           ],
499           "edges": []
500         }
501       ]
502     }

```

Tax Rule 4

```

505 {
506   "name": "4",
507   "description": "If a Person has an un-processed IncomeSource of type 'DIVIDEND', then add that source's revenue to the Person's total_income and Mark the source as processed (exactly once).",
508   "typedOver": "TaxTypeGraph",
509   "nodes": [
510     {
511       "id": "p",
512       "type": "Person",
513       "action": "preserve",
514       "attributes": {
515         "total_income": "set: total_income + is.revenue"
516       }
517     },
518     {
519       "id": "is",
520       "type": "IncomeSource",
521       "action": "preserve",
522       "attributes": {
523         "processed": "set: true"
524       }
525     }
526   ],
527   "edges": [
528     {
529       "id": "ef1",
530       "type": "EarnsFrom",
531       "source": "p",
532       "target": "is",
533       "action": "preserve"
534     }
535   ],
536   "applicationConditions": {
537     "acName": "rule4Cond",
538     "formula": "and(eq(is.source_type, 'DIVIDEND'), eq(is.processed, false))",
539     "graphs": [
540       {
541         "name": "C4",
542         "nodes": [
543           {
544             "id": "is",
545             "type": "IncomeSource",
546             "action": "preserve",
547             "attributes": {
548               "test": "and(eq(is.source_type, 'DIVIDEND'), eq(is.processed, false))"
549             }
550           }
551         ],
552         "edges": []
553       }
554     ]
555   }
556 }

```

Tax Rule 5

```

557 {
558   "name": "5",
559   "description": "If a Person has an IncomeSource of type 'EMPLOYMENT', then add that source's net_income to the Person's net_relevant_earnings.",
560   "typedOver": "TaxTypeGraph",
561   "nodes": [
562     {
563       "id": "p",
564       "type": "Person",
565       "action": "preserve",
566       "attributes": {
567         "net_relevant_earnings": "set: net_relevant_earnings + is.net_income"
568       }
569     },
570     {
571       "id": "is",
572       "type": "IncomeSource",
573       "action": "preserve"
574     }
575   ],
576   "edges": [
577     {
578       "id": "ef1",
579       "type": "EarnsFrom",
580       "source": "p",
581       "target": "is",
582       "action": "preserve"
583     }
584   ],
585   "applicationConditions": {
586     "acName": "rule5Cond",
587     "formula": "eq(is.source_type, 'EMPLOYMENT')",
588     "graphs": [
589       {
590         "name": "C4_1",
591         "nodes": [
592           {
593             "id": "is",
594             "type": "IncomeSource",
595             "action": "preserve",
596             "attributes": {
597               "test": "eq(is.source_type, 'EMPLOYMENT')"
598             }
599           }
600         ],
601         "edges": []
602       }
603     ]
604   }
605 }

```

Tax Rule 6

```

607     "name": "6",
608     "description": "If a Person has an IncomeSource of type 'SELF_EMPLOYMENT', then add that source's net_income to the Person's net_relevant_earnings.",
609     "typedOver": "TaxTypeGraph",
610     "nodes": [
611         {
612             "id": "p",
613             "type": "Person",
614             "action": "preserve",
615             "attributes": {
616                 "net_relevant_earnings": "set: net_relevant_earnings + is.net_income"
617             }
618         },
619         {
620             "id": "is",
621             "type": "IncomeSource",
622             "action": "preserve"
623         }
624     ],
625     "edges": [
626         {
627             "id": "e1",
628             "type": "EarnsFrom",
629             "source": "p",
630             "target": "is",
631             "action": "preserve"
632         }
633     ],
634     "applicationConditions": {
635         "acName": "rule4_2Cond",
636         "formula": "eq(is.source_type, 'SELF_EMPLOYMENT')",
637         "graphs": [
638             {
639                 "name": "C4_2",
640                 "nodes": [
641                     {
642                         "id": "is",
643                         "type": "IncomeSource",
644                         "action": "preserve",
645                         "attributes": {
646                             "test": "eq(is.source_type, 'SELF_EMPLOYMENT')"
647                         }
648                     }
649                 ],
650                 "edges": []
651             }
652         ]
653     }
654 },

```

Tax Rule 7a

```

656     "name": "7a",
657     "description": "If a Person's relief_at_source is false, set pension_relief to the minimum of pension_contributions and net_relevant_earnings.",
658     "typedOver": "TaxTypeGraph",
659     "nodes": [
660         {
661             "id": "p",
662             "type": "Person",
663             "action": "preserve",
664             "attributes": {
665                 "pension_relief": "set: min(pension_contributions, net_relevant_earnings)"
666             }
667         }
668     ],
669     "edges": [],
670     "applicationConditions": {
671         "acName": "rule07aCond",
672         "formula": "eq(p.relief_at_source, false)",
673         "graphs": [
674             {
675                 "name": "C1",
676                 "nodes": [
677                     {
678                         "id": "p",
679                         "type": "Person",
680                         "action": "preserve",
681                         "attributes": {
682                             "test": "eq(p.relief_at_source, false)"
683                         }
684                     }
685                 ],
686                 "edges": []
687             }
688         ]
689     }
690 },

```

Tax Rule 7b

```

691 {
692   "name": "7b",
693   "description": "If a Person's relief_at_source is true, set pension_relief to the minimum of pension_contributions * 1.25 and net_relevant_earnings.",
694   "typedOver": "TaxTypeGraph",
695   "nodes": [
696     {
697       "id": "p",
698       "type": "Person",
699       "action": "preserve",
700       "attributes": {
701         "pension_relief": "set: min(pension_contributions * 1.25, net_relevant_earnings)"
702       }
703     }
704   ],
705   "edges": [],
706   "applicationConditions": {
707     "acName": "rule07bCond",
708     "formula": "eq(p.relief_at_source, true)",
709     "graphs": [
710       {
711         "name": "C1",
712         "nodes": [
713           {
714             "id": "p",
715             "type": "Person",
716             "action": "preserve",
717             "attributes": {
718               "test": "eq(p.relief_at_source, true)"
719             }
720           }
721         ],
722         "edges": []
723       }
724     ]
725   }
726 },

```

Tax Rule 8

```

727 {
728   "name": "8",
729   "description": "If a Person's charitable_donations is greater than 0.0, set gift_aid_relief to charitable_donations * 1.25.",
730   "typedOver": "TaxTypeGraph",
731   "nodes": [
732     {
733       "id": "p",
734       "type": "Person",
735       "action": "preserve",
736       "attributes": {
737         "gift_aid_relief": "set: charitable_donations * 1.25"
738       }
739     }
740   ],
741   "edges": [],
742   "applicationConditions": {
743     "acName": "rule08Cond",
744     "formula": "gt(p.charitable_donations, 0.0)",
745     "graphs": [
746       {
747         "name": "C1",
748         "nodes": [
749           {
750             "id": "p",
751             "type": "Person",
752             "action": "preserve",
753             "attributes": {
754               "test": "gt(p.charitable_donations, 0.0)"
755             }
756           }
757         ],
758         "edges": []
759       }
760     ]
761   }
762 },

```

Tax Rule 9

```

763 {
764   "name": "9",
765   "description": "Set adjusted_net_income to total_income minus pension_relief and gift_aid_relief for each Person.",
766   "typedOver": "TaxTypeGraph",
767   "nodes": [
768     {
769       "id": "p",
770       "type": "Person",
771       "action": "preserve",
772       "attributes": {
773         "adjusted_net_income": "set: p.total_income - p.pension_relief - p.gift_aid_relief"
774       }
775     }
776   ],
777   "edges": []
778 },

```

Tax Rule 10

```

799 {
800   "name": "10",
801   "description": "If a Person's adjusted_net_income is greater than 100000 and less than 125140, then set their personal_allowance to 12570 minus half of the amount by which their income exceeds 100000 (exactly once).",
802   "typedOver": "TaxTypeGraph",
803   "nodes": [
804     {
805       "id": "p",
806       "type": "Person",
807       "action": "preserve",
808       "attributes": {
809         "personal_allowance": "set: sub(12570.00, div(sub(adjusted_net_income,100000.00),2.00))"
810       }
811     }
812   ],
813   "edges": [],
814   "applicationConditions": {
815     "acName": "rule10Cond",
816     "formula": "and(gt(p.adjusted_net_income,100000.00), lt(p.adjusted_net_income,125140.00))",
817     "graphs": [
818       {
819         "name": "C6",
820         "nodes": [
821           {
822             "id": "p",
823             "type": "IncomeSource",
824             "action": "preserve",
825             "attributes": {
826               "test": "and(gt(p.adjusted_net_income,100000.00), lt(p.adjusted_net_income,125140.00))"
827             }
828           }
829         ],
830         "edges": []
831       }
832     ]
833   }
834 }

```

Tax Rule 11

```

816 "name": "11",
817 "description": "If a Person's adjusted_net_income is less than or equal to 100000.00, set personal_allowance to 12570.00.",
818 "typedOver": "TaxTypeGraph",
819 "nodes": [
820   {
821     "id": "p",
822     "type": "Person",
823     "action": "preserve",
824     "attributes": {
825       "personal_allowance": "set: 12570.00"
826     }
827   },
828 ],
829 "edges": [],
830 "applicationConditions": {
831   "acName": "rule11Cond",
832   "formula": "le(p.adjusted_net_income, 100000.00)",
833   "graphs": [
834     {
835       "name": "C1",
836       "nodes": [
837         {
838           "id": "p",
839           "type": "Person",
840           "action": "preserve",
841           "attributes": {
842             "test": "le(p.adjusted_net_income, 100000.00)"
843           }
844         }
845       ],
846       "edges": []
847     }
848   ]
849 }

```


Tax Rule 12

```

851 {
852   "name": "12",
853   "description": "If a Person's adjusted_net_income is at least 125140.00, then set their personal_allowance to 0.00.",
854   "typedOver": "TaxTypeGraph",
855   "nodes": [
856     {
857       "id": "p",
858       "type": "Person",
859       "action": "preserve",
860       "attributes": {
861         "personal_allowance": "set: 0.00"
862       }
863     }
864   ],
865   "edges": [],
866   "applicationConditions": {
867     "acName": "rule7Cond",
868     "formula": "ge(p.adjusted_net_income, 125140.00)",
869     "graphs": [
870       {
871         "name": "C7",
872         "nodes": [
873           {
874             "id": "p",
875             "type": "Person",
876             "action": "preserve",
877             "attributes": {
878               "test": "ge(p.adjusted_net_income, 125140.00)"
879             }
880           }
881         ],
882         "edges": []
883       }
884     ]
885   }
886 },

```

Tax Rule 13

```

887 {
888   "name": "13",
889   "description": "If a Person's taxpayer_registered_blind is true then set that Person's blind_persons_allowance to 3130.00.",
890   "typedOver": "TaxTypeGraph",
891   "nodes": [
892     {
893       "id": "p",
894       "type": "Person",
895       "action": "preserve",
896       "attributes": {
897         "blind_persons_allowance": "set: 3130.00"
898       }
899     }
900   ],
901   "edges": [],
902   "applicationConditions": {
903     "acName": "rule9Cond",
904     "formula": "eq(p.taxpayer_registered_blind, true)",
905     "graphs": [
906       {
907         "name": "C9",
908         "nodes": [
909           {
910             "id": "p",
911             "type": "Person",
912             "action": "preserve",
913             "attributes": {
914               "test": "eq(p.taxpayer_registered_blind, true)"
915             }
916           }
917         ],
918         "edges": []
919       }
920     ]
921   }
922 },

```

Tax Rule 14

```

924 {
925   "name": "14",
926   "description": "Set taxable_income to the maximum of 0.0 and adjusted_net_income minus the sum of personal_allowance and blind_persons_allowance for each Person.",
927   "typedOver": "TaxTypeGraph",
928   "nodes": [
929     {
930       "id": "p",
931       "type": "Person",
932       "action": "preserve",
933       "attributes": {
934         "taxable_income": "set: max(0.0, adjusted_net_income - (personal_allowance + blind_persons_allowance))"
935       }
936     }
937   ],
938   "edges": []
939 },

```

Tax Rule 15

```

940 {
941   "name": "I1",
942   "description": "For each Person with both a SELF_EMPLOYMENT and an EMPLOYMENT IncomeSource, set total_other_income to the maximum of 0.0 and (IS.net_income + I.net_income) minus (p.personal_allowance + blind_persons_allowance)",
943   "typeOver": "TaxTypeGraph",
944   "nodes": [
945     {
946       "id": "p",
947       "type": "Person",
948       "action": "preserve",
949       "attributes": {
950         "total_other_income": "set: max(0.0, sub(add(IS.net_income, I.net_income), add(p.personal_allowance, blind_persons_allowance)))"
951       }
952     },
953     {
954       "id": "IS",
955       "type": "IncomeSource",
956       "action": "preserve"
957     },
958     {
959       "id": "I",
960       "type": "IncomeSource",
961       "action": "preserve"
962     }
963   ],
964   "edges": [
965     {
966       "id": "e1",
967       "type": "EarnsFrom",
968       "source": "p",
969       "target": "IS",
970       "action": "preserve"
971     },
972     {
973       "id": "e2",
974       "type": "EarnsFrom",
975       "source": "p",
976       "target": "I",
977       "action": "preserve"
978     }
979   ]
980 }

```

```

979 "applicationConditions": {
980   "ruleName": "rule15Cond",
981   "formula": "and(eq(IS.source_type, 'SELF_EMPLOYMENT'), eq(I.source_type, 'EMPLOYMENT'))",
982   "graphs": [
983     {
984       "name": "C1",
985       "nodes": [
986         {
987           "id": "IS",
988           "type": "IncomeSource",
989           "action": "preserve",
990           "attributes": {
991             "test": "eq(IS.source_type, 'SELF_EMPLOYMENT')"
992           }
993         },
994         {
995           "id": "I",
996           "type": "IncomeSource",
997           "action": "preserve",
998           "attributes": {
999             "test": "eq(I.source_type, 'EMPLOYMENT')"
1000           }
1001         }
1002       ],
1003       "edges": []
1004     }
1005   ]
1006 }
1007 },

```

Tax Rule 16

```

1008 {
1009   "name": "I6",
1010   "description": "If a Person has an IncomeSource of type SAVINGS_INTEREST, set amount_in_starting_savings to the minimum of max(0.0, 5000.00 - total_other_income) and that IncomeSource's net_income.",
1011   "typeOver": "TaxTypeGraph",
1012   "nodes": [
1013     {
1014       "id": "p",
1015       "type": "Person",
1016       "action": "preserve",
1017       "attributes": {
1018         "amount_in_starting_savings": "set: min(max(0.0, 5000.00 - total_other_income), I.net_income)"
1019       }
1020     },
1021     {
1022       "id": "I",
1023       "type": "IncomeSource",
1024       "action": "preserve"
1025     }
1026   ],
1027   "edges": [
1028     {
1029       "id": "e1",
1030       "type": "EarnsFrom",
1031       "source": "p",
1032       "target": "I",
1033       "action": "preserve"
1034     }
1035   ],
1036   "applicationConditions": {
1037     "ruleName": "rule16Cond",
1038     "formula": "eq(I.source_type, 'SAVINGS_INTEREST')",
1039     "graphs": [
1040       {
1041         "name": "C1",
1042         "nodes": [
1043           {
1044             "id": "I",
1045             "type": "IncomeSource",
1046             "action": "preserve",
1047             "attributes": {
1048               "test": "eq(I.source_type, 'SAVINGS_INTEREST')"
1049             }
1050           }
1051         ],
1052         "edges": []
1053       }
1054     ]
1055   }
1056 },

```

Tax Rule 17a

```

1050 "name": "17a",
1051 "description": "When a Person (p) has a SAVINGS_INTEREST IncomeSource (I) with I.net_income > 0.00 and p.total_other_income ≤ 5000.00, create a new SAVINGS node (s) with bounds and a FallsIn edge from p to s.",
1052 "typedOver": "TaxTypeGraph",
1053 "nodes": [
1054   {
1055     "id": "p",
1056     "type": "Person",
1057     "action": "preserve"
1058   },
1059   {
1060     "id": "I",
1061     "type": "IncomeSource",
1062     "action": "preserve"
1063   },
1064   {
1065     "id": "s",
1066     "type": "SAVINGS",
1067     "action": "create",
1068     "attributes": {
1069       "lower_bound": "set: 0.00",
1070       "upper_bound": "set: min(max(0.0, 5000.00 - p.total_other_income), I.net_income)"
1071     }
1072   }
1073 ],
1074 "edges": [
1075   {
1076     "id": "e1",
1077     "type": "FallsIn",
1078     "source": "p",
1079     "target": "s",
1080     "action": "create"
1081   }
1082 ],
1083 "applicationConditions": {
1084   "acName": "rule17aCond",
1085   "formula": "and(and(gt(I.net_income, 0.00), eq(I.source_type, 'SAVINGS_INTEREST')), le(p.total_other_income, 5000.00))",
1086   "graphs": [
1087     {
1088       "name": "C1",
1089       "nodes": [
1090         {
1091           "id": "I",
1092           "type": "IncomeSource",
1093           "action": "preserve",
1094           "attributes": {
1095             "test": "and(and(gt(I.net_income, 0.00), eq(I.source_type, 'SAVINGS_INTEREST')), le(p.total_other_income, 5000.00))"
1096           }
1097         }
1098       ],
1099       "edges": []
1100     }
1101   ]
1102 }
1103

```

Tax Rule 17b

```

1120 {
1121   "name": "17b",
1122   "description": "If a Person's taxable_income minus amount_in_starting_savings is greater than 0.00, create a BASIC band with bounds and link it to the Person.",
1123   "typedOver": "TaxTypeGraph",
1124   "nodes": [
1125     {
1126       "id": "p",
1127       "type": "Person",
1128       "action": "preserve"
1129     },
1130     {
1131       "id": "b",
1132       "type": "BASIC",
1133       "action": "create",
1134       "attributes": {
1135         "lower_bound": "set: 0.00",
1136         "upper_bound": "set: 37700.00"
1137       }
1138     }
1139   ],
1140   "edges": [
1141     {
1142       "id": "e1",
1143       "type": "FallsIn",
1144       "source": "p",
1145       "target": "b",
1146       "action": "create"
1147     }
1148   ],
1149   "applicationConditions": {
1150     "acName": "rule17bCond",
1151     "formula": "gt(p.taxable_income - p.amount_in_starting_savings, 0.00)",
1152     "graphs": [
1153       {
1154         "name": "C1",
1155         "nodes": [
1156           {
1157             "id": "p",
1158             "type": "Person",
1159             "action": "preserve",
1160             "attributes": {
1161               "test": "gt(p.taxable_income - p.amount_in_starting_savings, 0.00)"
1162             }
1163           }
1164         ],
1165         "edges": []
1166       }
1167     ]
1168   }
1169 }
1170

```

Tax Rule 17c

```

1172     "name": "17c",
1173     "description": "If a Person's taxable_income minus amount_in_starting_savings is greater than 37700.00, create a HIGHER band with bounds and link it to the Person.",
1174     "typedOver": "TaxTypeGraph",
1175     "nodes": [
1176       {
1177         "id": "p",
1178         "type": "Person",
1179         "action": "preserve"
1180       },
1181       {
1182         "id": "h",
1183         "type": "HIGHER",
1184         "action": "create",
1185         "attributes": {
1186           "lower_bound": "set: 37700.00",
1187           "rate": "set: 0.40",
1188           "upper_bound": "set: 125140.00"
1189         }
1190       }
1191     ],
1192     "edges": [
1193       {
1194         "id": "ef1",
1195         "type": "FallsIn",
1196         "source": "p",
1197         "target": "h",
1198         "action": "create"
1199       }
1200     ],
1201     "applicationConditions": {
1202       "acName": "rule17cCond",
1203       "formula": "gt(p.taxable_income - p.amount_in_starting_savings, 37700.00)",
1204       "graphs": [
1205         {
1206           "name": "CI",
1207           "nodes": [
1208             {
1209               "id": "p",
1210               "type": "Person",
1211               "action": "preserve",
1212               "attributes": {
1213                 "test": "gt(p.taxable_income - p.amount_in_starting_savings, 37700.00)"
1214               }
1215             }
1216           ],
1217           "edges": []
1218         }
1219       ]
1220     }
1221   },

```

Tax Rule 17d

```

1223     "name": "17d",
1224     "description": "If a Person's taxable_income minus amount_in_starting_savings is greater than 125140.00, create an ADDITIONAL band with bounds and link it to the Person.",
1225     "typedOver": "TaxTypeGraph",
1226     "nodes": [
1227       {
1228         "id": "p",
1229         "type": "Person",
1230         "action": "preserve"
1231       },
1232       {
1233         "id": "b",
1234         "type": "ADDITIONAL",
1235         "action": "create",
1236         "attributes": {
1237           "lower_bound": "set: 125140.00",
1238           "rate": "set: 0.45"
1239         }
1240       }
1241     ],
1242     "edges": [
1243       {
1244         "id": "ef1",
1245         "type": "FallsIn",
1246         "source": "p",
1247         "target": "b",
1248         "action": "create"
1249       }
1250     ],
1251     "applicationConditions": {
1252       "acName": "rule17dCond",
1253       "formula": "gt(p.taxable_income - p.amount_in_starting_savings, 125140.00)",
1254       "graphs": [
1255         {
1256           "name": "CI",
1257           "nodes": [
1258             {
1259               "id": "p",
1260               "type": "Person",
1261               "action": "preserve",
1262               "attributes": {
1263                 "test": "gt(p.taxable_income - p.amount_in_starting_savings, 125140.00)"
1264               }
1265             }
1266           ],
1267           "edges": []
1268         }
1269       ]
1270     }
1271   },

```

Tax Rule 18

```

1272 {
1273   "name": "18",
1274   "description": "If a Person has edge to BASIC, set personal_savings_allowance to 1000.00",
1275   "typedOver": "TaxTypeGraph",
1276   "nodes": [
1277     {
1278       "id": "p",
1279       "type": "Person",
1280       "action": "preserve",
1281       "attributes": {
1282         "personal_savings_allowance": "set: 1000.00"
1283       }
1284     },
1285     {
1286       "id": "b",
1287       "type": "BASIC",
1288       "action": "preserve"
1289     }
1290   ],
1291   "edges": [
1292     {
1293       "type": "FallsIn",
1294       "source": "p",
1295       "target": "b",
1296       "action": "preserve"
1297     }
1298   ]
1299 },

```

Tax Rule 19

```

1300 {
1301   "name": "19",
1302   "description": "If a Person has a FallsIn edge to HIGHER, set personal_savings_allowance to 500.00",
1303   "typedOver": "TaxTypeGraph",
1304   "nodes": [
1305     {
1306       "id": "p",
1307       "type": "Person",
1308       "action": "preserve",
1309       "attributes": {
1310         "personal_savings_allowance": "set: 500.00"
1311       }
1312     },
1313     {
1314       "id": "b",
1315       "type": "HIGHER",
1316       "action": "preserve"
1317     }
1318   ],
1319   "edges": [
1320     {
1321       "type": "FallsIn",
1322       "source": "p",
1323       "target": "b",
1324       "action": "preserve"
1325     }
1326   ]
1327 },

```

Tax Rule 20

```

1328 {
1329   "name": "20",
1330   "description": "If a Person has a FallsIn edge to ADDITIONAL, set personal_savings_allowance to 0.00",
1331   "typedOver": "TaxTypeGraph",
1332   "nodes": [
1333     {
1334       "id": "p",
1335       "type": "Person",
1336       "action": "preserve",
1337       "attributes": {
1338         "personal_savings_allowance": "set: 0.00"
1339       }
1340     },
1341     {
1342       "id": "b",
1343       "type": "ADDITIONAL",
1344       "action": "preserve"
1345     }
1346   ],
1347   "edges": [
1348     {
1349       "type": "FallsIn",
1350       "source": "p",
1351       "target": "b",
1352       "action": "preserve"
1353     }
1354   ]
1355 },

```

Tax Rule 21

```

1357   "name": "21",
1358   "description": "If a Person has an IncomeSource of type 'DIVIDEND' with revenue greater than 0.0, set that Person's dividend_allowance to 500.00.",
1359   "typedOver": "TaxTypeGraph",
1360   "nodes": [
1361     {
1362       "id": "p",
1363       "type": "Person",
1364       "action": "preserve",
1365       "attributes": {
1366         "dividend_allowance": "set: 500.00"
1367       }
1368     },
1369     {
1370       "id": "is",
1371       "type": "IncomeSource",
1372       "action": "preserve"
1373     }
1374   ],
1375   "edges": [
1376     {
1377       "id": "e1",
1378       "type": "EarnsFrom",
1379       "source": "p",
1380       "target": "is",
1381       "action": "preserve"
1382     }
1383   ],
1384   "applicationConditions": [
1385     {
1386       "name": "rule1&Cond",
1387       "formula": "and(eq(is.source_type,'DIVIDEND'), gt(is.revenue, 0.0))",
1388       "graphs": [
1389         {
1390           "name": "C16",
1391           "nodes": [
1392             {
1393               "id": "is",
1394               "type": "IncomeSource",
1395               "action": "preserve",
1396               "attributes": {
1397                 "test": "and(eq(is.source_type,'DIVIDEND'), gt(is.revenue, 0.0))"
1398               }
1399             }
1400           ],
1401           "edges": []
1402         }
1403       ]
1404     }
1405   ],

```

Tax Rule 22a

```

1406   "name": "22a",
1407   "description": "For each Person and linked BASIC band, increment p.tax by (min(p.taxable_income - p.amount_in_starting_savings, b.upper_bound) - b.lower_bound) * b.rate.",
1408   "typedOver": "TaxTypeGraph",
1409   "nodes": [
1410     {
1411       "id": "p",
1412       "type": "Person",
1413       "action": "preserve",
1414       "attributes": {
1415         "tax": "set: add(p.tax, (min(p.taxable_income - p.amount_in_starting_savings, b.upper_bound) - b.lower_bound) * b.rate)"
1416       }
1417     },
1418     {
1419       "id": "b",
1420       "type": "BASIC",
1421       "action": "preserve"
1422     }
1423   ],
1424   "edges": [
1425     {
1426       "id": "ef1",
1427       "type": "FallsIn",
1428       "source": "p",
1429       "target": "b",
1430       "action": "preserve"
1431     }
1432   ],
1433 },

```

Tax Rule 22b

```

1434   {
1435     "name": "22b",
1436     "description": "For each Person and linked HIGHER band, increment p.tax by (min(p.taxable_income - p.amount_in_starting_savings, h.upper_bound) - h.lower_bound) * h.rate.",
1437     "typedOver": "TaxTypeGraph",
1438     "nodes": [
1439       {
1440         "id": "p",
1441         "type": "Person",
1442         "action": "preserve",
1443         "attributes": {
1444           "tax": "set: add(p.tax, (min(p.taxable_income - p.amount_in_starting_savings, h.upper_bound) - h.lower_bound) * h.rate)"
1445         }
1446       },
1447       {
1448         "id": "h",
1449         "type": "HIGHER",
1450         "action": "preserve"
1451       }
1452     ],
1453     "edges": [
1454       {
1455         "id": "ef1",
1456         "type": "FallsIn",
1457         "source": "p",
1458         "target": "h",
1459         "action": "preserve"
1460       }
1461     ]
1462   },

```

Tax Rule 22c

```

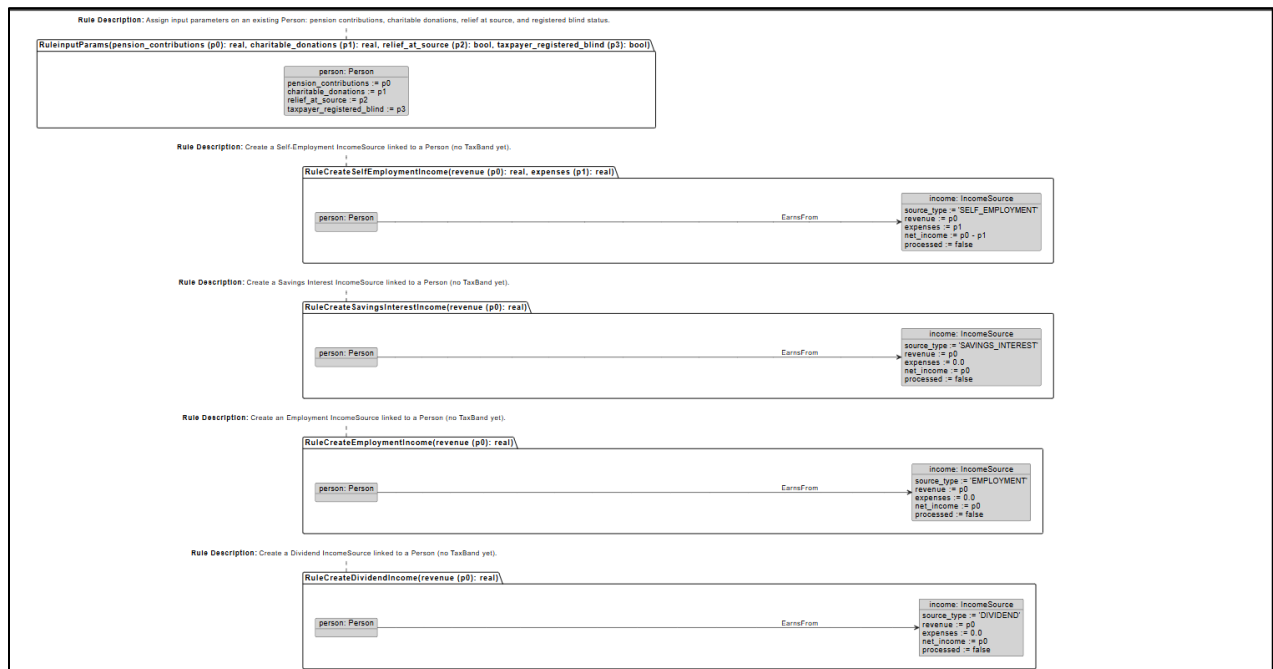
1463 {
1464   "name": "22c",
1465   "description": "For each Person and linked ADDITIONAL band, increment p.tax by (p.taxable_income - amount_in_starting_savings - a.lower_bound) * a.rate.",
1466   "typedOver": "TaxTypeGraph",
1467   "nodes": [
1468     {
1469       "id": "p",
1470       "type": "Person",
1471       "action": "preserve",
1472       "attributes": {
1473         "tax": "set: add(p.tax, (p.taxable_income - amount_in_starting_savings - a.lower_bound) * a.rate)"
1474       }
1475     },
1476     {
1477       "id": "a",
1478       "type": "ADDITIONAL",
1479       "action": "preserve"
1480     }
1481   ],
1482   "edges": [
1483     {
1484       "id": "ef1",
1485       "type": "FallsIn",
1486       "source": "p",
1487       "target": "a",
1488       "action": "preserve"
1489     }
1490   ]
1491 },
1492 ],
1493 "constraints": [],
1494 "ruleApplication": [],
1495 "controls": []
1496 }

```

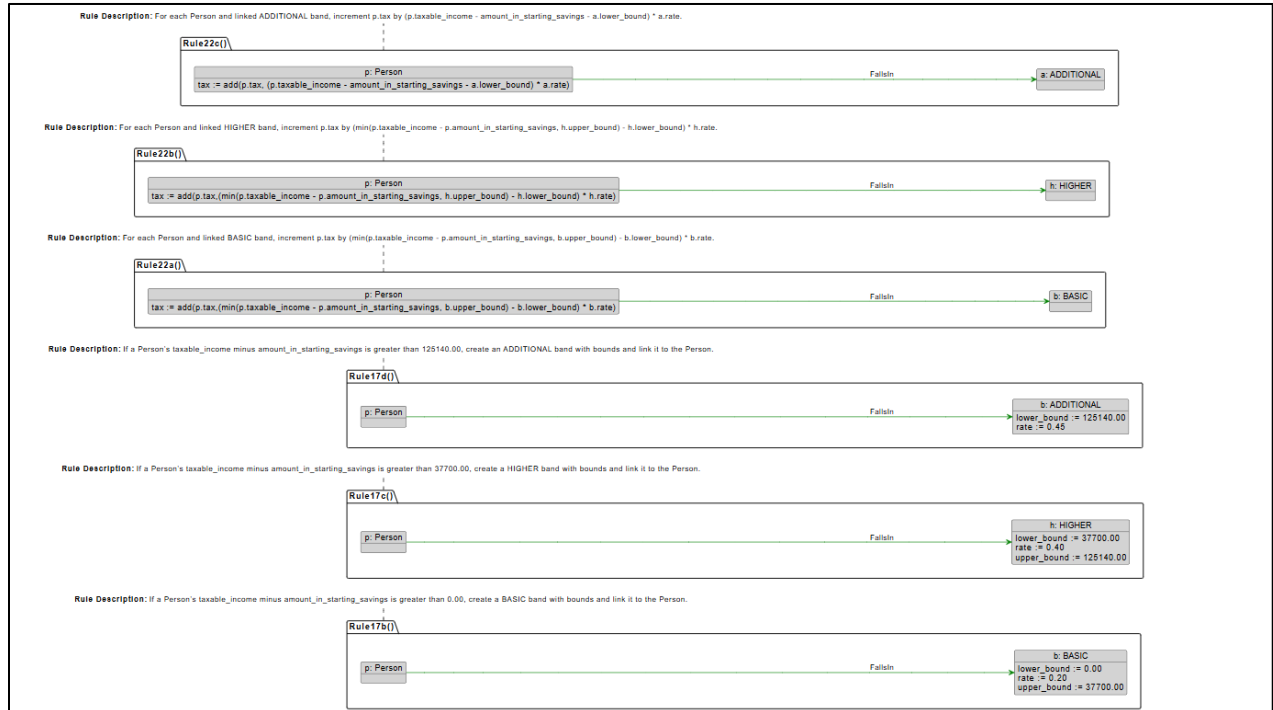
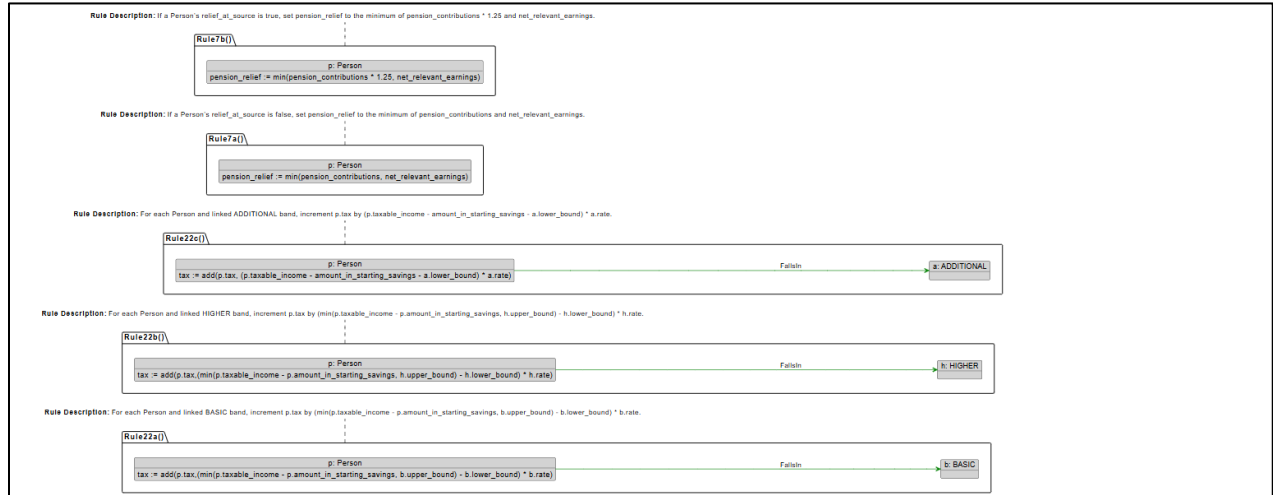
Appendix 2 – UML Diagrams

Note: All these UML are generated by python code can be find in gitlab rapo

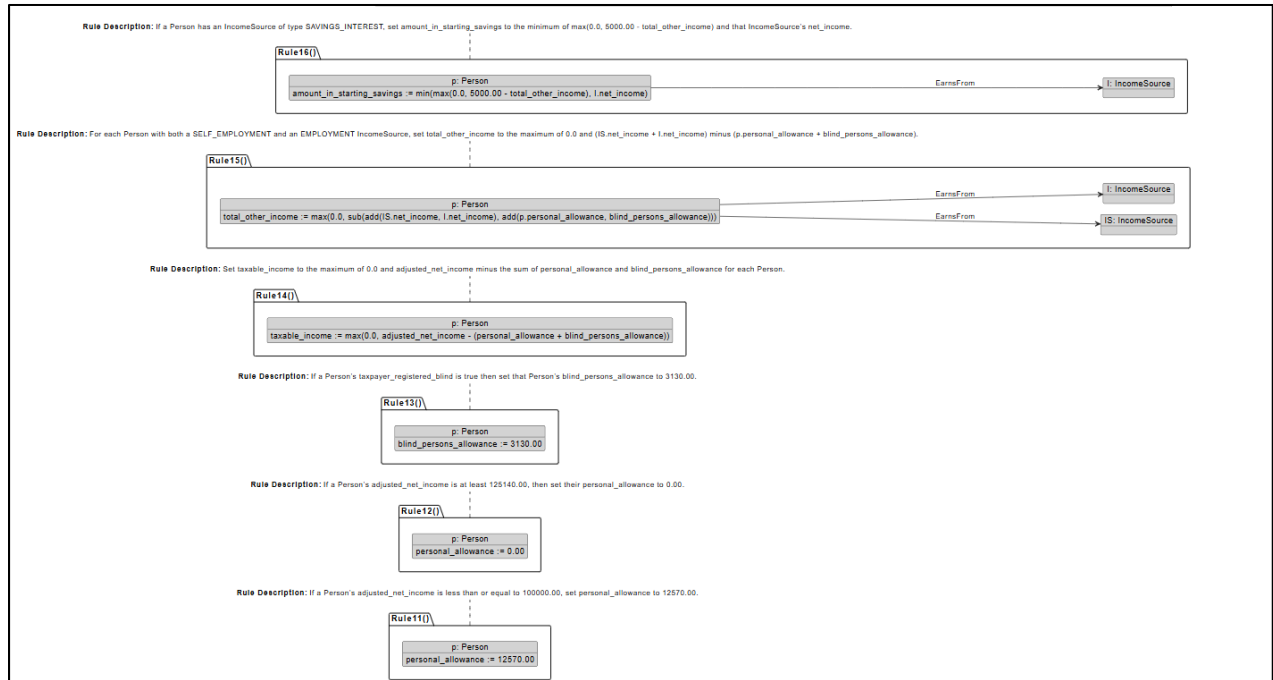
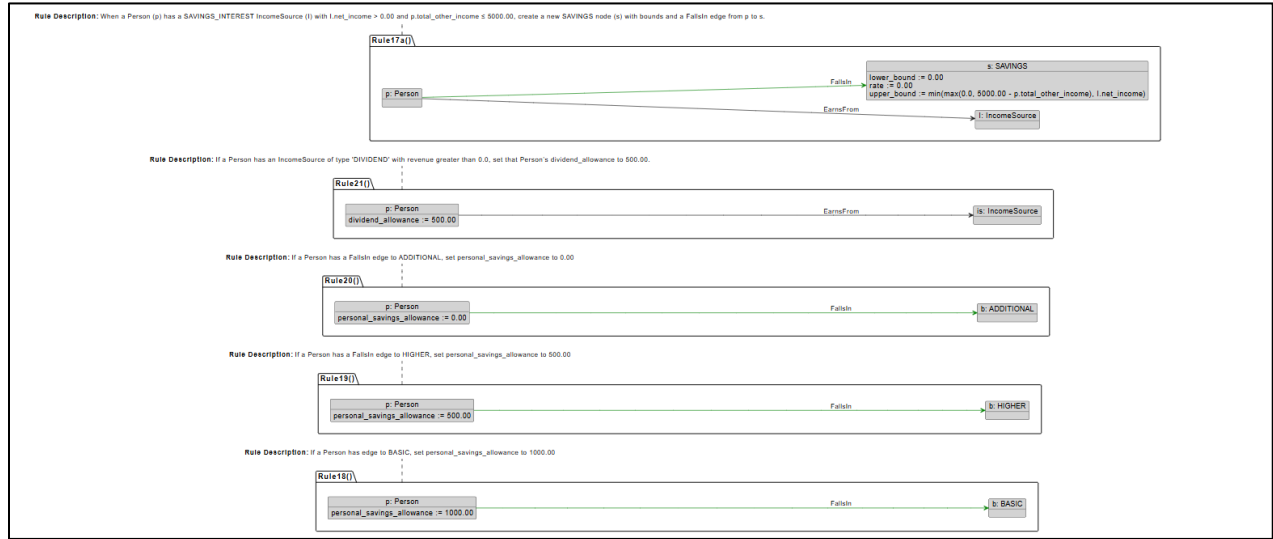
File name: *Helper Python Scripts.ipynb*



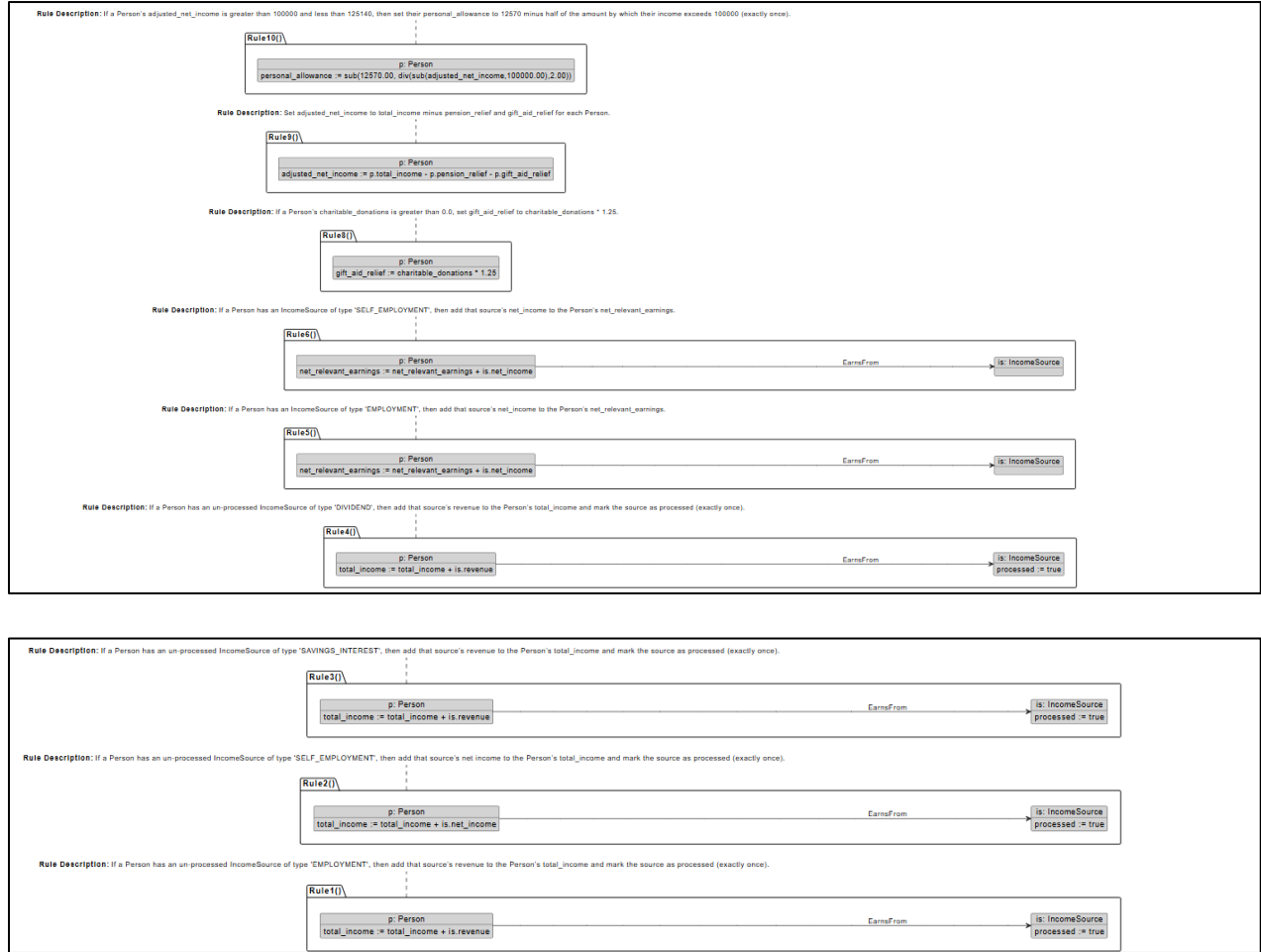
Automated Validation of GPT-Based Tax Query Responses through Graph-Based Reasoning

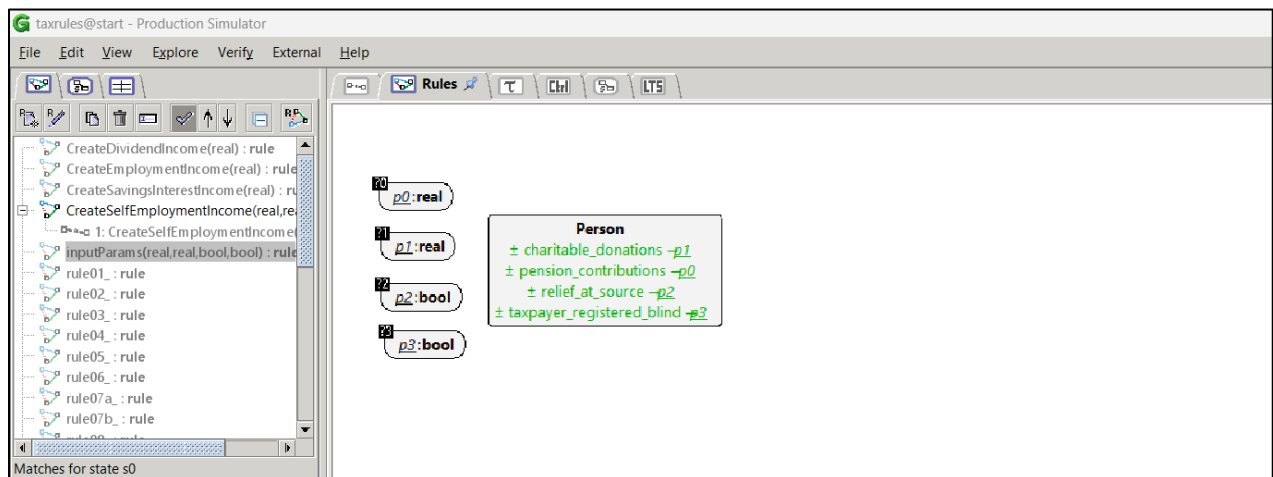
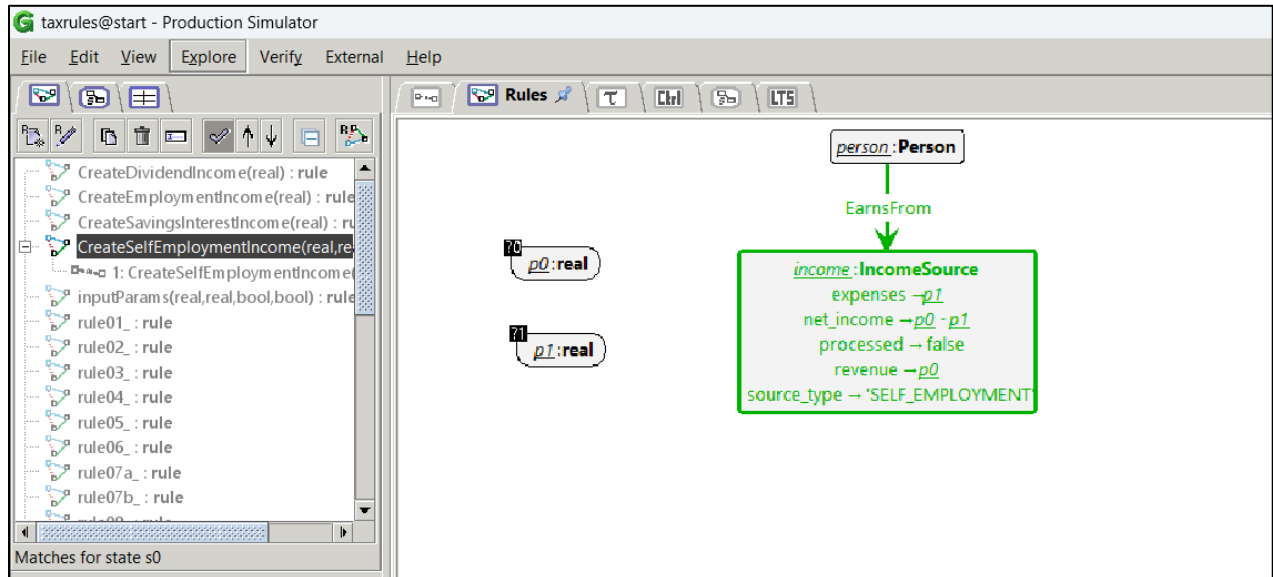
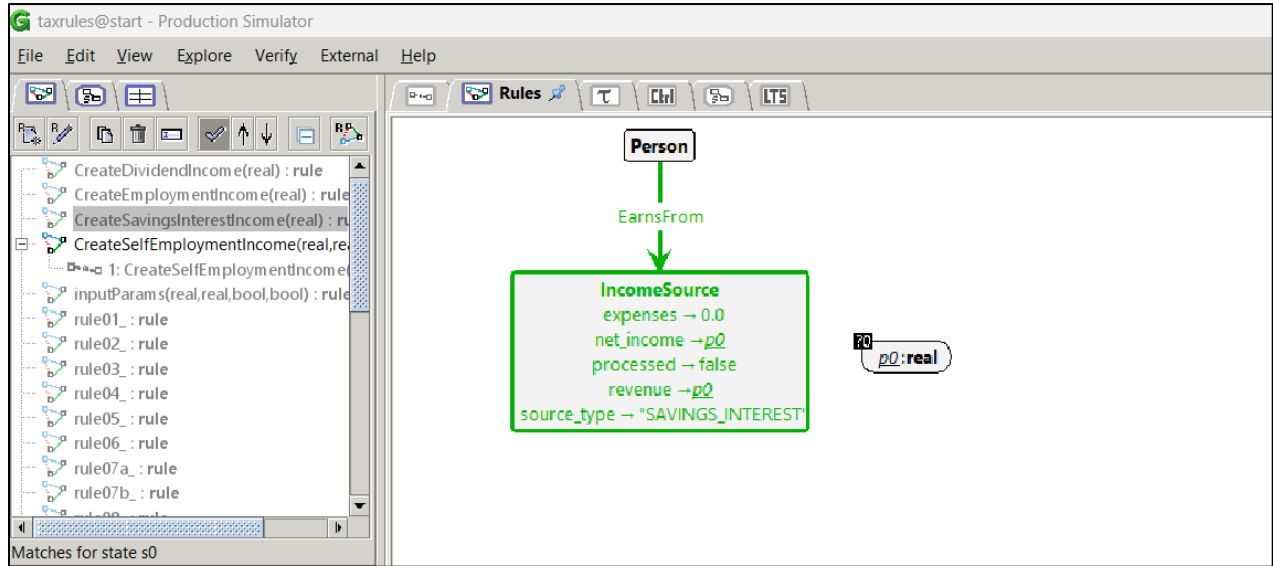


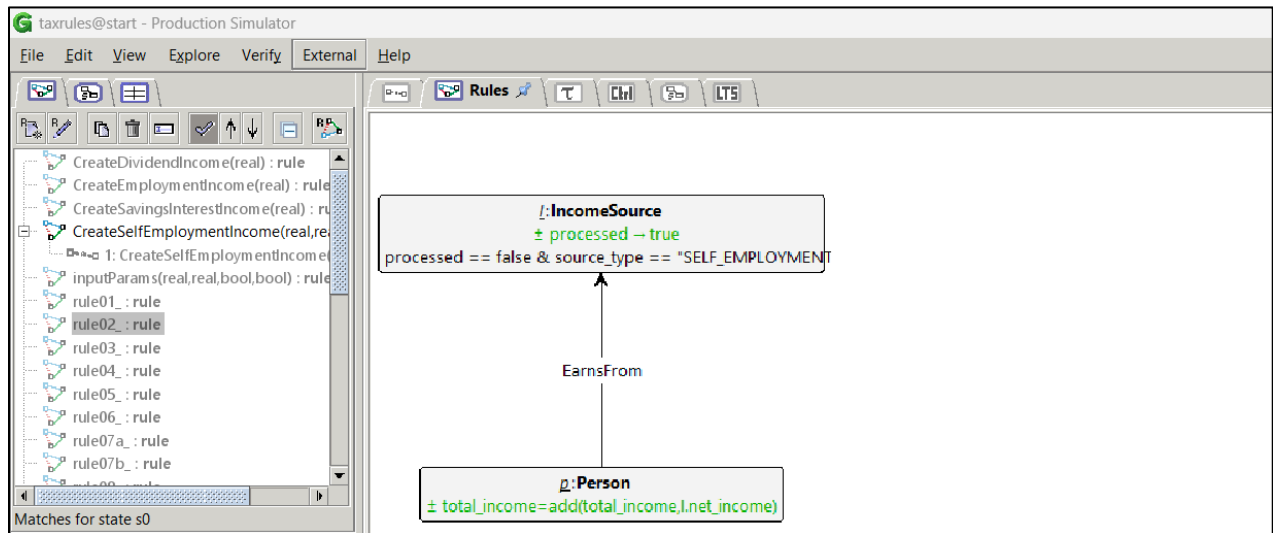
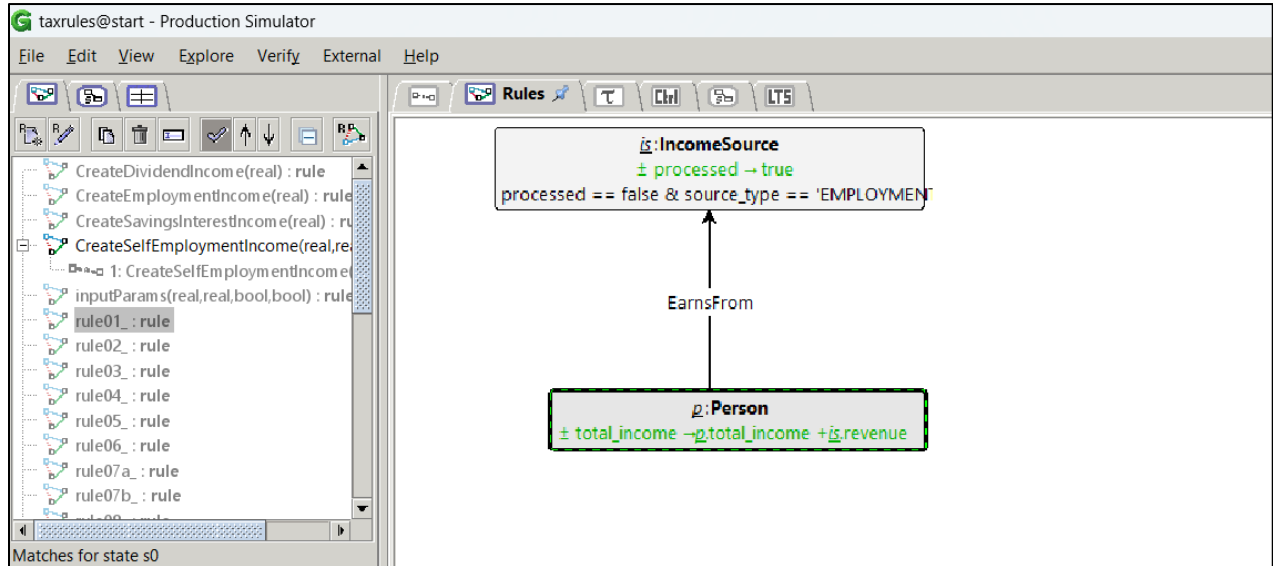
Automated Validation of GPT-Based Tax Query Responses through Graph-Based Reasoning

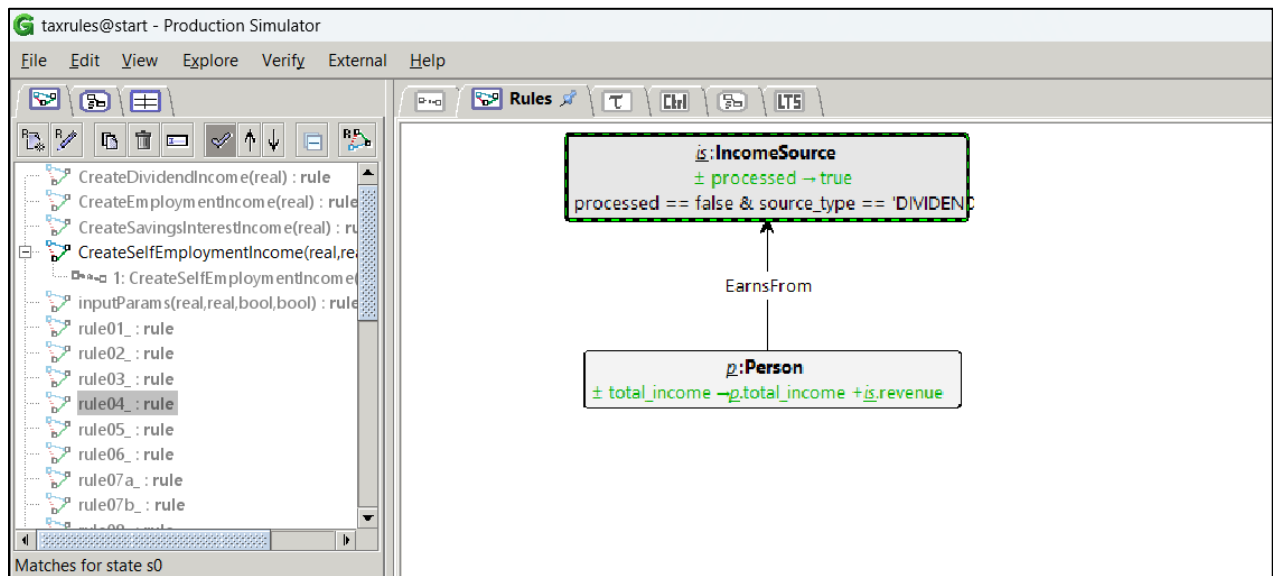
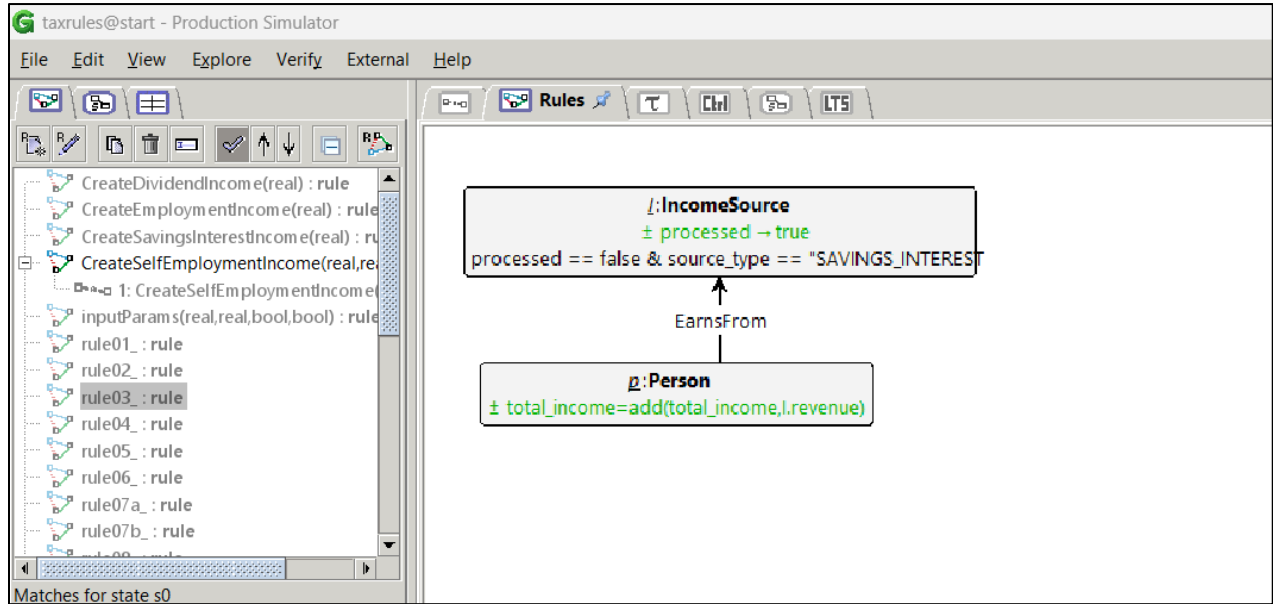


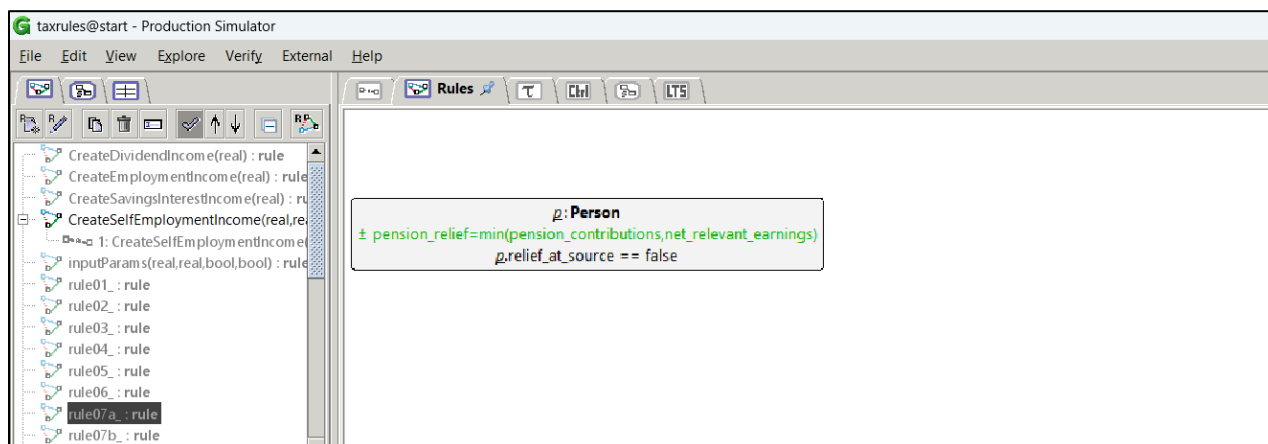
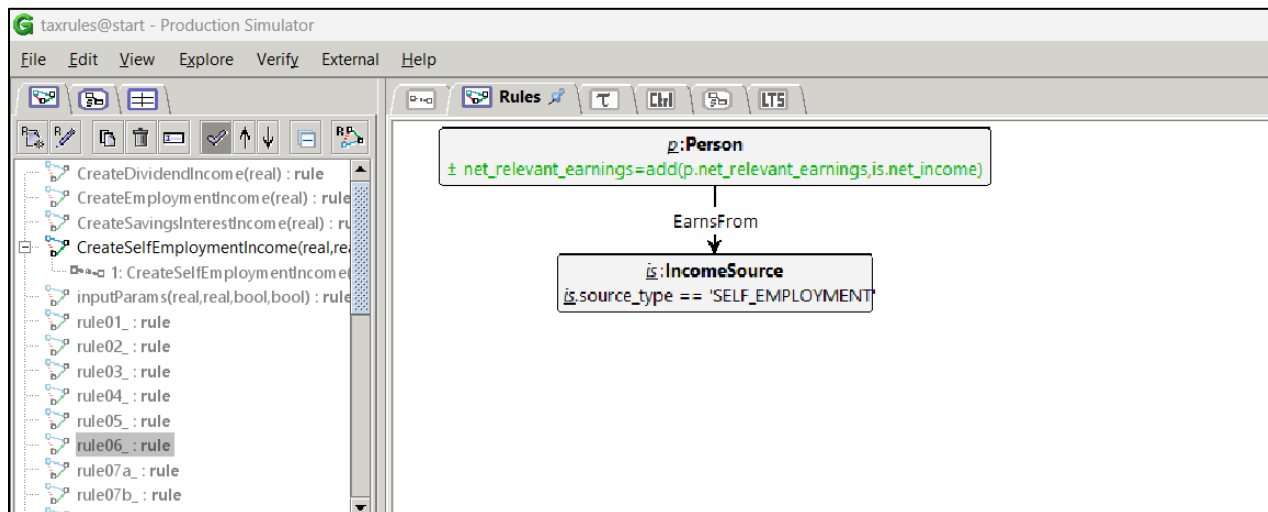
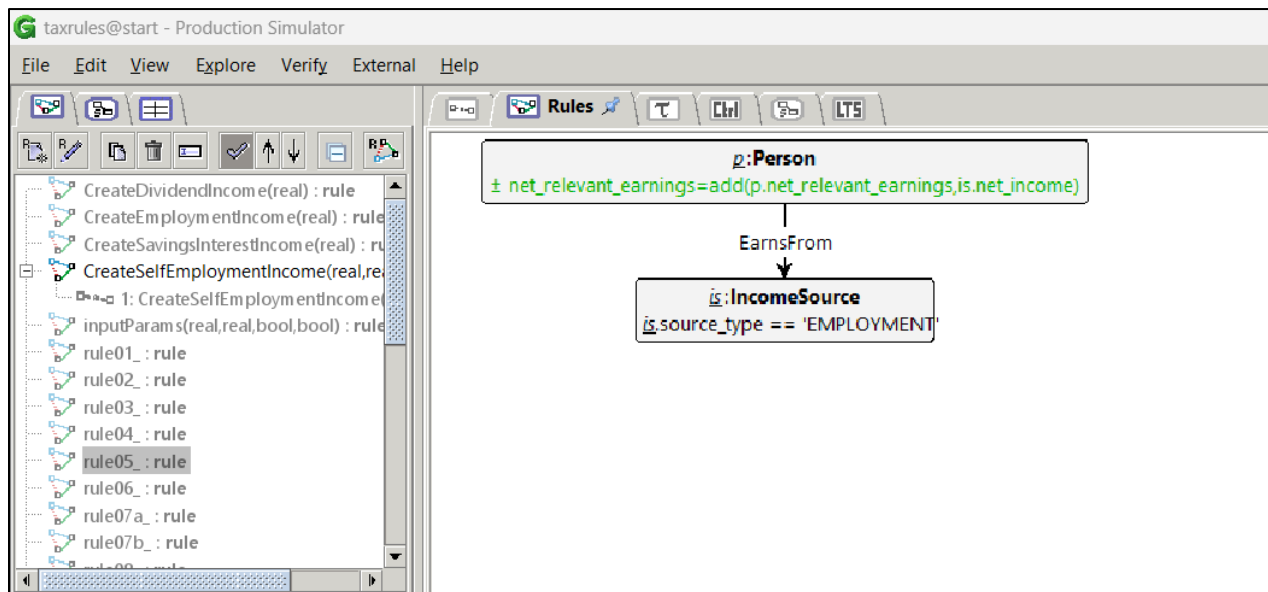
Automated Validation of GPT-Based Tax Query Responses through Graph-Based Reasoning

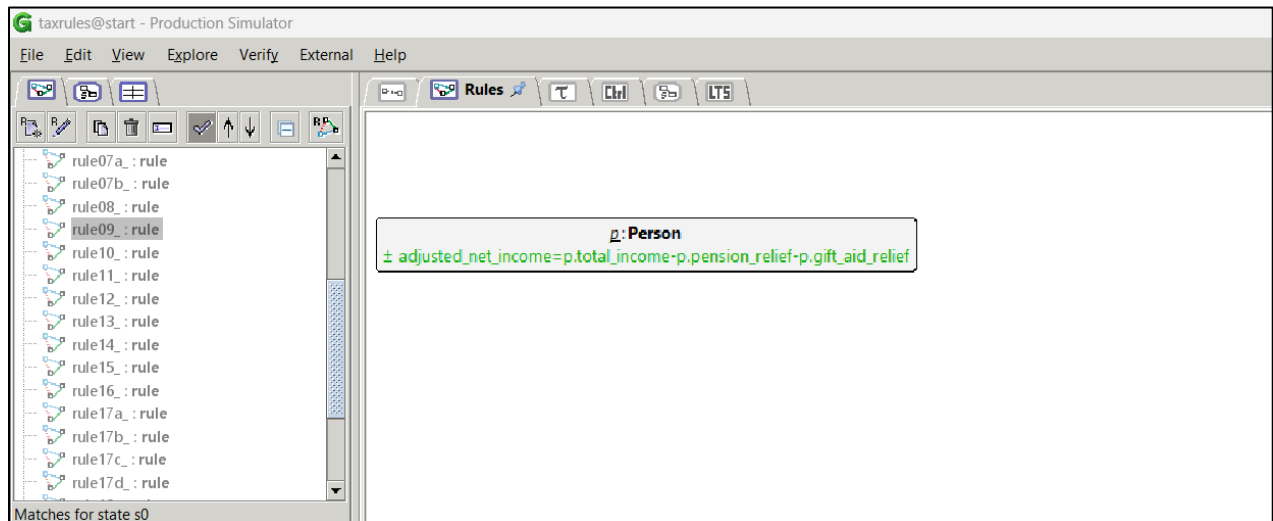
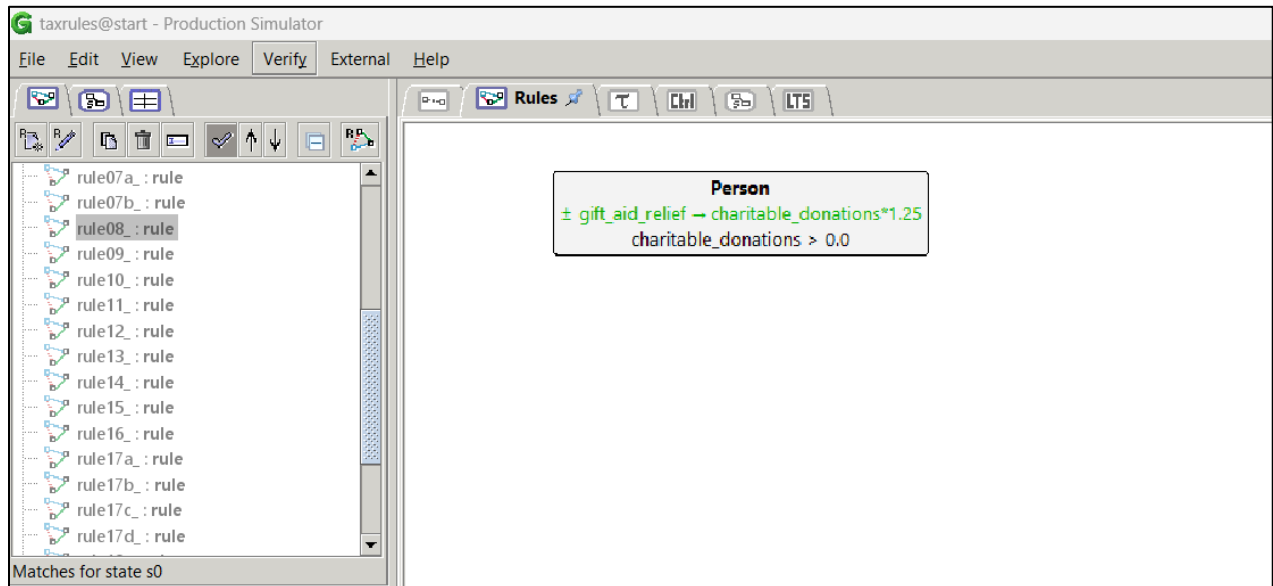
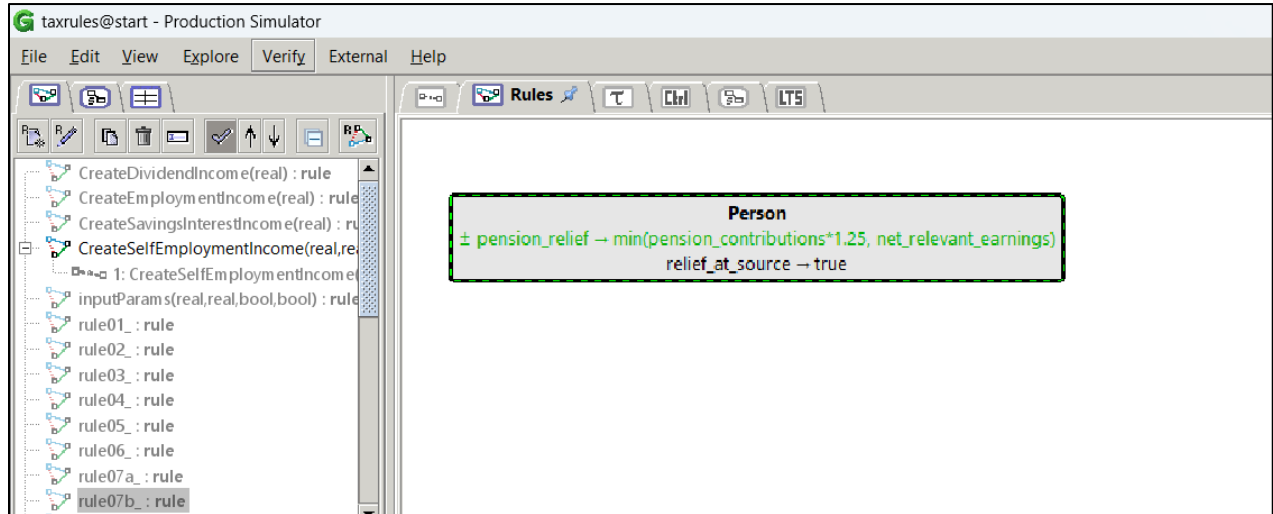




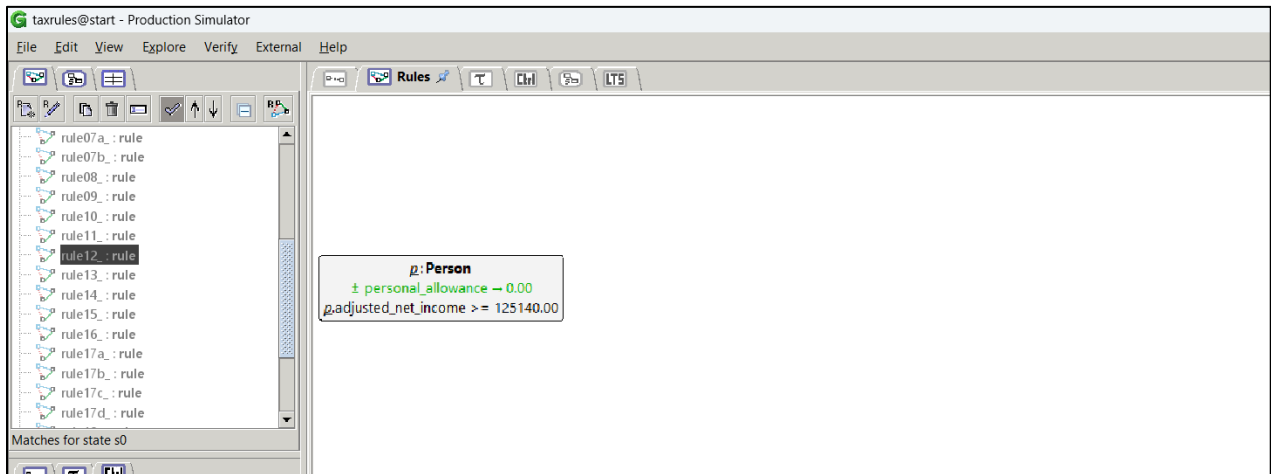
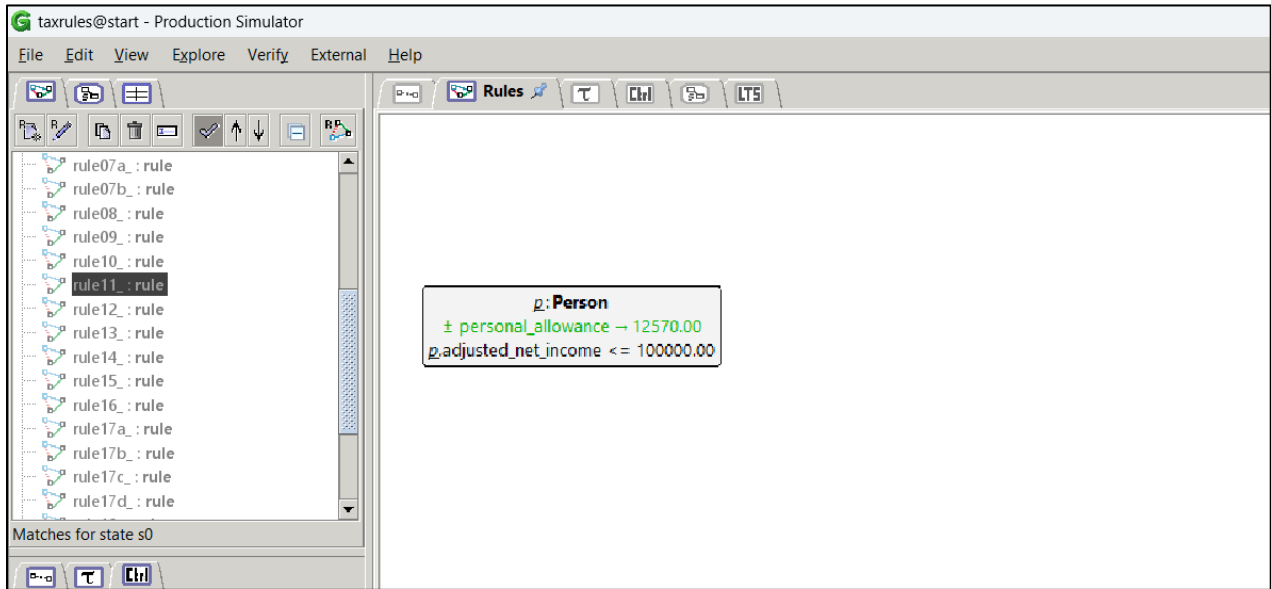
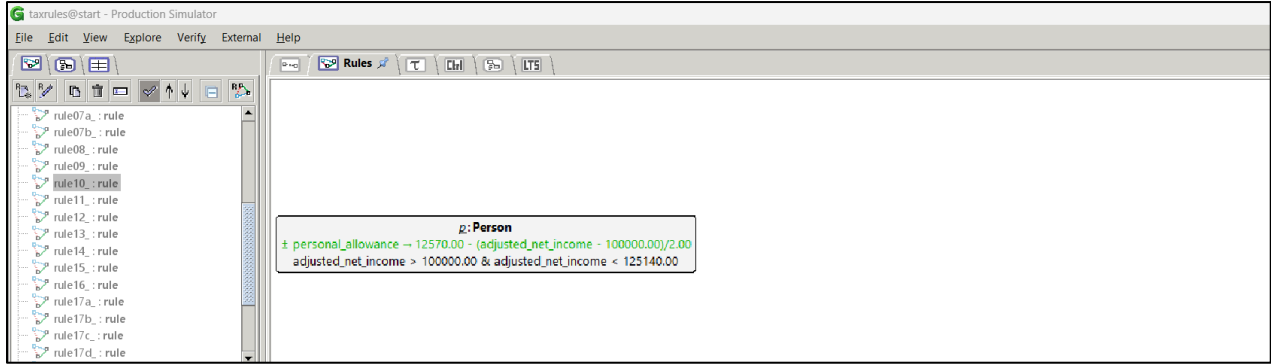




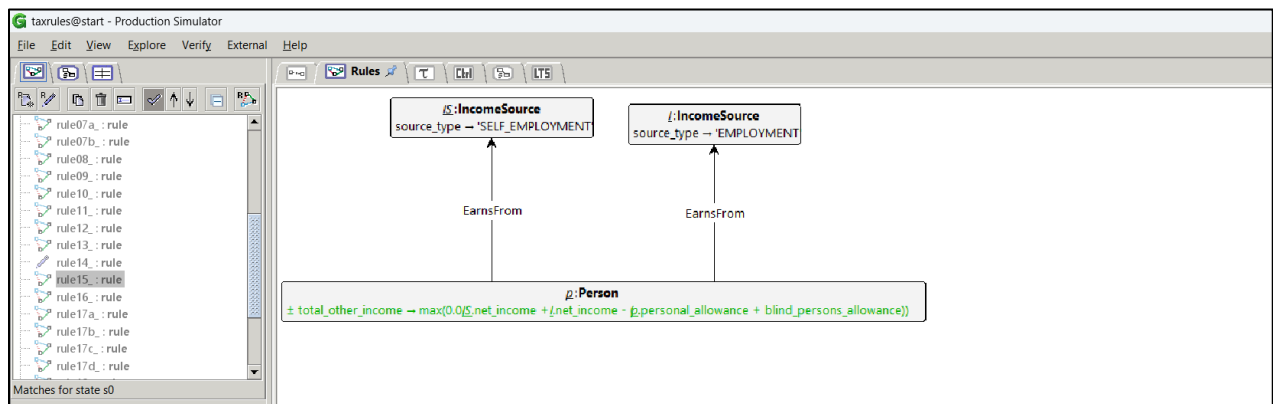
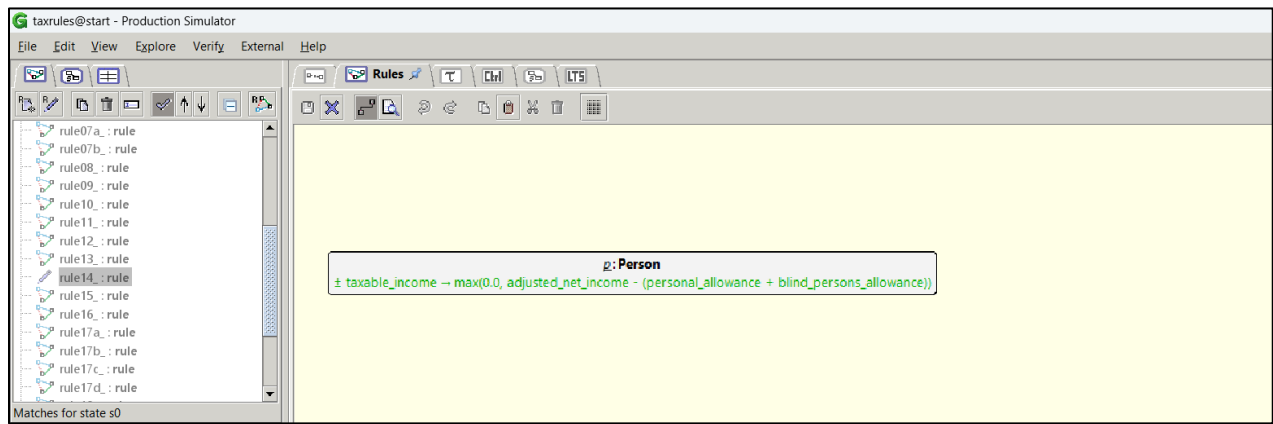
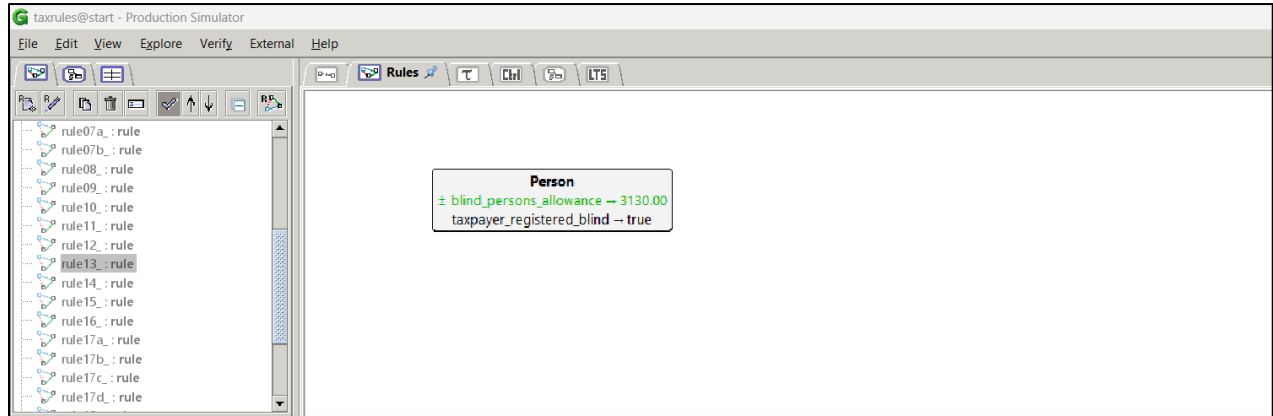




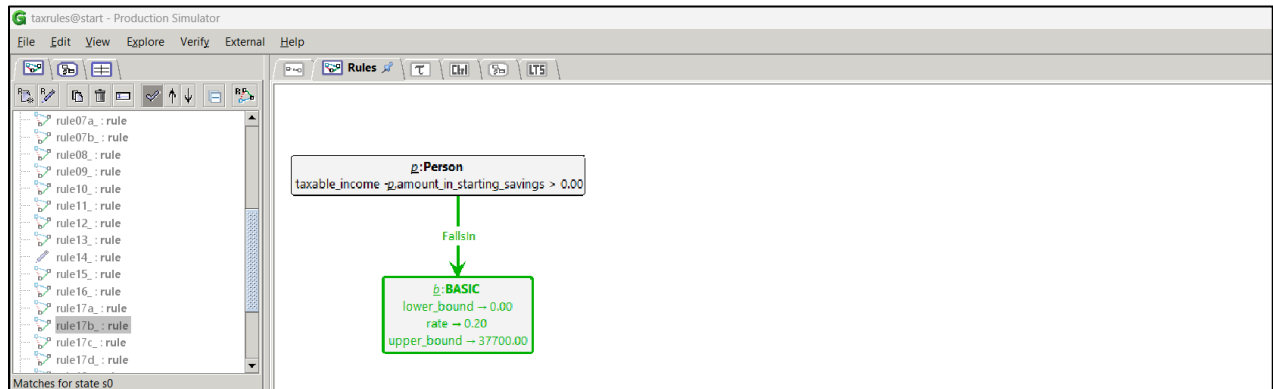
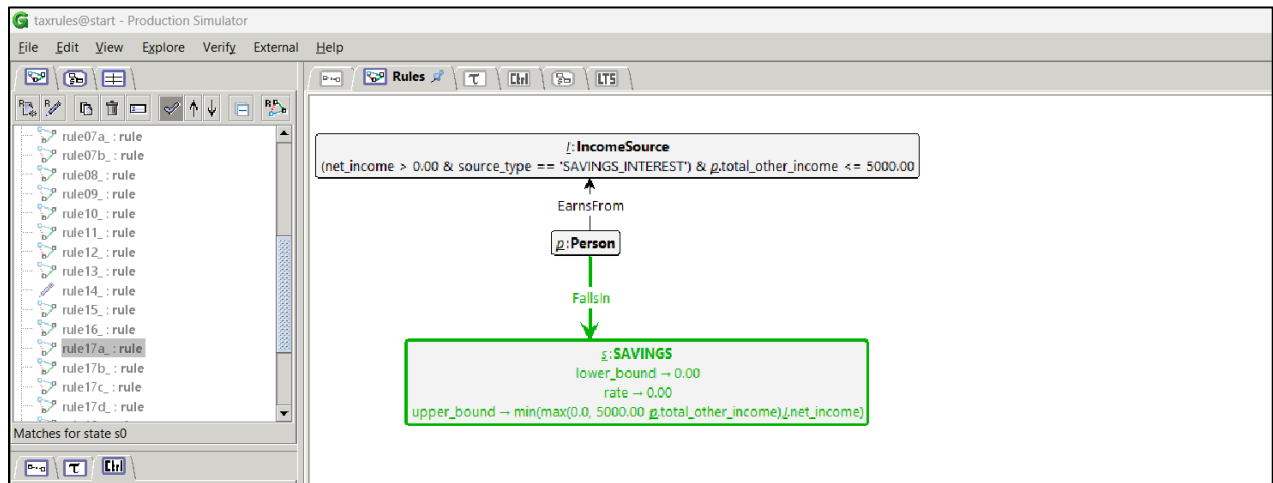
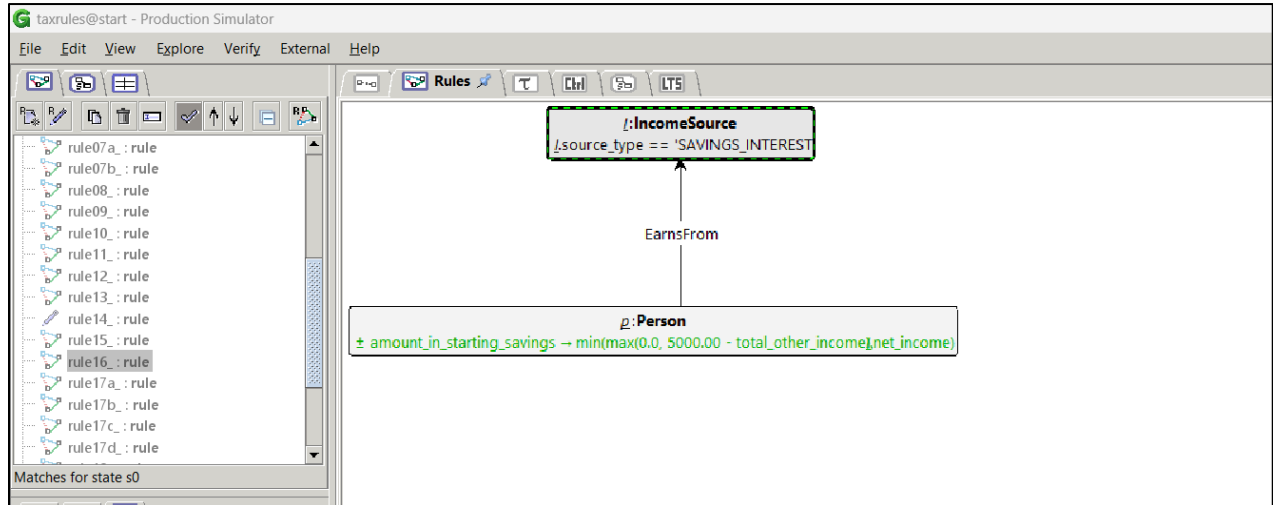
Automated Validation of GPT-Based Tax Query Responses through Graph-Based Reasoning



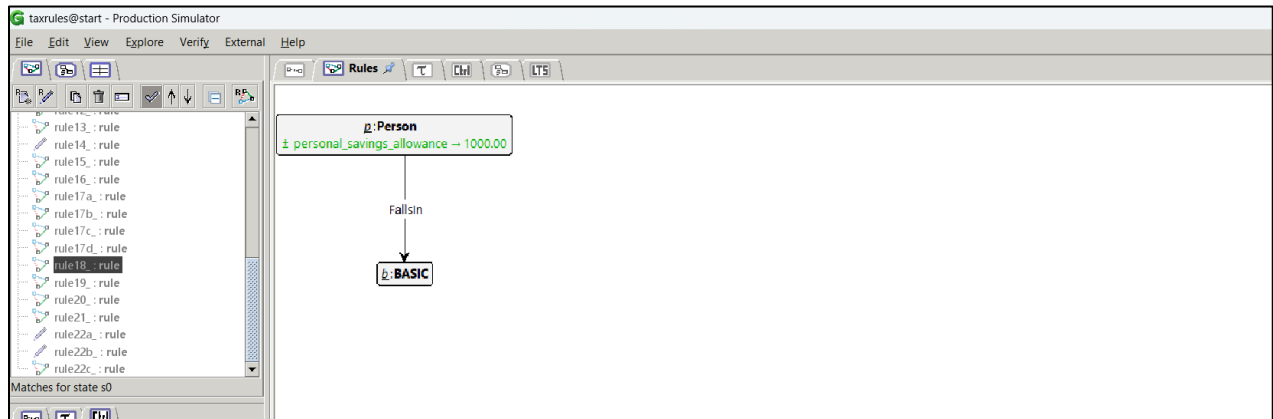
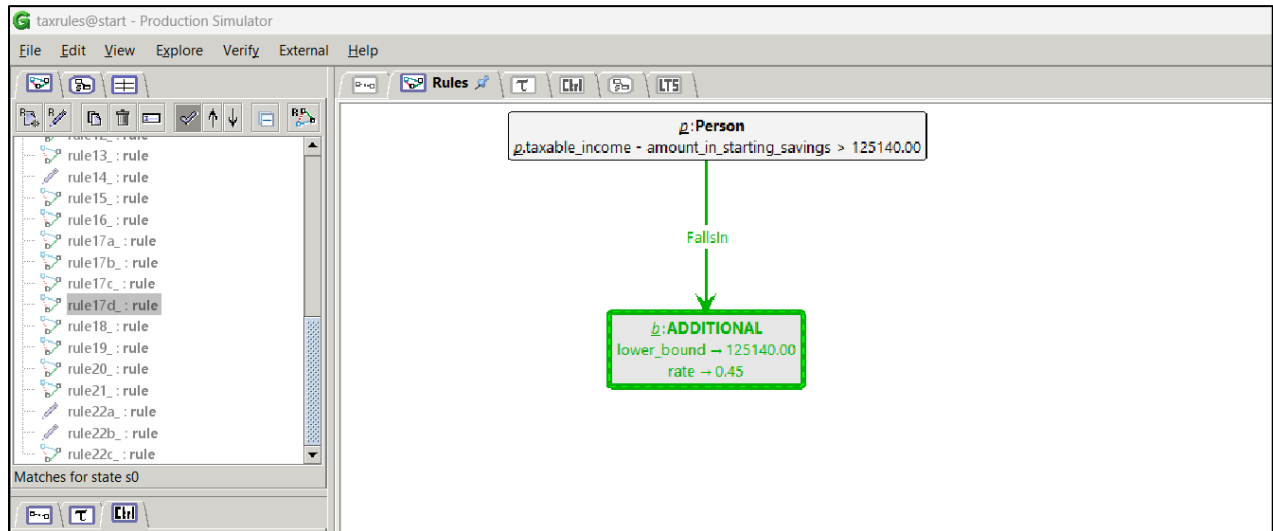
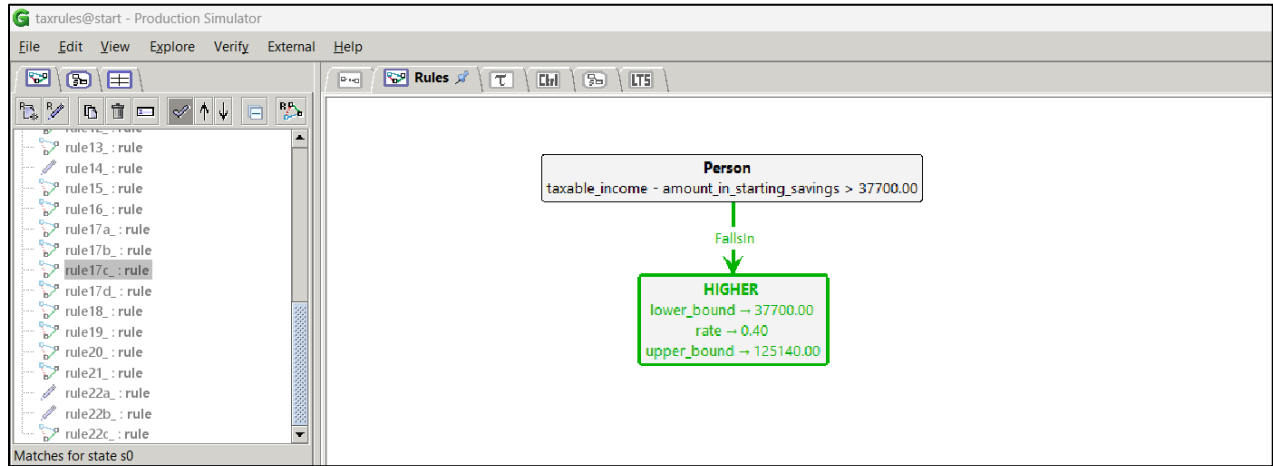
Automated Validation of GPT-Based Tax Query Responses through Graph-Based Reasoning



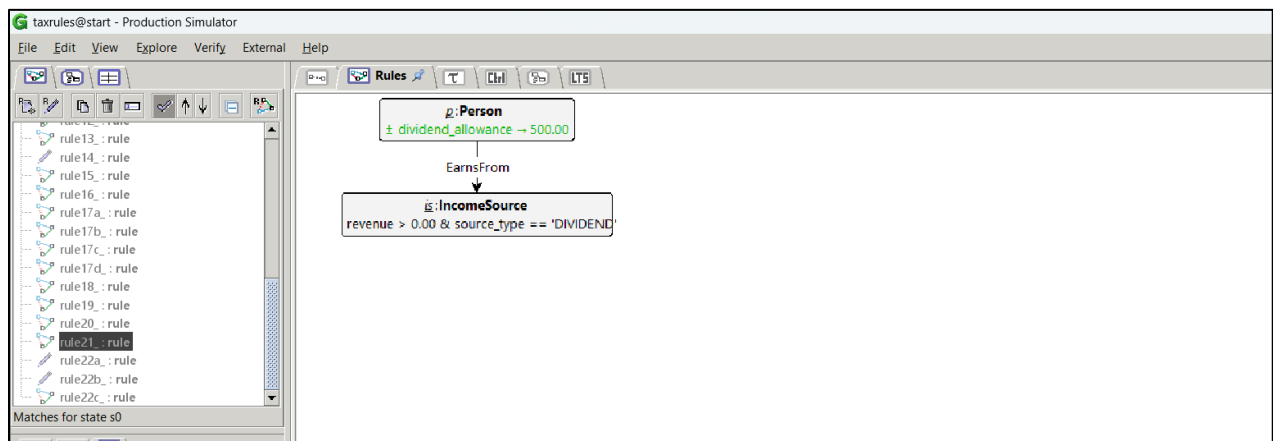
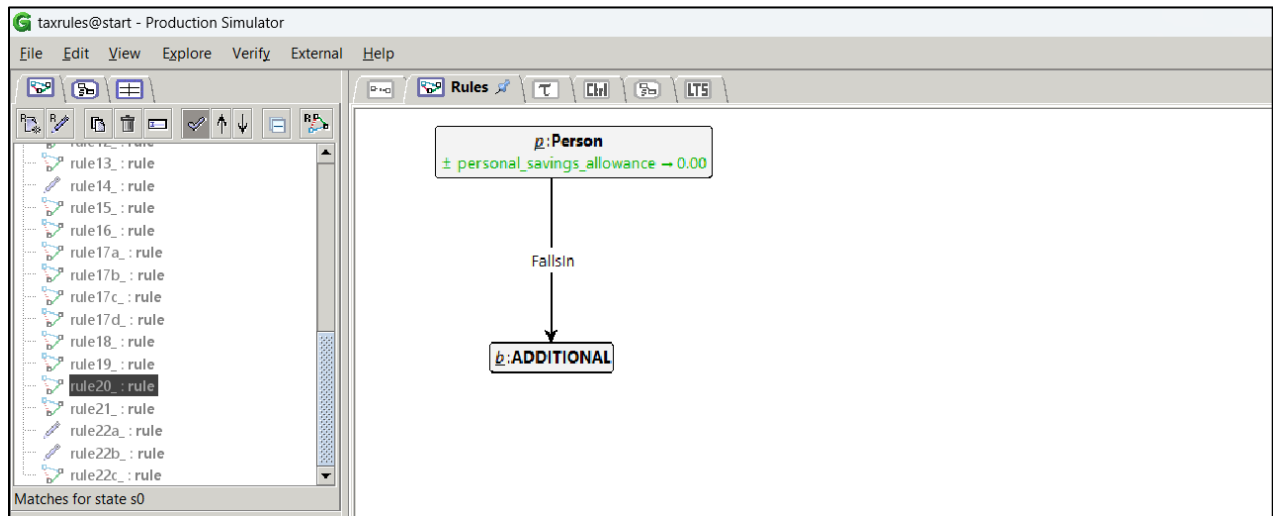
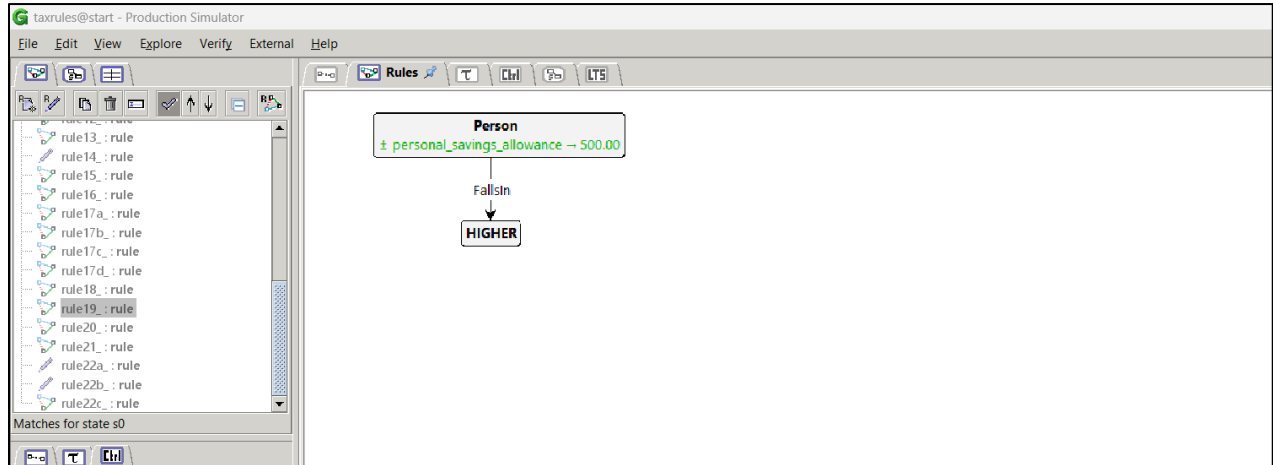
Automated Validation of GPT-Based Tax Query Responses through Graph-Based Reasoning



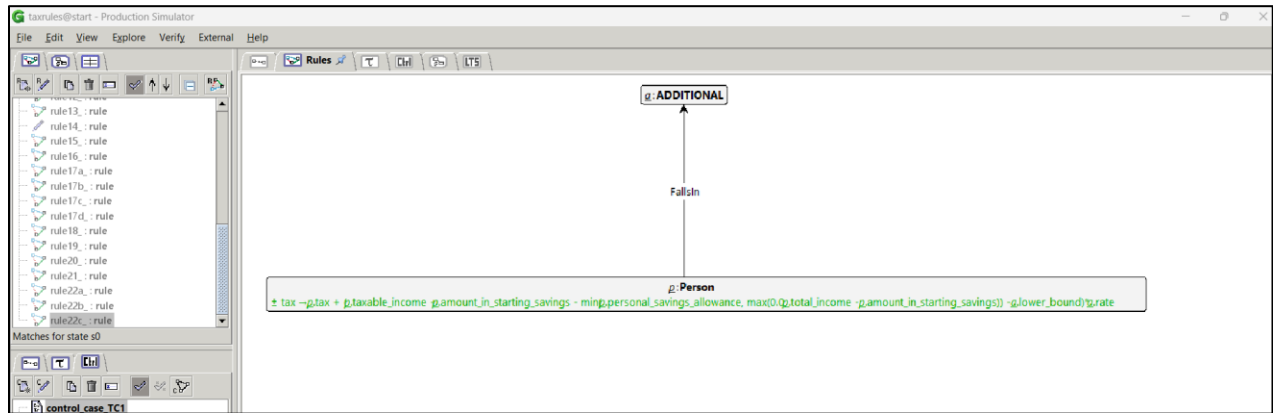
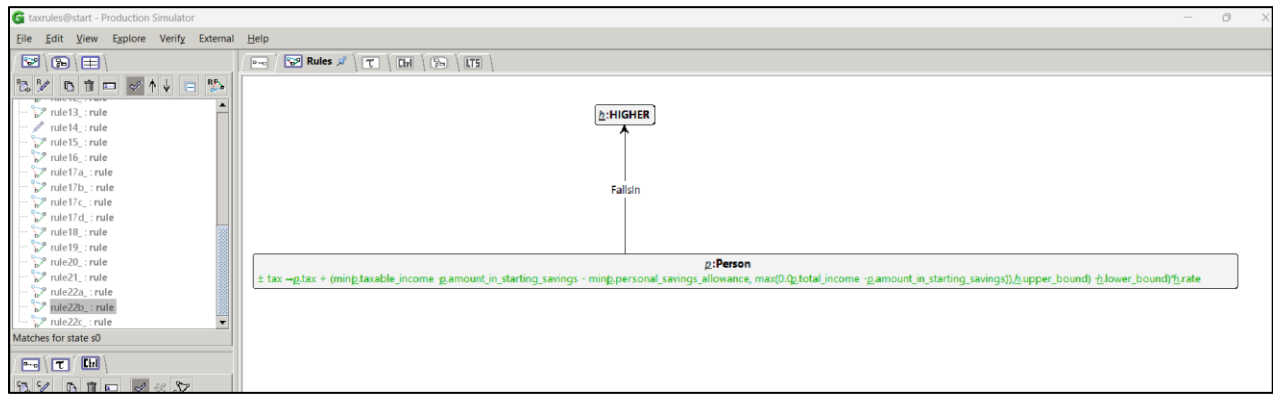
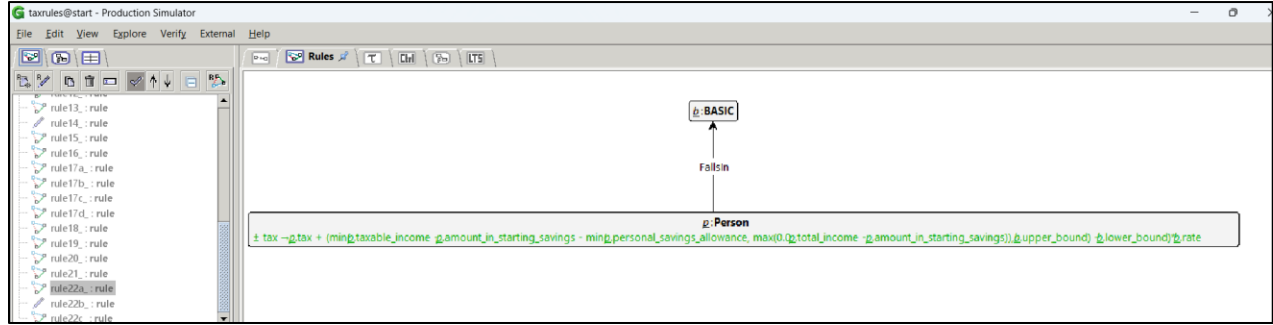
Automated Validation of GPT-Based Tax Query Responses through Graph-Based Reasoning



Automated Validation of GPT-Based Tax Query Responses through Graph-Based Reasoning



Automated Validation of GPT-Based Tax Query Responses through Graph-Based Reasoning



Appendix 4 – XML Structure for GROOVE

Note: all files are located gitlab name [groove-rules.zip](#)

Type Graph

```
<?xml version="1.0" encoding="UTF-8" standalone="yes"?>
<gx1 xmlns="http://www.gupro.de/GXL/gxl-1.0.dtd">
  <graph role="type" edgeids="false" edgemode="directed" id="type">
    <attr name="$version">
      <string>curly</string>
    </attr>
    <node id="n1">
      <attr name="layout">
        <string>222 157 19 19</string>
      </attr>
    </node>
    <node id="n2">
      <attr name="layout">
        <string>795 35 19 19</string>
      </attr>
    </node>
    <node id="n3">
      <attr name="layout">
        <string>796 118 19 19</string>
      </attr>
    </node>
    <node id="n4">
      <attr name="layout">
        <string>793 204 19 19</string>
      </attr>
    </node>
    <node id="n5">
      <attr name="layout">
        <string>794 287 19 19</string>
      </attr>
    </node>
    <node id="n0">
      <attr name="layout">
        <string>389 7 185 324</string>
      </attr>
    </node>
  </graph>

```

```
<edge from="n1" to="n1">
  <attr name="label">
    <string>bool:processed</string>
  </attr>
</edge>
<edge from="n1" to="n1">
  <attr name="label">
    <string>real:expenses</string>
  </attr>
</edge>
<edge from="n1" to="n1">
  <attr name="label">
    <string>real:net_income</string>
  </attr>
</edge>
<edge from="n1" to="n1">
  <attr name="label">
    <string>real:revenue</string>
  </attr>
</edge>
<edge from="n1" to="n1">
  <attr name="label">
    <string>string:source_type</string>
  </attr>
</edge>
<edge from="n2" to="n2">
  <attr name="label">
    <string>type:SAVINGS</string>
  </attr>
</edge>
<edge from="n2" to="n2">
  <attr name="label">
    <string>real:lower_bound</string>
  </attr>
</edge>

```

```

</edge>
<edge from="n2" to="n2">
  <attr name="label">
    <string>real:lower_bound</string>
  </attr>
</edge>
<edge from="n2" to="n2">
  <attr name="label">
    <string>real:rate</string>
  </attr>
</edge>
<edge from="n2" to="n2">
  <attr name="label">
    <string>real:upper_bound</string>
  </attr>
</edge>
<edge from="n3" to="n3">
  <attr name="label">
    <string>type:BASIC</string>
  </attr>
</edge>
<edge from="n3" to="n3">
  <attr name="label">
    <string>real:lower_bound</string>
  </attr>
</edge>
<edge from="n3" to="n3">
  <attr name="label">
    <string>real:rate</string>
  </attr>
</edge>
<edge from="n3" to="n3">
  <attr name="label">
    <string>real:upper_bound</string>
  </attr>
</edge>
</graph>
</gxl>

```

```

</edge>
<edge from="n0" to="n0">
  <attr name="label">
    <string>real:total_income</string>
  </attr>
</edge>
<edge from="n0" to="n0">
  <attr name="label">
    <string>real:total_other_income</string>
  </attr>
</edge>
<edge from="n0" to="n4">
  <attr name="label">
    <string>FallsIn</string>
  </attr>
</edge>
<edge from="n0" to="n2">
  <attr name="label">
    <string>FallsIn</string>
  </attr>
</edge>
<edge from="n0" to="n1">
  <attr name="label">
    <string>EarnsFrom</string>
  </attr>
</edge>
<edge from="n0" to="n3">
  <attr name="label">
    <string>FallsIn</string>
  </attr>
</edge>
<edge from="n0" to="n5">
  <attr name="label">
    <string>FallsIn</string>
  </attr>
</edge>
</graph>
</gxl>

```


Start Graph XML

```
<?xml version="1.0" encoding="UTF-8" standalone="yes"?>
<gxl xmlns="http://www.gupro.de/GXL/gxl-1.0.dtd">
  <graph role="graph" edgeids="false" edgemode="directed" id="start">
    <attr name="$version">
      <string>curly</string>
    </attr>
    <node id="n0">
      <attr name="layout">
        <string>36 37 195 324</string>
      </attr>
    </node>
    <edge from="n0" to="n0">
      <attr name="label">
        <string>type:Person</string>
      </attr>
    </edge>
    <edge from="n0" to="n0">
      <attr name="label">
        <string>let:adjusted_net_income=0.0</string>
      </attr>
    </edge>
    <edge from="n0" to="n0">
      <attr name="label">
        <string>let:amount_in_psa=0.0</string>
      </attr>
    </edge>
    <edge from="n0" to="n0">
      <attr name="label">
        <string>let:amount_in_starting_savings=0.0</string>
      </attr>
    </edge>
    <edge from="n0" to="n0">
      <attr name="label">
        <string>let:blind_persons_allowance=0.0</string>
      </attr>
    </edge>
  </graph>
</gxl>
```

```
    <edge from="n0" to="n0">
      <attr name="label">
        <string>let:charitable_donations=0.0</string>
      </attr>
    </edge>
    <edge from="n0" to="n0">
      <attr name="label">
        <string>let:dividend_allowance=0.0</string>
      </attr>
    </edge>
    <edge from="n0" to="n0">
      <attr name="label">
        <string>let:gift_aid_relief=0.0</string>
      </attr>
    </edge>
    <edge from="n0" to="n0">
      <attr name="label">
        <string>let:net_relevant_earnings=0.0</string>
      </attr>
    </edge>
    <edge from="n0" to="n0">
      <attr name="label">
        <string>let:pension_contributions=0.0</string>
      </attr>
    </edge>
    <edge from="n0" to="n0">
      <attr name="label">
        <string>let:pension_relief=0.0</string>
      </attr>
    </edge>
    <edge from="n0" to="n0">
      <attr name="label">
        <string>let:personal_allowance=12570.0</string>
      </attr>
    </edge>
  </graph>
</gxl>
```

```

<edge from="n0" to="n0">
  <attr name="label">
    <string>let:personal_allowance=12570.0</string>
  </attr>
</edge>
<edge from="n0" to="n0">
  <attr name="label">
    <string>let:personal_savings_allowance=0.0</string>
  </attr>
</edge>
<edge from="n0" to="n0">
  <attr name="label">
    <string>let:relief_at_source=false</string>
  </attr>
</edge>
<edge from="n0" to="n0">
  <attr name="label">
    <string>let:tax=0.0</string>
  </attr>
</edge>
<edge from="n0" to="n0">
  <attr name="label">
    <string>let:taxable_income=0.0</string>
  </attr>
</edge>
<edge from="n0" to="n0">
  <attr name="label">
    <string>let:taxpayer_registered_blind=false</string>
  </attr>
</edge>
<edge from="n0" to="n0">
  <attr name="label">
    <string>let:total_income=0.0</string>
  </attr>
</edge>

```

```

    <string>let:personal_allowance=12570.0</string>
  </attr>
</edge>
<edge from="n0" to="n0">
  <attr name="label">
    <string>let:personal_savings_allowance=0.0</string>
  </attr>
</edge>
<edge from="n0" to="n0">
  <attr name="label">
    <string>let:relief_at_source=false</string>
  </attr>
</edge>
<edge from="n0" to="n0">
  <attr name="label">
    <string>let:tax=0.0</string>
  </attr>
</edge>
<edge from="n0" to="n0">
  <attr name="label">
    <string>let:taxable_income=0.0</string>
  </attr>
</edge>
<edge from="n0" to="n0">
  <attr name="label">
    <string>let:taxpayer_registered_blind=false</string>
  </attr>
</edge>
<edge from="n0" to="n0">
  <attr name="label">
    <string>let:total_income=0.0</string>
  </attr>
</edge>
<edge from="n0" to="n0">
  <attr name="label">
    <string>let:total_other_income=0.0</string>
  </attr>
</edge>
</graph>
</gxl>

```

Appendix 5 – Structure for GROOVE Control Files

Control Files

TC-1

```
<
  CreateSelfEmploymentIncome(50000.00, 0.00);
  CreateDividendIncome(0.00);
  CreateEmploymentIncome(0.00);
  CreateSavingsInterestIncome(0.00);
  inputParams(0.00, 0.00, false, false);
>
try { rule01_(); } else {}
try { rule02_(); } else {}
try { rule03_(); } else {}
try { rule04_(); } else {}
try { rule05_(); } else {}
try { rule06_(); } else {}
try { rule07a_(); } else {}
try { rule07b_(); } else {}
try { rule08_(); } else {}
try { rule09_(); } else {}
try { rule10_(); } else {}
try { rule11_(); } else {}
try { rule12_(); } else {}
try { rule13_(); } else {}
try { rule14_(); } else {}
try { rule15_(); } else {}
try { rule16_(); } else {}
try { rule17a_(); } else {}
try { rule17b_(); } else {}
try { rule17c_(); } else {}
try { rule17d_(); } else {}
try { rule18_(); } else {}
try { rule19_(); } else {}
try { rule20_(); } else {}
try { rule21_(); } else {}
try { rule22a_(); } else {}
try { rule22b_(); } else {}
try { rule22c_(); } else {}
```

TC-2

```
<
  CreateSelfEmploymentIncome(50000.00, 10000.00);
  CreateDividendIncome(0.00);
  CreateEmploymentIncome(0.00);
  CreateSavingsInterestIncome(0.00);
  inputParams(0.00, 0.00, false, false);
>
try { rule01_(); } else {}
try { rule02_(); } else {}
try { rule03_(); } else {}
try { rule04_(); } else {}
try { rule05_(); } else {}
try { rule06_(); } else {}
try { rule07a_(); } else {}
try { rule07b_(); } else {}
try { rule08_(); } else {}
try { rule09_(); } else {}
try { rule10_(); } else {}
try { rule11_(); } else {}
try { rule12_(); } else {}
try { rule13_(); } else {}
try { rule14_(); } else {}
try { rule15_(); } else {}
try { rule16_(); } else {}
try { rule17a_(); } else {}
try { rule17b_(); } else {}
try { rule17c_(); } else {}
try { rule17d_(); } else {}
try { rule18_(); } else {}
try { rule19_(); } else {}
try { rule20_(); } else {}
try { rule21_(); } else {}
try { rule22a_(); } else {}
try { rule22b_(); } else {}
try { rule22c_(); } else {}
```

TC-3

```

<
  CreateSelfEmploymentIncome(0.00, 0.00);
  CreateDividendIncome(0.00);
  CreateEmploymentIncome(20000.00);
  CreateSavingsInterestIncome(800.00);
  inputParams(0.00, 0.00, false, false);
>
try { rule01_(); } else {}
try { rule02_(); } else {}
try { rule03_(); } else {}
try { rule04_(); } else {}
try { rule05_(); } else {}
try { rule06_(); } else {}
try { rule07a_(); } else {}
try { rule07b_(); } else {}
try { rule08_(); } else {}
try { rule09_(); } else {}
try { rule10_(); } else {}
try { rule11_(); } else {}
try { rule12_(); } else {}
try { rule13_(); } else {}
try { rule14_(); } else {}
try { rule15_(); } else {}
try { rule16_(); } else {}
try { rule17a_(); } else {}
try { rule17b_(); } else {}
try { rule17c_(); } else {}
try { rule17d_(); } else {}
try { rule18_(); } else {}
try { rule19_(); } else {}
try { rule20_(); } else {}
try { rule21_(); } else {}
try { rule22a_(); } else {}
try { rule22b_(); } else {}
try { rule22c_(); } else {}

```

TC-4

```

<
  CreateSelfEmploymentIncome(0.00, 0.00);
  CreateDividendIncome(0.00);
  CreateEmploymentIncome(20000.00);
  CreateSavingsInterestIncome(1200.00);
  inputParams(0.00, 0.00, false, false);
>
try { rule01_(); } else {}
try { rule02_(); } else {}
try { rule03_(); } else {}
try { rule04_(); } else {}
try { rule05_(); } else {}
try { rule06_(); } else {}
try { rule07a_(); } else {}
try { rule07b_(); } else {}
try { rule08_(); } else {}
try { rule09_(); } else {}
try { rule10_(); } else {}
try { rule11_(); } else {}
try { rule12_(); } else {}
try { rule13_(); } else {}
try { rule14_(); } else {}
try { rule15_(); } else {}
try { rule16_(); } else {}
try { rule17a_(); } else {}
try { rule17b_(); } else {}
try { rule17c_(); } else {}
try { rule17d_(); } else {}
try { rule18_(); } else {}
try { rule19_(); } else {}
try { rule20_(); } else {}
try { rule21_(); } else {}
try { rule22a_(); } else {}
try { rule22b_(); } else {}
try { rule22c_(); } else {}

```

TC- 5

```

<
  CreateSelfEmploymentIncome(0.00, 0.00);
  CreateDividendIncome(2000.00);
  CreateEmploymentIncome(0.00);
  CreateSavingsInterestIncome(0.00);
  inputParams(0.00, 0.00, false, false);
>
try { rule01_(); } else {}
try { rule02_(); } else {}
try { rule03_(); } else {}
try { rule04_(); } else {}
try { rule05_(); } else {}
try { rule06_(); } else {}
try { rule07a_(); } else {}
try { rule07b_(); } else {}
try { rule08_(); } else {}
try { rule09_(); } else {}
try { rule10_(); } else {}
try { rule11_(); } else {}
try { rule12_(); } else {}
try { rule13_(); } else {}
try { rule14_(); } else {}
try { rule15_(); } else {}
try { rule16_(); } else {}
try { rule17a_(); } else {}
try { rule17b_(); } else {}
try { rule17c_(); } else {}
try { rule17d_(); } else {}
try { rule18_(); } else {}
try { rule19_(); } else {}
try { rule20_(); } else {}
try { rule21_(); } else {}
try { rule22a_(); } else {}
try { rule22b_(); } else {}
try { rule22c_(); } else {}

```

TC-6

```

k
  CreateSelfEmploymentIncome(150000.00, 0.00);
  CreateDividendIncome(0.00);
  CreateEmploymentIncome(0.00);
  CreateSavingsInterestIncome(0.00);
  inputParams(0.00, 0.00, false, false);
>
try { rule01_(); } else {}
try { rule02_(); } else {}
try { rule03_(); } else {}
try { rule04_(); } else {}
try { rule05_(); } else {}
try { rule06_(); } else {}
try { rule07a_(); } else {}
try { rule07b_(); } else {}
try { rule08_(); } else {}
try { rule09_(); } else {}
try { rule10_(); } else {}
try { rule11_(); } else {}
try { rule12_(); } else {}
try { rule13_(); } else {}
try { rule14_(); } else {}
try { rule15_(); } else {}
try { rule16_(); } else {}
try { rule17a_(); } else {}
try { rule17b_(); } else {}
try { rule17c_(); } else {}
try { rule17d_(); } else {}
try { rule18_(); } else {}
try { rule19_(); } else {}
try { rule20_(); } else {}
try { rule21_(); } else {}
try { rule22a_(); } else {}
try { rule22b_(); } else {}
try { rule22c_(); } else {}

```

TC-7

```

k
  CreateSelfEmploymentIncome(0.00, 0.00);
  CreateDividendIncome(0.00);
  CreateEmploymentIncome(40000.00);
  CreateSavingsInterestIncome(0.00);
  inputParams(0.00, 1000.00, false, false);
>
try { rule01_(); } else {}
try { rule02_(); } else {}
try { rule03_(); } else {}
try { rule04_(); } else {}
try { rule05_(); } else {}
try { rule06_(); } else {}
try { rule07a_(); } else {}
try { rule07b_(); } else {}
try { rule08_(); } else {}
try { rule09_(); } else {}
try { rule10_(); } else {}
try { rule11_(); } else {}
try { rule12_(); } else {}
try { rule13_(); } else {}
try { rule14_(); } else {}
try { rule15_(); } else {}
try { rule16_(); } else {}
try { rule17a_(); } else {}
try { rule17b_(); } else {}
try { rule17c_(); } else {}
try { rule17d_(); } else {}
try { rule18_(); } else {}
try { rule19_(); } else {}
try { rule20_(); } else {}
try { rule21_(); } else {}
try { rule22a_(); } else {}
try { rule22b_(); } else {}
try { rule22c_(); } else {}

```

TC-8

```

<
  CreateSelfEmploymentIncome(0.00, 0.00);
  CreateDividendIncome(0.00);
  CreateEmploymentIncome(30000.00);
  CreateSavingsInterestIncome(0.00);
  inputParams(0.00, 0.00, false, true);
>
try { rule01_(); } else {}
try { rule02_(); } else {}
try { rule03_(); } else {}
try { rule04_(); } else {}
try { rule05_(); } else {}
try { rule06_(); } else {}
try { rule07a_(); } else {}
try { rule07b_(); } else {}
try { rule08_(); } else {}
try { rule09_(); } else {}
try { rule10_(); } else {}
try { rule11_(); } else {}
try { rule12_(); } else {}
try { rule13_(); } else {}
try { rule14_(); } else {}
try { rule15_(); } else {}
try { rule16_(); } else {}
try { rule17a_(); } else {}
try { rule17b_(); } else {}
try { rule17c_(); } else {}
try { rule17d_(); } else {}
try { rule18_(); } else {}
try { rule19_(); } else {}
try { rule20_(); } else {}
try { rule21_(); } else {}
try { rule22a_(); } else {}
try { rule22b_(); } else {}
try { rule22c_(); } else {}

```

TC-9

```

k
CreateSelfEmploymentIncome(0.00, 0.00);
CreateDividendIncome(0.00);
CreateEmploymentIncome(50000.00);
CreateSavingsInterestIncome(0.00);
inputParams(5000.00, 0.00, true, false);
>
try { rule01_(); } else {}
try { rule02_(); } else {}
try { rule03_(); } else {}
try { rule04_(); } else {}
try { rule05_(); } else {}
try { rule06_(); } else {}
try { rule07a_(); } else {}
try { rule07b_(); } else {}
try { rule08_(); } else {}
try { rule09_(); } else {}
try { rule10_(); } else {}
try { rule11_(); } else {}
try { rule12_(); } else {}
try { rule13_(); } else {}
try { rule14_(); } else {}
try { rule15_(); } else {}
try { rule16_(); } else {}
try { rule17a_(); } else {}
try { rule17b_(); } else {}
try { rule17c_(); } else {}
try { rule17d_(); } else {}
try { rule18_(); } else {}
try { rule19_(); } else {}
try { rule20_(); } else {}
try { rule21_(); } else {}
try { rule22a_(); } else {}
try { rule22b_(); } else {}
try { rule22c_(); } else {}

```

TC-10

```

k
CreateSelfEmploymentIncome(60000.00, 20000.00);
CreateDividendIncome(1000.00);
CreateEmploymentIncome(60000.00);
CreateSavingsInterestIncome(1500.00);
inputParams(3000.00, 500.00, false, false);
>
try { rule01_(); } else {}
try { rule02_(); } else {}
try { rule03_(); } else {}
try { rule04_(); } else {}
try { rule05_(); } else {}
try { rule06_(); } else {}
try { rule07a_(); } else {}
try { rule07b_(); } else {}
try { rule08_(); } else {}
try { rule09_(); } else {}
try { rule10_(); } else {}
try { rule11_(); } else {}
try { rule12_(); } else {}
try { rule13_(); } else {}
try { rule14_(); } else {}
try { rule15_(); } else {}
try { rule16_(); } else {}
try { rule17a_(); } else {}
try { rule17b_(); } else {}
try { rule17c_(); } else {}
try { rule17d_(); } else {}
try { rule18_(); } else {}
try { rule19_(); } else {}
try { rule20_(); } else {}
try { rule21_(); } else {}
try { rule22a_(); } else {}
try { rule22b_(); } else {}
try { rule22c_(); } else {}

```

TC-11

```

<
  CreateSelfEmploymentIncome(0.00, 0.00);
  CreateDividendIncome(0.00);
  CreateEmploymentIncome(0.00);
  CreateSavingsInterestIncome(0.00);
  inputParams(0.00, 0.00, false, false);
>
try { rule01_(); } else {}
try { rule02_(); } else {}
try { rule03_(); } else {}
try { rule04_(); } else {}
try { rule05_(); } else {}
try { rule06_(); } else {}
try { rule07a_(); } else {}
try { rule07b_(); } else {}
try { rule08_(); } else {}
try { rule09_(); } else {}
try { rule10_(); } else {}
try { rule11_(); } else {}
try { rule12_(); } else {}
try { rule13_(); } else {}
try { rule14_(); } else {}
try { rule15_(); } else {}
try { rule16_(); } else {}
try { rule17a_(); } else {}
try { rule17b_(); } else {}
try { rule17c_(); } else {}
try { rule17d_(); } else {}
try { rule18_(); } else {}
try { rule19_(); } else {}
try { rule20_(); } else {}
try { rule21_(); } else {}
try { rule22a_(); } else {}
try { rule22b_(); } else {}
try { rule22c_(); } else {}

```

TC-12

```

<
  CreateSelfEmploymentIncome(10000.00, 2000.00);
  CreateDividendIncome(1000.00);
  CreateEmploymentIncome(40000.00);
  CreateSavingsInterestIncome(500.00);
  inputParams(10000.00, 2000.00, true, true);
>
try { rule01_(); } else {}
try { rule02_(); } else {}
try { rule03_(); } else {}
try { rule04_(); } else {}
try { rule05_(); } else {}
try { rule06_(); } else {}
try { rule07a_(); } else {}
try { rule07b_(); } else {}
try { rule08_(); } else {}
try { rule09_(); } else {}
try { rule10_(); } else {}
try { rule11_(); } else {}
try { rule12_(); } else {}
try { rule13_(); } else {}
try { rule14_(); } else {}
try { rule15_(); } else {}
try { rule16_(); } else {}
try { rule17a_(); } else {}
try { rule17b_(); } else {}
try { rule17c_(); } else {}
try { rule17d_(); } else {}
try { rule18_(); } else {}
try { rule19_(); } else {}
try { rule20_(); } else {}
try { rule21_(); } else {}
try { rule22a_(); } else {}
try { rule22b_(); } else {}
try { rule22c_(); } else {}

```


TC-13

```

k
CreateSelfEmploymentIncome(0.00, 0.00);
CreateDividendIncome(0.00);
CreateEmploymentIncome(62840.00);
CreateSavingsInterestIncome(0.00);
inputParams(0.00, 0.00, false, false);
>
try { rule01_(); } else {}
try { rule02_(); } else {}
try { rule03_(); } else {}
try { rule04_(); } else {}
try { rule05_(); } else {}
try { rule06_(); } else {}
try { rule07a_(); } else {}
try { rule07b_(); } else {}
try { rule08_(); } else {}
try { rule09_(); } else {}
try { rule10_(); } else {}
try { rule11_(); } else {}
try { rule12_(); } else {}
try { rule13_(); } else {}
try { rule14_(); } else {}
try { rule15_(); } else {}
try { rule16_(); } else {}
try { rule17a_(); } else {}
try { rule17b_(); } else {}
try { rule17c_(); } else {}
try { rule17d_(); } else {}
try { rule18_(); } else {}
try { rule19_(); } else {}
try { rule20_(); } else {}
try { rule21_(); } else {}
try { rule22a_(); } else {}
try { rule22b_(); } else {}
try { rule22c_(); } else {}

```

TC14

```

<
CreateSelfEmploymentIncome(0.00, 0.00);
CreateDividendIncome(1000.00);
CreateEmploymentIncome(0.00);
CreateSavingsInterestIncome(0.00);
inputParams(0.00, 0.00, false, false);
>
try { rule01_(); } else {}
try { rule02_(); } else {}
try { rule03_(); } else {}
try { rule04_(); } else {}
try { rule05_(); } else {}
try { rule06_(); } else {}
try { rule07a_(); } else {}
try { rule07b_(); } else {}
try { rule08_(); } else {}
try { rule09_(); } else {}
try { rule10_(); } else {}
try { rule11_(); } else {}
try { rule12_(); } else {}
try { rule13_(); } else {}
try { rule14_(); } else {}
try { rule15_(); } else {}
try { rule16_(); } else {}
try { rule17a_(); } else {}
try { rule17b_(); } else {}
try { rule17c_(); } else {}
try { rule17d_(); } else {}
try { rule18_(); } else {}
try { rule19_(); } else {}
try { rule20_(); } else {}
try { rule21_(); } else {}
try { rule22a_(); } else {}
try { rule22b_(); } else {}
try { rule22c_(); } else {}

```

TC-15

```

<
  CreateSelfEmploymentIncome(0.00, 0.00);
  CreateDividendIncome(0.00);
  CreateEmploymentIncome(0.00);
  CreateSavingsInterestIncome(2000.00);
  inputParams(0.00, 0.00, false, false);
>
try { rule01_(); } else {}
try { rule02_(); } else {}
try { rule03_(); } else {}
try { rule04_(); } else {}
try { rule05_(); } else {}
try { rule06_(); } else {}
try { rule07a_(); } else {}
try { rule07b_(); } else {}
try { rule08_(); } else {}
try { rule09_(); } else {}
try { rule10_(); } else {}
try { rule11_(); } else {}
try { rule12_(); } else {}
try { rule13_(); } else {}
try { rule14_(); } else {}
try { rule15_(); } else {}
try { rule16_(); } else {}
try { rule17a_(); } else {}
try { rule17b_(); } else {}
try { rule17c_(); } else {}
try { rule17d_(); } else {}
try { rule18_(); } else {}
try { rule19_(); } else {}
try { rule20_(); } else {}
try { rule21_(); } else {}
try { rule22a_(); } else {}
try { rule22b_(); } else {}
try { rule22c_(); } else {}

```

Appendix 6 – Test Cases Results

TC-1

Experimental Comparison

Compare multiple AI models side-by-side with identical inputs

Calculator
Experimental BETA

SELF-EMPLOYED REVENUE *

£ 0

SELF-EMPLOYED EXPENSES *

£ 0

EMPLOYMENT REVENUE *

£ 30000

SAVINGS REVENUE *

£ 0

DIVIDEND REVENUE *

£ 0

PENSION CONTRIBUTIONS *

£ 0

CHARITABLE DONATIONS *

£ 0

☐ Relief at Source

☐ Registered Blind

⚡ Run All Models

Completed.

Model Comparison Results

MODEL	AI Tax	Model A	Model B	Groove
personal_allowance	£12,570.00	£12,570.00	£12,570.00	£12,570.00
adjusted_net_income	£30,000.00	£30,000.00	£30,000.00	£30,000.00
	0	0	0	0
total_income	£30,000.00	£30,000.00	£30,000.00	£30,000.00
	0	0	0	0
net_relevant_earnings	£30,000.00	£30,000.00	£30,000.00	£30,000.00
	0	0	0	0
personal_savings_allowance	£1,000.00	0	£1,000.00	£1,000.00
taxable_income	£17,430.00	£17,430.00	£17,430.00	£17,430.00
tax	£3,486.00	£3,486.00	£3,486.00	£3,486.00

TC-2

Experimental Comparison

Compare multiple AI models side-by-side with identical inputs

Calculator
Experimental
BETA

SELF-EMPLOYED REVENUE *

£ 50000

EMPLOYMENT REVENUE *

£ 0

DIVIDEND REVENUE *

£ 0

CHARITABLE DONATIONS *

£ 0

☐ Registered Blind

SELF-EMPLOYED EXPENSES *

£ 10000

SAVINGS REVENUE *

£ 0

PENSION CONTRIBUTIONS *

£ 0

☐ Relief at Source

⚡ Run All Models

Completed.

Model Comparison Results

MODEL	AI Tax	Model A	Model B	Groove
personal_allowance	£12,570.00	£12,570.00	£12,570.00	£12,570.00
adjusted_net_income	£50,000.00	£40,000.00	£40,000.00	£40,000.00
total_income	£50,000.00	£40,000.00	£40,000.00	£40,000.00
net_relevant_earnings	£50,000.00	£40,000.00	£40,000.00	£40,000.00
personal_savings_allowance	0	0	£1,000.00	£1,000.00
taxable_income	£37,430.00	£27,430.00	£27,430.00	£27,430.00
tax	£7,486.00	£5,486.00	£5,486.00	£5,486.00

TC-3

Compare multiple AI models side-by-side with identical inputs

Calculator Experimental **BETA**

SELF-EMPLOYED REVENUE *

£ 0

SELF-EMPLOYED EXPENSES *

£ 0

EMPLOYMENT REVENUE *

£ 20000

SAVINGS REVENUE *

£ 800

DIVIDEND REVENUE *

£ 0

PENSION CONTRIBUTIONS *

£ 0

CHARITABLE DONATIONS *

£ 0

☐ Relief at Source

☐ Registered Blind

⚡ Run All Models

Completed.

Model Comparison Results

MODEL	AI Tax	Model A	Model B	Groove
personal_allowance	£12,570.00	£12,570.00	£12,570.00	£12,570.00
adjusted_net_income	£20,800.00	£20,800.00	£20,800.00	£20,800.00
total_income	£20,800.00	£20,800.00	£20,800.00	£20,800.00
amount_in_starting_savings	0	0	£800.00	0
dividend_allowance	£2,000.00	0	0	0
net_relevant_earnings	£20,000.00	£20,000.00	£20,000.00	£20,000.00
personal_savings_allowance	£1,000.00	£800.00	£1,000.00	£1,000.00
taxable_income	£8,230.00	£8,230.00	£8,230.00	£8,230.00
tax	£1,646.00	£1,486.00	£1,486.00	£1,646.00

TC-4

Experimental Comparison

Compare multiple AI models side-by-side with identical inputs

Calculator
Experimental
BETA

SELF-EMPLOYED REVENUE *

£ 0

EMPLOYMENT REVENUE *

£ 20000

DIVIDEND REVENUE *

£ 0

CHARITABLE DONATIONS *

£ 0

☐ Registered Blind

SELF-EMPLOYED EXPENSES *

£ 0

SAVINGS REVENUE *

£ 1200

PENSION CONTRIBUTIONS *

£ 0

☐ Relief at Source

⚡ Run All Models

Completed.

Model Comparison Results

MODEL	AI Tax	Model A	Model B	Groove
personal_allowance	£12,570.00	£12,570.00	£12,570.00	£12,570.00
adjusted_net_income	£21,200.00	£21,200.00	£21,200.00	£21,200.00
total_income	£21,200.00	£21,200.00	£21,200.00	£21,200.00
amount_in_starting_savings	0	0	£1,200.00	0
dividend_allowance	£2,000.00	0	0	0
net_relevant_earnings	£20,000.00	£20,000.00	£20,000.00	£20,000.00
personal_savings_allowance	£1,000.00	£1,000.00	£1,000.00	£1,000.00
taxable_income	£8,640.00	£8,630.00	£8,630.00	£8,630.00
tax	£1,728.00	£1,526.00	£1,486.00	£1,726.00

TC-5

Experimental Comparison

Compare multiple AI models side-by-side with identical inputs

Calculator
Experimental
BETA

SELF-EMPLOYED REVENUE *

£ 0

EMPLOYMENT REVENUE *

£ 0

DIVIDEND REVENUE *

£ 2000

CHARITABLE DONATIONS *

£ 0

☐ Registered Blind

SELF-EMPLOYED EXPENSES *

£ 0

SAVINGS REVENUE *

£ 0

PENSION CONTRIBUTIONS *

£ 0

☐ Relief at Source

⚡ Run All Models

Completed.

Model Comparison Results

MODEL	AI Tax	Model A	Model B	Groove
personal_allowance	£12,570.00	£12,570.00	£12,570.00	£12,570.00
adjusted_net_income	£2,000.00	£2,000.00	£2,000.00	£2,000.00
total_income	£2,000.00	£2,000.00	£2,000.00	£2,000.00
dividend_allowance	£2,000.00	0	£500.00	£500.00

TC-6

Experimental Comparison

Compare multiple AI models side-by-side with identical inputs

Calculator
Experimental BETA

SELF-EMPLOYED REVENUE *

£ 150000

SELF-EMPLOYED EXPENSES *

£ 0

EMPLOYMENT REVENUE *

£ 0

SAVINGS REVENUE *

£ 0

DIVIDEND REVENUE *

£ 0

PENSION CONTRIBUTIONS *

£ 0

CHARITABLE DONATIONS *

£ 0

☐ Relief at Source

☐ Registered Blind

⚡ Run All Models

Completed.

Model Comparison Results

MODEL	AI Tax	Model A	Model B	Groove
personal_allowance	£12,570.00	0	0	0
adjusted_net_income	£150,000.0	£150,000.0	£150,000.0	£150,000.0
	0	0	0	0
total_income	£150,000.0	£150,000.0	£150,000.0	£150,000.0
	0	0	0	0
net_relevant_earnings	£150,000.0	£150,000.0	£150,000.0	£150,000.0
	0	0	0	0
personal_savings_allowance	£1,000.00	0	0	0
taxable_income	£137,430.0	£150,000.0	£150,000.0	£150,000.0
	0	0	0	0
tax	£40,000.0	£54,331.5	£53,703.0	£53,703.0
	0	0	0	0

TC-7

Experimental Comparison

Compare multiple AI models side-by-side with identical inputs

Calculator
Experimental
BETA

SELF-EMPLOYED REVENUE *

£ 0

EMPLOYMENT REVENUE *

£ 40000

DIVIDEND REVENUE *

£ 0

CHARITABLE DONATIONS *

£ 1000

☐ Registered Blind

SELF-EMPLOYED EXPENSES *

£ 0

SAVINGS REVENUE *

£ 0

PENSION CONTRIBUTIONS *

£ 0

☐ Relief at Source

⚡ Run All Models

Completed.

Model Comparison Results

MODEL	AI Tax	Model A	Model B	Groove
personal_allowance	£12,570.00	£12,570.00	£12,570.00	£12,570.00
adjusted_net_income	£37,750.00	£38,750.00	£38,750.00	£38,750.00
total_income	£40,000.00	£40,000.00	£40,000.00	£40,000.00
gift_aid_relief	£250.00	£1,250.00	£1,250.00	£1,250.00
net_relevant_earnings	£40,000.00	£40,000.00	£40,000.00	£40,000.00
personal_savings_allowance	£1,000.00	0	£1,000.00	£1,000.00
taxable_income	£25,180.00	£27,430.00	£26,180.00	£26,180.00
tax	£5,026.00	£5,486.00	£5,236.00	£5,236.00

TC-8

Experimental Comparison

Compare multiple AI models side-by-side with identical inputs

Calculator
Experimental
BETA

SELF-EMPLOYED REVENUE *

£ 0

EMPLOYMENT REVENUE *

£ 30000

DIVIDEND REVENUE *

£ 0

CHARITABLE DONATIONS *

£ 0

☒ Registered Blind

SELF-EMPLOYED EXPENSES *

£ 0

SAVINGS REVENUE *

£ 0

PENSION CONTRIBUTIONS *

£ 0

☐ Relief at Source

⚡ Run All Models

Completed.

Model Comparison Results

MODEL	AI Tax	Model A	Model B	Groove
personal_allowance	£12,570.00	£12,570.00	£12,570.00	£12,570.00
adjusted_net_income	£30,000.00	£30,000.00	£30,000.00	£30,000.00
total_income	£30,000.00	£30,000.00	£30,000.00	£30,000.00
blind_persons_allowance	£2,500.00	0	£3,130.00	£3,130.00
net_relevant_earnings	£30,000.00	£30,000.00	£30,000.00	£30,000.00
personal_savings_allowance	£1,000.00	0	£1,000.00	£1,000.00
taxable_income	£15,030.00	£17,430.00	£14,300.00	£14,300.00
tax	£3,006.00	£3,486.00	£2,860.00	£2,860.00

TC-9

Experimental Comparison

Compare multiple AI models side-by-side with identical inputs

Calculator
Experimental
BETA

SELF-EMPLOYED REVENUE *

£ 0

EMPLOYMENT REVENUE *

£ 50000

DIVIDEND REVENUE *

£ 0

CHARITABLE DONATIONS *

£ 0

☐ Registered Blind

SELF-EMPLOYED EXPENSES *

£ 0

SAVINGS REVENUE *

£ 0.00

PENSION CONTRIBUTIONS *

£ 5000

☒ Relief at Source

⚡ Run All Models

Completed.

Model Comparison Results

MODEL	AI Tax	Model A	Model B	Groove
personal_allowance	£12,570.00	£12,570.00	£12,570.00	£12,570.00
adjusted_net_income	£43,750.00	£43,750.00	£43,750.00	£43,750.00
total_income	£50,000.00	£50,000.00	£50,000.00	£50,000.00
net_relevant_earnings	£45,000.00	£50,000.00	£50,000.00	£50,000.00
pension_relief	£1,250.00	£1,250.00	£6,250.00	£6,250.00
personal_savings_allowance	0	0	£1,000.00	£1,000.00
taxable_income	£31,180.00	£37,430.00	£31,180.00	£31,180.00
tax	£6,236.00	£7,486.00	£6,236.00	£6,236.00

TC-10

Experimental Comparison

Compare multiple AI models side-by-side with identical inputs

Calculator
Experimental BETA

SELF-EMPLOYED REVENUE *

£ 20000

SELF-EMPLOYED EXPENSES *

£ 0

EMPLOYMENT REVENUE *

£ 60000

SAVINGS REVENUE *

£ 1500

DIVIDEND REVENUE *

£ 1000

PENSION CONTRIBUTIONS *

£ 3000

CHARITABLE DONATIONS *

£ 500

☐ Relief at Source

☐ Registered Blind

⚡ Run All Models

Completed.

Model Comparison Results

MODEL	AI Tax	Model A	Model B	Groove
personal_allowance	£12,570.00	£12,570.00	£12,570.00	£12,570.00
adjusted_net_income	£57,000.00	£81,875.00	£78,875.00	£78,875.00
total_income	£83,500.00	£82,500.00	£82,500.00	£82,500.00
dividend_allowance	£2,000.00	£500.00	£500.00	£500.00
gift_aid_relief	0	£625.00	£625.00	£625.00
net_relevant_earnings	£57,000.00	£80,000.00	£80,000.00	£80,000.00
pension_relief	0	0	£3,000.00	£3,000.00
personal_savings_allowance	0	£500.00	£500.00	£500.00
taxable_income	£44,430.00	£69,930.00	£66,305.00	£66,305.00
tax	£8,836.00	£20,000.75	£18,982.00	£18,982.00

TC-11

Experimental Comparison

Compare multiple AI models side-by-side with identical inputs

Calculator Experimental **BETA**

SELF-EMPLOYED REVENUE *

£ 0

SELF-EMPLOYED EXPENSES *

£ 0

EMPLOYMENT REVENUE *

£ 0

SAVINGS REVENUE *

£ 0

DIVIDEND REVENUE *

£ 0

PENSION CONTRIBUTIONS *

£ 0

CHARITABLE DONATIONS *

£ 0

☐ Relief at Source

☐ Registered Blind

⚡ Run All Models

Completed.

Model Comparison Results

MODEL	AI Tax	Model A	Model B	Groove
personal_all	0	£12,570.0	£12,570.0	£12,570.0
owance		0	0	0

TC-12

Experimental Comparison

Compare multiple AI models side-by-side with identical inputs

Calculator
Experimental
BETA

SELF-EMPLOYED REVENUE *

£ 2000

EMPLOYMENT REVENUE *

£ 40000

DIVIDEND REVENUE *

£ 1000

CHARITABLE DONATIONS *

£ 2000

☒ Registered Blind

SELF-EMPLOYED EXPENSES *

£ 0

SAVINGS REVENUE *

£ 500

PENSION CONTRIBUTIONS *

£ 10000

☒ Relief at Source

⚡ Run All Models

Completed.

Model Comparison Results

MODEL	AI Tax	Model A	Model B	Groove
personal_allowance	£12,570.00	£12,570.00	£12,570.00	£12,570.00
adjusted_net_income	£36,500.00	£28,500.00	£28,500.00	£28,500.00
	0	0	0	0
total_income	£44,000.00	£43,500.00	£43,500.00	£43,500.00
	0	0	0	0
blind_persons_allowance	£2,500.00	0	£3,130.00	£3,130.00
dividend_allowance	£2,000.00	£500.00	£500.00	£500.00
gift_aid_relief	£500.00	£2,500.00	£2,500.00	£2,500.00
net_relevant_earnings	£30,000.00	£42,000.00	£42,000.00	£42,000.00
	0	0	0	0
pension_relief	£2,500.00	£2,500.00	£12,500.00	£12,500.00
personal_savings_allowance	0	£500.00	£1,000.00	£1,000.00
taxable_income	£21,530.00	£30,930.00	£12,800.00	£12,800.00
		0		
tax	£4,306.00	£5,929.75	£2,560.00	£2,560.00

TC-13

Experimental Comparison

Compare multiple AI models side-by-side with identical inputs

Calculator
Experimental
BETA

SELF-EMPLOYED REVENUE *

£ 0

EMPLOYMENT REVENUE *

£ 62840

DIVIDEND REVENUE *

£ 0

CHARITABLE DONATIONS *

£ 0

☐ Registered Blind

SELF-EMPLOYED EXPENSES *

£ 0

SAVINGS REVENUE *

£ 0

PENSION CONTRIBUTIONS *

£ 0

☐ Relief at Source

⚡ Run All Models

Completed.

Model Comparison Results

MODEL	AI Tax	Model A	Model B	Groove
personal_allowance	£12,570.00	£12,570.00	£12,570.00	£12,570.00
adjusted_net_income	£62,840.00	£62,840.00	£62,840.00	£62,840.00
	0	0	0	0
total_income	£62,840.00	£62,840.00	£62,840.00	£62,840.00
	0	0	0	0
net_relevant_earnings	£62,840.00	£62,840.00	£62,840.00	£62,840.00
	0	0	0	0
personal_savings_allowance	£1,000.00	0	£500.00	£500.00
taxable_income	£50,270.00	£50,270.00	£50,270.00	£50,270.00
	0	0	0	0
tax	£10,294.00	£12,568.00	£12,568.00	£12,568.00
	0	0	0	0

TC-14

Experimental Comparison

Compare multiple AI models side-by-side with identical inputs

Calculator
Experimental
BETA

SELF-EMPLOYED REVENUE *

£ 0

SELF-EMPLOYED EXPENSES *

£ 0

EMPLOYMENT REVENUE *

£ 0

SAVINGS REVENUE *

£ 0

DIVIDEND REVENUE *

£ 1000

PENSION CONTRIBUTIONS *

£ 0

CHARITABLE DONATIONS *

£ 0

☐ Relief at Source

☐ Registered Blind

⚡ Run All Models

Completed.

Model Comparison Results

MODEL	AI Tax	Model A	Model B	Groove
personal_allowance	£12,570.00	£12,570.00	£12,570.00	£12,570.00
adjusted_net_income	£1,000.00	£1,000.00	£1,000.00	£1,000.00
total_income	£1,000.00	£1,000.00	£1,000.00	£1,000.00
dividend_allowance	£2,000.00	0	£500.00	£500.00

TC-15

Experimental Comparison

Compare multiple AI models side-by-side with identical inputs

Calculator
Experimental
BETA

SELF-EMPLOYED REVENUE *

£ 0

SELF-EMPLOYED EXPENSES *

£ 0

EMPLOYMENT REVENUE *

£ 0

SAVINGS REVENUE *

£ 2000

DIVIDEND REVENUE *

£ 0

PENSION CONTRIBUTIONS *

£ 0

CHARITABLE DONATIONS *

£ 0

☐ Relief at Source

☐ Registered Blind

Model Comparison Results

MODEL	AI Tax	Model A	Model B	Groove
personal_allowance	£12,570.00	£12,570.00	£12,570.00	£12,570.00
adjusted_net_income	£2,000.00	£2,000.00	£2,000.00	£2,000.00
total_income	£2,000.00	£2,000.00	£2,000.00	£2,000.00
amount_in_starting_savings	0	0	£2,000.00	£2,000.00
personal_savings_allowance	£1,000.00	0	0	0

⚡ Run All Models

Completed.